AgriTech Project Documentation

Complete source-grounded project guide

Combined from every Markdown file in PROJECT_DOCUMENTATION

15 Markdown files · beginner guide, architecture, source inventory, API routes, data models, workflows, setup, audit and interview preparation

Contents

- README
- 01 Project Overview
- 02 Project Structure
- 03 System Architecture
- 04 Frontend Documentation
- 05 Backend Documentation
- 06 Database Documentation
- 07 Authentication Authorization
- 08 API Documentation
- 09 Application Workflows
- 10 Setup and Installation
- 11 Security and Code Audit
- 12 Deployment and DevOps
- 13 Troubleshooting
- 14 Interview Preparation

README

AgriTech Project Documentation

This folder contains 14 requested guides grounded in source inspection. Start with `01_Project_Overview.md`, then `03_System_Architecture.md`, `09_Application_Workflows.md`, and `14_Interview_Preparation.md`.

Scope and caveats

- Route appendix contains 444 route decorators scanned across Python files and identifies source file/line. Blueprint prefixes may compose; use runtime `app.url_map` as final authority.
- Model inventory contains 184 ORM model classes detected in `backend/models/`. It summarizes declared columns; inspect each model and migration for full schema details.
- Repository scan counted 55082 source/config/document-like files after exclusions. Generated dependencies, notebooks and model binaries were not expanded line by line; see `02_Project_Structure.md`.
- Private `.env` values and runtime database contents were not read or included.
- No tests, dependency scanner, penetration test, or deployment was run during documentation generation.
- Findings are qualified by source evidence and environment; modules and routes may need credentials, model files, database state or queue services.

01 Project Overview

Project Overview

What AgriTech is

AgriTech is a multi-page agricultural web application. Its main server is Python/Flask (`app.py`). The browser uses HTML, CSS and JavaScript. The server connects agricultural pages, model inference, database-backed services, authentication, background jobs and external integrations. The repository contains a wide set of feature modules; their presence in source does not prove every one is operational in a deployed environment.

Purpose

The product brings crop decision support and related farmer services into one web entry point. Main areas include crop recommendation, fertilizer advice, yield estimation, plant disease classification/advisory, weather and climate information, irrigation, crop rotation, market prices, marketplace, community and operational services. Advice should be treated as decision support and verified locally.

Technology stack (verified in source/config)

- Frontend: standalone HTML pages, CSS stylesheets, browser JavaScript; no main React/Vue/Angular application found. `package.json` describes a separate Express chatbot server.
- Backend: Python 3 application, Flask, Flask Blueprints; optional Node.js/Express service in `server.js`.
- Database: Flask-SQLAlchemy ORM and Flask-Migrate/Alembic; development config defaults to SQLite; `DATABASE_URL` can change it.
- Authentication: project has more than one auth path: `backend.api.v1.auth` routes and legacy `auth_utils.py` JWT decorators. Some API handlers use Flask-JWT-Extended. Inspect each endpoint's decorator; do not assume uniform protection. Password hashing uses Werkzeug in the User model.
- Async/realtime: Celery, Redis configuration, Flask-SocketIO event handlers. Workers/brokers must actually run.
- ML/data: PyTorch, TensorFlow/Keras, scikit-learn, XGBoost/CatBoost, pandas/numpy/joblib and image tooling appear in requirements and modules. Specific models are feature-specific.
- Integrations: Firebase configuration, Gemini, SMTP, weather/FX/translation APIs, optional S3 and messaging credentials are represented in configuration. Availability depends on credentials/network.
- Tests: pytest configuration and Python test directories; security-related tests also exist.
- Deployment: Dockerfile and docker-compose for Flask, Redis, MailHog, Celery. GitHub Actions workflows are mostly repository automation; no verified production hosting target found.

Entry points

- Main Flask server: `app.py`; README instructs `python app.py`.
- Optional Node server: `server.js`; `npm start` launches it. It is not the primary Flask app.
- Main homepage: `/` serves root `index.html`.

Feature status rules

The `ENABLE_*` settings are defined in `backend/config/settings.py`, but code search found no feature consumer beyond their definitions; treat them as currently non-operative configuration. Disease API Blueprints are registered under API v1; the old `disease_prediction.routes` import in `app.py` is commented and the file was not found. Some routes require trained model artifacts, external credentials, queue services or DB models.

Repository facts

This scan found 822 files after excluding virtual environment, dependency, runtime database/log/temp, notebook, and large model artifacts. The repository has many feature directories and duplicate/legacy interfaces. Use the folder guide and route appendix to navigate; this documentation prioritizes application-owned code over generated dependencies and data artifacts.

02 Project Structure

Project Structure

Top-level directory inventory

- `.github/` — 17 tracked-like files in this directory tree.
- `.vscode/` — 2 tracked-like files in this directory tree.
- `AgriBot_folder/` — 9 tracked-like files in this directory tree.
- `AgriTech/` — 0 tracked-like files in this directory tree.
- `AgriTech_Project_Report/` — 0 tracked-like files in this directory tree.
- `assets/` — 1 tracked-like files in this directory tree.
- `backend/` — 381 tracked-like files in this directory tree.
- `Community/` — 4 tracked-like files in this directory tree.
- `Crop Yield Prediction/` — 44 tracked-like files in this directory tree.
- `Crop_Planning/` — 6 tracked-like files in this directory tree.
- `Crop_Prices_Tracker/` — 3 tracked-like files in this directory tree.
- `crop_recommendation/` — 13 tracked-like files in this directory tree.
- `css/` — 7 tracked-like files in this directory tree.
- `Disease prediction/` — 17995 tracked-like files in this directory tree.
- `disease_prediction/` — 17982 tracked-like files in this directory tree.
- `docs/` — 9 tracked-like files in this directory tree.
- `domain/` — 8 tracked-like files in this directory tree.
- `examples/` — 19 tracked-like files in this directory tree.
- `ExpenseFlow/` — 0 tracked-like files in this directory tree.
- `Fertiliser Recommendation System/` — 10 tracked-like files in this directory tree.
- `Forum/` — 3 tracked-like files in this directory tree.
- `frontend/` — 1 tracked-like files in this directory tree.
- `Gov_schemes/` — 3 tracked-like files in this directory tree.
- `images/` — 34 tracked-like files in this directory tree.
- `javascript/` — 1 tracked-like files in this directory tree.
- `js/` — 14 tracked-like files in this directory tree.
- `Labour_Alerts/` — 4 tracked-like files in this directory tree.
- `locals/` — 4 tracked-like files in this directory tree.
- `migrations/` — 8 tracked-like files in this directory tree.
- `Plant Disease Detection/` — 41 tracked-like files in this directory tree.
- `Plant Seedlings Classification/` — 22 tracked-like files in this directory tree.
- `scripts/` — 1 tracked-like files in this directory tree.
- `server/` — 15 tracked-like files in this directory tree.
- `Soil Classification Model/` — 4 tracked-like files in this directory tree.
- `Soil Classifier CNN/` — 162 tracked-like files in this directory tree.
- `spatial_analytics/` — 4 tracked-like files in this directory tree.
- `src/` — 8 tracked-like files in this directory tree.

- `static/` — 3 tracked-like files in this directory tree.
- `styles/` — 1 tracked-like files in this directory tree.
- `templates/` — 2 tracked-like files in this directory tree.
- `tests/` — 13 tracked-like files in this directory tree.
- `tomato disease detection/` — 17985 tracked-like files in this directory tree.
- `translations/` — 2 tracked-like files in this directory tree.
- `unlock mechanism/` — 1 tracked-like files in this directory tree.

Important files

- `app.py` — Flask app factory-like bootstrap at module scope: configuration, extension setup, Blueprint registration, direct routes, error handlers and Socket.IO startup.
- `backend/` — primary backend packages: API modules, auth, config, docs, extensions, middleware, models, ML, monitoring, schemas, services, sockets, tasks, tests and utilities.
- `templates/` — Flask-rendered templates, including the yield prediction page.
- Root `*.html`, `*.css`, `*.js` — multi-page web UI and feature scripts.
- `requirements.txt` — Python dependency declarations. It includes application, ML, testing and utility packages.
- `package.json`, `package-lock.json` — optional Node/Express chatbot service and npm dependency lock.
- `server.js` — optional Node server entry point.
- `migrations/` — Alembic/Flask-Migrate revisions.
- `docker-compose.yml`, `Dockerfile` — local multi-service container setup.
- `.env.example` — variable names/placeholders; actual `.env` is private and intentionally not read for documentation.
- `.github/workflows/` — issue/PR automation; workflows should not be mistaken for app deployment.
- `README.md` — setup and high-level architecture, checked against source.
- `pytest.ini`, `tests/`, `backend/tests/` — test configuration and suites.

Reading strategy

Start at `app.py`, follow its `register_blueprint` calls into `backend/api/v1/__init__.py`, then follow a route into the feature service and models. For browser journeys, follow a page's `<script>` reference and search JavaScript for `fetch`, form action, or URL. A Blueprint-local route prefix is combined with registration prefixes; runtime `app.url_map` is the definitive path list.

Duplication and legacy areas

The project contains similarly named root pages and feature directories, legacy URLs, standalone model/training notebooks, and both legacy and versioned API modules. Some model scripts are experiments, not loaded production paths. Confirm imports from `app.py` before assuming a file is live.

Complete repository file index

The list below covers the repository files included in the Git inventory at documentation time. It is a navigation index, not a claim that every file is imported at runtime. Files under notebooks, datasets and model directories are source/training artifacts; confirm runtime imports before treating them as active.

`(repository root)`

- `.dockerignore`
- `.env.example`
- `.eslintrc.json`
- `.flake8`
- `.git_diff.txt`
- `.git_status.txt`

.github

- .github/ISSUE_TEMPLATE/bug_report.yml
- .github/ISSUE_TEMPLATE/config.yml
- .github/ISSUE_TEMPLATE/documentation_update.md
- .github/ISSUE_TEMPLATE/enhancement_request.md
- .github/ISSUE_TEMPLATE/feature_request.yml
- .github/ISSUE_TEMPLATE/other.md
- .github/ISSUE_TEMPLATE/refactor_request.md
- .github/pull_request_template.md
- .github/workflows/.stylelintrc.json
- .github/workflows/assign-pr-author-on-merge.yml
- .github/workflows/auto-label-issues.yml
- .github/workflows/auto-label-pr.yml
- .github/workflows/issue-create-automate-message.yml
- .github/workflows/package.json
- .github/workflows/pr-create-automate-message.yml
- .github/workflows/pr-merge-automate-message.yml
- .github/workflows/python-app.yml

(repository root)

- .gitignore

.vscode

- .vscode/PythonImportHelper-v2-Completion.json
- .vscode/settings.json

AgriBot_folder

- AgriBot_folder/AgriBot.py
- AgriBot_folder/readme.md
- AgriBot_folder/requirement.txt

(repository root)

- AgriTech_Interview_and_Presentation_Guide.pdf
- AgriTech_Project_Explained.pdf
- AgriTech_Project_Report.docx
- AgriTech_Project_Report_Draft.html
- CHANGELOG.md
- CHANGES_SUMMARY.md
- CODE_OF_CONDUCT.md
- CONTRIBUTING.md
- CONTRIBUTION_SUMMARY.md

Community

- Community/app.py
- Community/static/chat.js
- Community/static/style.css

- Community/templates/index.html

Crop Yield Prediction

- Crop Yield Prediction/Modelbackup.ipynb
- Crop Yield Prediction/README.md
- Crop Yield Prediction/RUN_INSTRUCTIONS.md
- Crop Yield Prediction/catboost_info/catboost_training.json
- Crop Yield Prediction/catboost_info/learn/events.out.tfevents
- Crop Yield Prediction/catboost_info/learn_error.tsv
- Crop Yield Prediction/catboost_info/test/events.out.tfevents
- Crop Yield Prediction/catboost_info/test_error.tsv
- Crop Yield Prediction/catboost_info/time_left.tsv
- Crop Yield Prediction/cropYieldPrediction1.ipynb
- Crop Yield Prediction/crop_yield_app/app.py
- Crop Yield Prediction/crop_yield_app/models/metadata.json
- Crop Yield Prediction/crop_yield_app/static/js/validation.js
- Crop Yield Prediction/crop_yield_app/static/script.js
- Crop Yield Prediction/crop_yield_app/static/style.css
- Crop Yield Prediction/crop_yield_app/templates/index.html
- Crop Yield Prediction/crop_yield_app/templates/input.html
- Crop Yield Prediction/crop_yield_predictor.py
- Crop Yield Prediction/outputs/baseline_results.csv
- Crop Yield Prediction/outputs/feature_importance.csv
- Crop Yield Prediction/outputs/predictions.csv
- Crop Yield Prediction/predict.py
- Crop Yield Prediction/predict_yield_cli.py

Crop_Planning

- Crop_Planning/__init__.py
- Crop_Planning/app.py
- Crop_Planning/static/feedback.js
- Crop_Planning/static/script.js
- Crop_Planning/static/style.css
- Crop_Planning/templates/cropplan.html

Crop_Prices_Tracker

- Crop_Prices_Tracker/app.py
- Crop_Prices_Tracker/static/styles.css
- Crop_Prices_Tracker/templates/crop_price_tracker.html

Disease prediction

- Disease prediction/MODEL_SETUP.md
- Disease prediction/app.py
- Disease prediction/model.py
- Disease prediction/plant-disease-cnn-1 (1).ipynb

- Disease prediction/plant_disease_classes.json
- Disease prediction/plant_disease_model.pth
- Disease prediction/static/index.css
- Disease prediction/static/result.css
- Disease prediction/template/index.html
- Disease prediction/template/result.html
- Disease prediction/tomato disease detection/ReadMe.md
- Disease prediction/tomato disease detection/Tomato_Disease_Prediction.ipynb
- Disease prediction/utils.py

(repository root)

- Dockerfile

Fertiliser Recommendation System

- Fertiliser Recommendation System/README.md
- Fertiliser Recommendation System/app.py
- Fertiliser Recommendation System/improvement.md
- Fertiliser Recommendation System/src/model.py
- Fertiliser Recommendation System/src/train.py
- Fertiliser Recommendation System/src/utils.py

Forum

- Forum/app.py
- Forum/forum.html
- Forum/forum.js

Gov_schemes

- Gov_schemes/app_scheme.js
- Gov_schemes/index_scheme.html
- Gov_schemes/styles_scheme.css

(repository root)

- IMPLEMENTATION_SUMMARY.md
- IPM_README.md
- Implement Smart Crop Disease Detection & Advisory System
- LEDGER_IMPLEMENTATION_GUIDE.md
- LICENSE.md

Labour_Alerts

- Labour_Alerts/app.py
- Labour_Alerts/static/script.js
- Labour_Alerts/static/style.css
- Labour_Alerts/templates/labour.html

(repository root)

- MIT License.md
- PROPOSAL_FEATURE_ENHANCEMENTS.md

- PULL_REQUEST.md
- PULL_REQUEST_TEMPLATE.md

Plant Disease Detection

- Plant Disease Detection/Modelbackup.ipynb
- Plant Disease Detection/README.md
- Plant Disease Detection/plant-disease-model-complete.pth
- Plant Disease Detection/plant-disease-model.pth
- Plant Disease Detection/plant-disease.ipynb

Plant Seedlings Classification

- Plant Seedlings Classification/Dockerfile
- Plant Seedlings Classification/Model/Modelbackup.ipynb
- Plant Seedlings Classification/Model/Plant_seedlings_classification.ipynb
- Plant Seedlings Classification/README.md
- Plant Seedlings Classification/TUTORIAL.md
- Plant Seedlings Classification/api.py
- Plant Seedlings Classification/app.py
- Plant Seedlings Classification/docker-compose.yml
- Plant Seedlings Classification/requirements.txt
- Plant Seedlings Classification/src/__init__.py
- Plant Seedlings Classification/src/config.py
- Plant Seedlings Classification/src/data_loader.py
- Plant Seedlings Classification/src/evaluate.py
- Plant Seedlings Classification/src/models.py
- Plant Seedlings Classification/src/predict.py
- Plant Seedlings Classification/src/train.py
- Plant Seedlings Classification/src/visualize.py

(repository root)

- README.md
- SECURITY.md
- SECURITY_FIXES.md
- SUSTAINABILITY_README.md

Soil Classification Model

- Soil Classification Model/Modelbackp.ipynb
- Soil Classification Model/Readme.md
- Soil Classification Model/Soil Classification model.ipynb
- Soil Classification Model/main.py

Soil Classifier CNN

- Soil Classifier CNN/README.md
- Soil Classifier CNN/src/data_loader.py
- Soil Classifier CNN/src/model.py
- Soil Classifier CNN/src/train.py

- Soil Classifier CNN/src/utils.py

(repository root)

- TODO.md
- ThemeSelector.html
- about.css
- about.html
- admin.html
- agri_utils.py
- ai_disease.html
- ai_disease.js
- app.py
- audit_dashboard.html
- auth.js
- auth_utils.py
- babel.cfg

backend

- backend/__init__.py
- backend/api/__init__.py
- backend/api/ingestion.py
- backend/api/v1/__init__.py
- backend/api/v1/advisories.py
- backend/api/v1/ai_disease.py
- backend/api/v1/answers.py
- backend/api/v1/arbitrage.py
- backend/api/v1/assets.py
- backend/api/v1/audit.py
- backend/api/v1/auth.py
- backend/api/v1/barter.py
- backend/api/v1/biomass.py
- backend/api/v1/biosecurity.py
- backend/api/v1/bookings.py
- backend/api/v1/bot_webhook.py
- backend/api/v1/carbon.py
- backend/api/v1/carbon_v2.py
- backend/api/v1/circular.py
- backend/api/v1/climate_control.py
- backend/api/v1/config.py
- backend/api/v1/contributions.py
- backend/api/v1/credits.py
- backend/api/v1/crop_advisory.py
- backend/api/v1/dashboard.py
- backend/api/v1/diagnostics_v2.py

- backend/api/v1/disease.py
- backend/api/v1/energy_grid.py
- backend/api/v1/equipment.py
- backend/api/v1/farm_members.py
- backend/api/v1/farms.py
- backend/api/v1/files.py
- backend/api/v1/financials.py
- backend/api/v1/forum.py
- backend/api/v1/futures.py
- backend/api/v1/genomics.py
- backend/api/v1/gews.py
- backend/api/v1/government_schemes.py
- backend/api/v1/insurance.py
- backend/api/v1/insurance_portal.py
- backend/api/v1/irrigation.py
- backend/api/v1/irrigation_v2.py
- backend/api/v1/labor_management.py
- backend/api/v1/loan.py
- backend/api/v1/loan_repayment.py
- backend/api/v1/logistics.py
- backend/api/v1/logistics_portal.py
- backend/api/v1/machinery.py
- backend/api/v1/maintenance.py
- backend/api/v1/maintenance_v2.py
- backend/api/v1/market.py
- backend/api/v1/model_versioning.py
- backend/api/v1/notifications.py
- backend/api/v1/nutrient_api.py
- backend/api/v1/pools.py
- backend/api/v1/processing.py
- backend/api/v1/procurement.py
- backend/api/v1/questions.py
- backend/api/v1/risk.py
- backend/api/v1/schemes.py
- backend/api/v1/soil_analysis.py
- backend/api/v1/soil_analytics.py
- backend/api/v1/spatial_yield.py
- backend/api/v1/supply_chain.py
- backend/api/v1/sustainability.py
- backend/api/v1/tasks.py
- backend/api/v1/traceability.py
- backend/api/v1/translations.py

- backend/api/v1/transparency.py
- backend/api/v1/vaults.py
- backend/api/v1/vendors.py
- backend/api/v1/virulence.py
- backend/api/v1/warehouse.py
- backend/api/v1/weather.py
- backend/auth/__init__.py
- backend/auth/decorators.py
- backend/auth/jwt_utils.py
- backend/auth/routes.py
- backend/auth_utils.py
- backend/celery_app.py
- backend/config/__init__.py
- backend/config/celery_config.py
- backend/config/settings.py
- backend/config_old.py
- backend/database_config.py
- backend/docs/swagger.py
- backend/extensions/__init__.py
- backend/extensions/babel.py
- backend/extensions/cache.py
- backend/extensions/database.py
- backend/extensions/limiter.py
- backend/extensions/mail.py
- backend/extensions/socketio.py
- backend/middleware/audit.py
- backend/middleware/ledger_audit.py
- backend/migrations/init_insurance_db.py
- backend/ml_models/CHANGELOG.md
- backend/ml_models/README.md
- backend/ml_models/__init__.py
- backend/ml_models/crop_recommendation/active_version.json
- backend/ml_models/crop_recommendation/v1.0.0/metadata.json
- backend/ml_models/crop_recommendation/v1.1.0/metadata.json
- backend/ml_models/migrate_models.py
- backend/ml_models/model_manager.py
- backend/ml_models/model_registry.py
- backend/ml_models/price_forecast.py
- backend/ml_models/test_versioning.py
- backend/models/__init__.py
- backend/models/ai_diagnostics.py
- backend/models/alert.py

- backend/models/arbitrage.py
- backend/models/audit_log.py
- backend/models/autonomous_supply.py
- backend/models/barter.py
- backend/models/biomass.py
- backend/models/carbon_escrow.py
- backend/models/circular.py
- backend/models/climate.py
- backend/models/disease.py
- backend/models/energy_grid.py
- backend/models/equipment.py
- backend/models/farm.py
- backend/models/fertigation_v2.py
- backend/models/financials.py
- backend/models/forum.py
- backend/models/freight_v2.py
- backend/models/genomics.py
- backend/models/gews.py
- backend/models/global_trade.py
- backend/models/government_schemes.py
- backend/models/insurance.py
- backend/models/insurance_v2.py
- backend/models/iot_maintenance.py
- backend/models/irrigation.py
- backend/models/knowledge.py
- backend/models/labor.py
- backend/models/ledger.py
- backend/models/loan_request.py
- backend/models/loan_v2.py
- backend/models/logistics_v2.py
- backend/models/machinery.py
- backend/models/market.py
- backend/models/marketplace.py
- backend/models/media_payload.py
- backend/models/misc.py
- backend/models/precision_irrigation.py
- backend/models/prediction_history.py
- backend/models/processing.py
- backend/models/procurement.py
- backend/models/reliability_log.py
- backend/models/soil_health.py
- backend/models/spatial_yield.py

- backend/models/sustainability.py
- backend/models/token.py
- backend/models/traceability.py
- backend/models/transparency.py
- backend/models/user.py
- backend/models/virulence.py
- backend/models/warehouse.py
- backend/models/weather.py
- backend/monitoring/__init__.py
- backend/monitoring/health.py
- backend/monitoring/metrics.py
- backend/monitoring/routes.py
- backend/schemas/__init__.py
- backend/schemas/asset_schema.py
- backend/schemas/climate_schema.py
- backend/schemas/farm_schema.py
- backend/schemas/forum_schema.py
- backend/schemas/insurance_schema.py
- backend/schemas/irrigation_schema.py
- backend/schemas/knowledge_schema.py
- backend/schemas/labor_schema.py
- backend/schemas/loan_schema.py
- backend/schemas/logistics_schema.py
- backend/schemas/machinery_schema.py
- backend/schemas/pool_schema.py
- backend/schemas/processing_schema.py
- backend/schemas/rental_schema.py
- backend/schemas/soil_schema.py
- backend/schemas/warehouse_schema.py
- backend/schemas/weather_schema.py
- backend/services/__init__.py
- backend/services/advisory_engine.py
- backend/services/ai_disease_detection_service.py
- backend/services/ai_moderator.py
- backend/services/alert_registry.py
- backend/services/arbitrage_service.py
- backend/services/asset_service.py
- backend/services/audit_service.py
- backend/services/audit_workflow.py
- backend/services/auth_service.py
- backend/services/barter_service.py
- backend/services/batch_service.py

- backend/services/bot_advisory_service.py
- backend/services/carbon_calculator.py
- backend/services/carbon_escrow_service.py
- backend/services/carbon_sequestration_engine.py
- backend/services/carbon_service.py
- backend/services/climate_service.py
- backend/services/compliance_engine.py
- backend/services/crop_advisory_service.py
- backend/services/diagnostic_engine.py
- backend/services/email_service.py
- backend/services/energy_token_service.py
- backend/services/escrow_service.py
- backend/services/farm_analytics.py
- backend/services/farm_service.py
- backend/services/fertigation_service.py
- backend/services/file_service.py
- backend/services/financial_service.py
- backend/services/forum_service.py
- backend/services/freight_match_service.py
- backend/services/fx_service.py
- backend/services/genomic_simulator.py
- backend/services/geospatial_service.py
- backend/services/government_scheme_service.py
- backend/services/grading_engine.py
- backend/services/hydro_engine.py
- backend/services/insurance_service.py
- backend/services/inventory_service.py
- backend/services/irrigation_orchestrator.py
- backend/services/irrigation_service.py
- backend/services/knowledge_service.py
- backend/services/ledger_service.py
- backend/services/loan_scheduler.py
- backend/services/logistics_orchestrator.py
- backend/services/logistics_service.py
- backend/services/machinery_service.py
- backend/services/maintenance_forecaster.py
- backend/services/maintenance_logic.py
- backend/services/mandi_profit_optimizer.py
- backend/services/market_service.py
- backend/services/ml/plant_disease_classifier.py
- backend/services/ml/yield_prediction.py
- backend/services/neutralization_engine.py

- backend/services/notification_service.py
- backend/services/nutrient_advisor.py
- backend/services/pathogen_service.py
- backend/services/payroll_service.py
- backend/services/pdf_service.py
- backend/services/pipeline_service.py
- backend/services/pool_service.py
- backend/services/precision_irrigation_engine.py
- backend/services/predictive_maintenance.py
- backend/services/pricing_engine.py
- backend/services/processing_service.py
- backend/services/procurement_service.py
- backend/services/prophet_engine.py
- backend/services/rental_service.py
- backend/services/reputation_service.py
- backend/services/risk_adjustment_service.py
- backend/services/risk_service.py
- backend/services/sc_orchestrator.py
- backend/services/scheme_service.py
- backend/services/soil_analytics_service.py
- backend/services/soil_service.py
- backend/services/solvency_forecaster.py
- backend/services/traceability_service.py
- backend/services/transformer_logic.py
- backend/services/translation_service.py
- backend/services/transparency_service.py
- backend/services/vault_service.py
- backend/services/velocity_engine.py
- backend/services/virulence_engine.py
- backend/services/weather/irrigation_service.py
- backend/services/weather_service.py
- backend/services/yield_resilience_service.py
- backend/sockets/USAGE.md.py
- backend/sockets/__init__.py
- backend/sockets/alert_socket.py
- backend/sockets/crisis_events.py
- backend/sockets/forum_events.py
- backend/sockets/knowledge_events.py
- backend/sockets/logistics_events.py
- backend/sockets/market_events.py
- backend/sockets/notifications.py
- backend/sockets/pool_events.py

- backend/sockets/rental_events.py
- backend/sockets/supply_events.py
- backend/sockets/task_events.py
- backend/tasks/__init__.py
- backend/tasks/alert_cleanup.py
- backend/tasks/audit_tasks.py
- backend/tasks/carbon_minting_task.py
- backend/tasks/carbon_sync.py
- backend/tasks/circular_sync.py
- backend/tasks/climate_alerts.py
- backend/tasks/climate_sync.py
- backend/tasks/climate_tasks.py
- backend/tasks/core.py
- backend/tasks/diagnostic_sync.py
- backend/tasks/farm_tasks.py
- backend/tasks/financial_tasks.py
- backend/tasks/forum_tasks.py
- backend/tasks/fx_tasks.py
- backend/tasks/genome_sync.py
- backend/tasks/grid_sync.py
- backend/tasks/insurance_tasks.py
- backend/tasks/irrigation_sync.py
- backend/tasks/irrigation_tasks.py
- backend/tasks/knowledge_tasks.py
- backend/tasks/labor_tasks.py
- backend/tasks/loan_tasks.py
- backend/tasks/logistics_tasks.py
- backend/tasks/machinery_tasks.py
- backend/tasks/maintenance_sync.py
- backend/tasks/maintenance_tasks.py
- backend/tasks/market_sync.py
- backend/tasks/nutrient_sync.py
- backend/tasks/pathogen_tasks.py
- backend/tasks/pricing_tasks.py
- backend/tasks/processing_tasks.py
- backend/tasks/procurement_tasks.py
- backend/tasks/prophet_sync.py
- backend/tasks/quarantine_tasks.py
- backend/tasks/rental_tasks.py
- backend/tasks/report_tasks.py
- backend/tasks/route_optimization.py
- backend/tasks/soil_sync.py

- backend/tasks/soil_tasks.py
- backend/tasks/supply_chain_sync.py
- backend/tasks/sustainability_tasks.py
- backend/tasks/traceability_tasks.py
- backend/tasks/transparency_tasks.py
- backend/tasks/warehouse_tasks.py
- backend/tasks/water_sync.py
- backend/tasks/weather_tasks.py
- backend/test_auth_flow.py
- backend/test_i18n.py
- backend/test_notifications.py
- backend/test_pdf_synthesis.py
- backend/test_services.py
- backend/test_uploads.py
- backend/tests/test_climate_automation.py
- backend/tests/test_farms.py
- backend/tests/test_forum_logic.py
- backend/tests/test_gemini_config_validation.py
- backend/tests/test_insurance_v2.py
- backend/tests/test_irrigation.py
- backend/tests/test_knowledge.py
- backend/tests/test_labor_payroll.py
- backend/tests/test_loan_scheduler.py
- backend/tests/test_logistics.py
- backend/tests/test_machinery.py
- backend/tests/test_processing.py
- backend/tests/test_procurement.py
- backend/tests/test_rentals.py
- backend/tests/test_soil_health.py
- backend/tests/test_sustainability.py
- backend/tests/test_traceability.py
- backend/tests/test_translation_service.py
- backend/tests/test_warehouse.py
- backend/tests/test_weather.py
- backend/utils/carbon_formulas.py
- backend/utils/climate_formulas.py
- backend/utils/credit_scoring.py
- backend/utils/financial_math.py
- backend/utils/firebase_auth.py
- backend/utils/fleet_logic.py
- backend/utils/i18n.py
- backend/utils/i18n_utils.py

- backend/utils/invoice_generator.py
- backend/utils/iot_simulator.py
- backend/utils/logger.py
- backend/utils/nutrient_formulas.py
- backend/utils/payout_calculator.py
- backend/utils/payroll_formulas.py
- backend/utils/qr_generator.py
- backend/utils/quality_formulas.py
- backend/utils/risk_calculators.py
- backend/utils/route_formulas.py
- backend/utils/stock_formulas.py
- backend/utils/validation.py
- backend/utils/validators.py
- backend/utils/weather_api_client.py

(repository root)

- barter_exchange.html
- blog-backup.html
- blog.css
- blog.html
- blog.js
- build_interview_pdf.py
- buyer.html
- carbon_portal.html
- carbon_portal.js
- career.html
- careers.html
- carrer.css
- carrers.html
- certificate.html
- changes
- chat.css
- chat.html
- chat.js
- chatbot-responses.json
- climate_dashboard.html
- climate_dashboard.js
- community_forum.html
- completed
- contact.css
- contact.html
- cookie-policy.html
- crisis_dashboard.html

- crop.html
- cropCalendar.css
- cropCalendar.html
- cropCalendar.js
- crop_advisory.html
- crop_advisory.js

crop_recommendation

- crop_recommendation/Crop_recommendation.csv
- crop_recommendation/__init__.py
- crop_recommendation/app.py
- crop_recommendation/auth_utils.py
- crop_recommendation/cropRecommendation.ipynb
- crop_recommendation/modelbackup.ipynb
- crop_recommendation/routes.py
- crop_recommendation/static/style.css
- crop_recommendation/templates/index.html
- crop_recommendation/templates/result.html
- crop_recommendation/train_model.py

(repository root)

- crop_rotation.html

css

- css/certificate.css
- css/eco_style.css
- css/irrigation.css
- css/levelup.css
- css/pest_style.css
- css/rotation_planner.css
- css/spatial.css

(repository root)

- diff.txt
- disease.css
- disease.html
- disease.js
- docker-compose.yml

docs

- docs/API_VERSIONING.md
- docs/CONFIGURATION.md
- docs/GEWS_FEATURE.md
- docs/PR_SUMMARY_GEWS.md
- docs/QUICK_START_GEWS.md
- docs/QUICK_START_TRACEABILITY.md

- docs/SECURITY_IMPLEMENTATION.md
- docs/SPATIAL_ANALYTICS_IMPLEMENTATION.md
- docs/TRACEABILITY_FEATURE.md

domain

- domain/README.md
- domain/crop.schema.js
- domain/fertilizer.schema.js
- domain/growthStage.schema.js
- domain/index.js
- domain/prediction.schema.js
- domain/soil.schema.js
- domain/validate.js

(repository root)

- eco_toolbox.html
- equipment.html
- equipment.js
- equipments.html
- error.txt

examples

- examples/Agritechreactdemo.jsx
- examples/api_usage_example.py
- examples/crop_recommendation_demo.ipynb
- examples/disease_detection_demo.ipynb
- examples/fetch_example.js
- examples/test_ai_diagnostics.py
- examples/test_autonomous_fulfillment.py
- examples/test_carbon_escrow.py
- examples/test_genomics_engine.py
- examples/test_irrigation_logic.py
- examples/test_nutrient_optimization.py
- examples/test_predictive_maintenance.py
- examples/test_suite.py
- examples/test_virulence_engine.py
- examples/traceability_workflow.py
- examples/yield_prediction_demo.ipynb

(repository root)

- expert.html
- extensions.py
- faq.css
- faq.html
- farm_dashboard.html

- farm_dashboard.js
- farmer.css
- farmer.html
- farmer.js
- favorites.html
- feed-back.css
- feed-back.html
- feed-back.js
- finance.js
- finance_bot.html
- financial-support.html
- firebase.js
- firestore.rules
- fix_viewport.bat
- fleet_manager.html
- fleet_manager.js
- footer.css
- footer.js
- footer_audit.json
- forgot-password.html
- forum.css
- forum.js

frontend

- frontend/i18n/translations.js

(repository root)

- grocery.html

images

- images/Plantation.avif
- images/crop.avif
- images/disease.avif
- images/farmer.avif
- images/login.avif
- images/registration.avif
- images/shopkeeper.avif
- images/weather.webp
- images/yield.avif

(repository root)

- index.css
- index.html
- index.js

instance

- instance/agritech_dev.db

(repository root)

- insurance_portal.html
- insurance_portal.js
- iot_monitoring.html
- ipm_config.json
- irrigation.html
- irrigation_control.html
- irrigation_control.js

javascript

- javascript/i18n.js

js

- js/agri-helpers.js
- js/alert_manager.js
- js/certificate.js
- js/config.d.ts
- js/config.js
- js/eco_logic.js
- js/footer.js
- js/irrigation.js
- js/js/weather.js
- js/levelup.js
- js/notifications.js
- js/pest_logic.js
- js/rotation_engine.js
- js/spatial.js

(repository root)

- json-chatbot.js
- knowledge_hub.css
- knowledge_hub.html
- knowledge_hub.js
- labor_portal.html
- labor_portal.js
- learn.md
- levelup.html
- livestock.html
- loan_dashboard.html
- loan_dashboard.js

locals

- locals/en.json

- locals/hi.json
- locals/ta.json
- locals/te.json

(repository root)

- login.css
- login.html
- login.js
- logistics_hub.html
- logistics_hub.js
- main.css
- main.html
- make_project_explainer.py
- marketplace.html
- masthead.html

migrations

- migrations/alembic.ini
- migrations/create_forum_activities.sql
- migrations/env.py
- migrations/init_assets_logistics_db.py
- migrations/init_gews_db.py
- migrations/init_traceability_db.py
- migrations/init_yield_pools_db.py
- migrations/versions/8be05540cfac_add_full_name_to_users.py

(repository root)

- mission.css
- mission.html
- mobile-base.css
- mobile-main.css
- mobile-main.html
- mobile_setup.bat
- model.py
- navbar.html
- news.css
- news.html
- news.js
- notfound.html
- notifications.html
- offline.html
- organic.css
- organic.html
- organic.js

- package-lock.json
- package.json
- pest_suite.html
- plant-disease-cnn-1 (1).ipynb
- plantation.css
- plantation.html
- plantation.js
- privacy-policy.html
- processing_hub.html
- processing_hub.js
- procurement_hub.html
- procurement_hub.js
- pytest.ini
- register.css
- register.html
- register.js
- requirements.txt
- reset-password.html
- responsive.css
- roadmap.js
- scheme.html
- script.js

scripts

- scripts/i18n.js

(repository root)

- security_test.py
- security_utils.py
- server.js

server

- server/Controllers/user.js
- server/Middlewares/authMiddleware.js
- server/Routes/irrigation_routes.py
- server/Routes/rotation_routes.py
- server/Routes/user.js
- server/Utils/email.js
- server/Utils/irrigation_helper.py
- server/Utils/soil_analysis_logic.py
- server/config/db.js
- server/index.js
- server/models/crop_database.json
- server/models/irrigation_data.json

- server/models/user.js
- server/package-lock.json
- server/package.json

(repository root)

- service-worker.js
- shopkeeper-data.json
- shopkeeper.css
- shopkeeper.html
- shopkeeper.js
- sitemap.html
- smart_loan.html
- soil_portal.html
- soil_portal.js
- spatial.html

spatial_analytics

- spatial_analytics/__init__.py
- spatial_analytics/models.py
- spatial_analytics/routes.py
- spatial_analytics/utils.py

src

- src/frontend/components/RecentPredictions.js
- src/frontend/css/components/favorites.css
- src/frontend/js/components/favorite-btn.js
- src/frontend/js/features/favorites.js
- src/frontend/js/i18n/languages.js
- src/frontend/js/predictionStorage.js
- src/frontend/js/translate.js
- src/frontend/pages/favorites.html

(repository root)

- start_services.bat

static

- static/js/farm_progress.js
- static/js/weather_widget.js
- static/js/why-crop.js

(repository root)

- style.css
- styles.css

styles

- styles/switcher.css

(repository root)

- supply-chain.html
- sustainable-farming.html
- sustainable_meta.json

templates

- templates/404.html
- templates/yield_prediction.html

(repository root)

- terms-and-conditions.html
- terms-and-service.css
- terms-and-service.html
- test-blog.html
- test-mobile.html
- test.py
- test_security.py

tests

- tests/__init__.py
- tests/conftest.py
- tests/test_api.py
- tests/test_api_endpoints.py
- tests/test_basic.py
- tests/test_crop_planning.py
- tests/test_gemini_config_validation.py
- tests/test_helpers.py
- tests/test_irrigation.py
- tests/test_predictions.py
- tests/test_registration_validation.py
- tests/test_traceability.py
- tests/test_translation_service.py

(repository root)

- theme.css
- theme.js
- tmp_crop_image.py
- tmp_debug.py
- tmp_docx_build.py
- tmp_fix_disease.py
- tmp_generate_docs.py

tomato disease detection

- tomato disease detection/ReadMe.md
- tomato disease detection/Tomato_Disease_Prediction.ipynb
- tomato disease detection/backupmodel.ipynb

(repository root)

- trace.html
- trace_batch.js
- translations.js

translations

- translations/hi/LC_MESSAGES/messages.po
- translations/mr/LC_MESSAGES/messages.po

(repository root)

- transparency_portal.html
- unauthorized.html

unlock mechanism

- unlock mechanism/Unlock Mechanism (#1693)

(repository root)

- utils.py
- verify-produce.html
- voice-input.js
- warehouse_control.html
- warehouse_control.js
- weather-banner.css
- weather-banner.js
- weather.css
- weather.html
- weather.js
- weather_dashboard.html
- weather_dashboard.js

03 System Architecture

System Architecture

Simple picture

A browser page sends a request to Flask. Flask routes it to a feature handler. The handler validates input, calls business logic, may query SQLAlchemy, run a model or call an outside service, then returns HTML/JSON. Long jobs can be sent to Celery through Redis; Socket.IO can deliver live events.

Mermaid diagram source

```
flowchart LR
 U[Farmer / buyer / admin] --> UI[HTML CSS JavaScript pages]
 UI -->|HTTP form or fetch| F[Flask app.py]
 F --> BP[Registered Blueprints and direct routes]
 BP --> S[Services and domain utilities]
 S --> ML[ML models where artifacts exist]
 S --> DB[(SQL database via SQLAlchemy)]
 S --> EXT[External APIs: optional credentials]
 BP --> Q[Celery task queue]
 Q --> R[(Redis broker/result backend)]
 Q --> W[Celery worker]
 W --> DB
 F --> UI
 F --> WS[Socket.IO events where configured]
```

Frontend

The primary UI is multi-page HTML/CSS/JS, not a detected React SPA. Pages serve as entry points; JavaScript handles interactions and fetches where implemented. Root static serving and Flask rendering coexist. `templates/` is a Flask template folder, while many pages live at repository root.

Backend and API design

`app.py` initializes Flask, loads configuration, initializes extensions, registers direct handlers and Blueprints, then starts Socket.IO. API v1 is a parent Blueprint with many feature Blueprints. Smaller modules isolate routes by domain. Some domain services may also be imported by separate pages or services. The project has both synchronous form routes and JSON API routes.

Database

Flask-SQLAlchemy maps Python model classes to relational tables. Flask-Migrate records schema evolution. Development defaults to SQLite; environment configuration can use a different database. `db.create_all()` is called at startup in `app.py`, while migrations also exist; deployment practices should choose a controlled migration flow. Model inventory is in `06_Database_Documentation.md`.

Integrations and deployment dependencies

Gemini, Firebase, SMTP, weather, FX, translation, S3, Redis and messaging settings are configurable. A credential name in `.env.example` means a possible integration, not a successful connection. Docker Compose includes Flask, Redis, MailHog and Celery worker. Production WSGI/hosting setup was not verified in repository configuration.

Example synchronous sequence

Mermaid diagram source

```
sequenceDiagram
 participant B as Browser
 participant F as Flask route
 participant V as Validation
 participant S as Feature service
 participant M as Model or DB
 B->>F: POST form / JSON
 F->>V: Check required fields and types
 V-->>F: Valid input
 F->>S: Call feature logic
 S->>M: Predict / query / persist
 M-->>S: Result
 S-->>F: Domain result
 F-->>B: HTML or JSON response
```

Async sequence

Mermaid diagram source

```
sequenceDiagram
 participant B as Browser
 participant F as Flask
 participant Q as Redis/Celery broker
 participant W as Worker
 B->>F: Submit long task
 F->>Q: enqueue task
 F-->>B: 202 + task_id
 Q->>W: deliver task
 W->>W: compute and store result
 B->>F: GET status(task_id)
 F-->>B: pending / processing / completed / failed
```

04 Frontend Documentation

Frontend Documentation

Framework and entry points

The main frontend is plain HTML, CSS and browser JavaScript. No root package script builds a frontend bundle. `/` serves root `index.html`; other direct Flask paths serve feature pages. `templates/` contains server-rendered pages such as `yield_prediction.html`. `server.js` is a separate Express/Socket.IO-oriented option, not proof of a frontend framework.

Page families

Root files include home/navigation, login/register/reset, crop, disease, yield, fertilizer, weather, dashboards, marketplace, forum, equipment, finance, government schemes, logistics, sustainability and service portals. Specific scripts typically match page names (for example `disease.html` and `disease.js`). Directories such as `src/`, `frontend/`, `static/`, `styles/`, `css/`, `js/`, and `javascript/` are present, but overlapping names mean verify which files are actually linked by the target HTML.

Components, state and routing

There is no verified React component hierarchy, props system, Redux/Zustand store or React Router. Each page owns its DOM and page script. State may be in JS variables, browser localStorage, session/auth mechanisms, or server responses. Navigation is mostly links and direct routes. `disease.js`, for example, tracks selected file and prediction history in localStorage.

Forms and API calls

Pages use HTML forms and JavaScript event listeners. Requests can be standard form posts (server returns HTML) or `fetch` calls (typically JSON). For any UI control, trace: input element → event listener → validation/serialization → URL/method → response parsing → DOM update. Client validation is usability only; backend validation is required.

Authentication UI

`login.html`, `register.html`, `auth.js` and backend auth routes exist. The repository contains multiple auth approaches; inspect the actual page script to learn whether it attaches Bearer tokens, uses cookies or uses Firebase. Do not say all pages are protected; backend decorator coverage differs by endpoint. `reset-password.html` is served by a page route, but token correctness and password-reset backend workflow must be confirmed in the auth module.

Loading, error and responsive behavior

Some scripts explicitly show loading states and inline errors (disease analysis is one example); this pattern is not guaranteed across every page. CSS files provide layouts and breakpoints, but a global design system/component framework was not detected. Confirm at target viewport sizes and inspect each page's script for error paths.

No inferred components

A page name does not establish a reusable component or complete backend integration. The source should be traced before documenting a frontend/backend contract. The disease classifier flow is: image selected → client downscales to <=1600px edge and JPEG → POST base64 JSON to `/api/v1/ai-disease/predict-trained` → display candidate / uncertainty and history.

05 Backend Documentation

Backend Documentation

Startup

`app.py` loads `.env`, imports extension and feature modules, builds the Flask app, applies a selected config, sets upload storage, initializes SQLAlchemy, migrations, mail and limiter, configures Celery and audit middleware, imports models, configures CORS, registers Blueprints, initializes Socket.IO, cache and Babel, creates tables in app context, then configures Gemini. Note: startup currently raises if `GEMINI_API_KEY` is absent outside testing; this is an operational coupling to an external key and should be considered when running locally.

Layers

- Routes/controllers: Flask route functions and Blueprint modules accept HTTP requests and choose response codes/forms.
- Validation/schemas: Marshmallow schemas and validation utilities check incoming data; coverage varies.
- Services/domain logic: `backend/services/`, `agri_utils.py`, `server/Routes/`, and feature packages implement actions.
- Models: `backend/models/` defines ORM entities.
- Middleware: audit, CORS, rate limiting and auth decorators.
- Async/realtime: Celery tasks and Socket.IO event handlers.

Main direct routes

`app.py` directly defines page-serving routes, Firebase config, task status, async crop, fertilizer, loan/report endpoints, yield prediction, reset-password/static catch-all, rotation alias and error handlers. See complete decorator appendix in `08_API_Documentation.md`.

API families

`backend/api/v1/__init__.py` registers a large set of versioned APIs for auth, tasks, assets/files, forum, pools, market, risk, schemes, weather, traceability, disease, insurance, farm, advisory, sustainability, procurement, irrigation, soil, climate, labor, logistics, audit, finance, carbon, warehouse and more. API v1 itself is mounted by `backend/api/__init__.py`. Endpoint specifics remain in each route module.

Request and response lifecycle

Mermaid diagram source

```
flowchart TD
 R[HTTP request] --> CORS[CORS / WSGI / Flask dispatch]
 CORS --> AUTH{Route has auth decorator?}
 AUTH -->|yes| TOKEN[Verify token and role]
 AUTH -->|no| VALID[Parse and validate request]
 TOKEN --> VALID
 VALID --> HANDLER[Route handler]
 HANDLER --> SERVICE[Business service / domain utility]
 SERVICE --> DB[(SQLAlchemy database)]
 SERVICE --> MODEL[ML model or external API when configured]
 DB --> FORMAT[Build safe result]
 MODEL --> FORMAT
 FORMAT --> RESP[HTML / JSON / file / error]
 RESP --> CLIENT[Browser]
```

Validation, errors and logging

Some routes perform manual required-field and range checks; schemas/utilities exist elsewhere. Flask has global 404 handling and logging. Error handling is not uniform: several routes catch broad exceptions and return message text. Production should avoid exposing exception details. Transactions should rollback after DB failures.

Uploads and storage

An upload folder is configured by `app.py`; config supports local/S3 settings. File API and ingestion Blueprints exist. Confirm each endpoint's limits, safe filename handling, authorization, content validation and ownership check before claiming secure upload behavior.

Feature flags and disabled paths

`FeatureFlags` fields exist, but source search found no callers reading them. Their environment values do not currently appear to gate routes. The old disease import path referenced in a commented line does not exist; disease APIs are registered via API v1. Crop inference can be unavailable if its model/encoder artifacts are absent.

06 Database Documentation

Database Documentation

Technology and connection

Flask-SQLAlchemy is the ORM; Flask-Migrate/Alembic handles schema revisions. `backend/config/settings.py` defaults to SQLite for development and reads `DATABASE_URL`/`DEV_DATABASE_URL`. The active environment may override this. The repository also contains `instance/agritech_dev.db`; documentation does not inspect or disclose user data.

How ORM persistence works

A route or service queries model classes, constructs/updates objects, calls `db.session.add`, then `commit`. A failed transaction should call `rollback`. SQLAlchemy generates parameterized SQL. Migrations should be applied deliberately in deployments; `db.create_all()` at application startup does not replace schema migration management.

Models and schema

The following per-model inventory is extracted from direct `db.Column` declarations in `backend/models/`. Relationships, inherited fields, constraints, indexes and migration-only details are not fully inferable from that summary; inspect each source and migration.

SQLAlchemy model inventory extracted from `backend/models/`

Fields below are the declarations found on model classes; inherited fields, relationships, constraints, runtime migrations and dynamically created columns may not appear. See each source file for authoritative definitions.

`CropDiagnosticReport` (table `CropDiagnosticReport`)

- Source: `backend/models/ai_diagnostics.py`
- Declared columns: `id (db.Integer)`, `farm_id (db.Integer)`, `user_id (db.Integer)`, `image_url (db.String(512))`, `identified_pathogen (db.String(100))`, `confidence_score (db.Float)`, `severity_index (db.Float)`, `recommended_treatment (db.Text)`, `estimated_yield_loss_pct (db.Float)`, `status (db.String(20))`, `created_at (db.DateTime)`

`DiagnosticVerification` (table `DiagnosticVerification`)

- Source: `backend/models/ai_diagnostics.py`
- Declared columns: `id (db.Integer)`, `report_id (db.Integer)`, `expert_id (db.Integer)`, `ai_was_correct (db.Boolean)`, `expert_notes (db.Text)`, `verified_at (db.DateTime)`

`PathogenKnowledgeBase` (table `PathogenKnowledgeBase`)

- Source: `backend/models/ai_diagnostics.py`
- Declared columns: `id (db.Integer)`, `pathogen_name (db.String(100))`, `typical_symptoms (db.Text)`, `suggested_chemical_treatment (db.Text)`, `organic_alternative (db.Text)`, `mortality_rate_potential (db.Float)`

`Alert` (table `Alert`)

- Source: `backend/models/alert.py`
- Declared columns: `id (db.Integer)`, `user_id (db.Integer)`, `title (db.String(255))`, `message (db.Text)`, `priority (db.String(20))`, `category (db.String(50))`, `group_key (db.String(100))`, `websocket_delivered (db.Boolean)`, `email_delivered (db.Boolean)`, `sms_delivered (db.Boolean)`, `push_delivered (db.Boolean)`, `expires_at (db.DateTime)`, `read_at (db.DateTime)`, `created_at (db.DateTime)`, `action_url (db.String(512))`, `metadata_json (db.Text)`

`AlertPreference` (table `AlertPreference`)

- Source: `backend/models/alert.py`
- Declared columns: `id (db.Integer)`, `user_id (db.Integer)`, `category (db.String(50))`, `email_enabled (db.Boolean)`, `sms_enabled (db.Boolean)`, `push_enabled (db.Boolean)`, `websocket_enabled (db.Boolean)`, `min_priority (db.String(20))`

`ArbitrageOpportunity` (table `ArbitrageOpportunity`)

- Source: `backend/models/arbitrage.py`
- Declared columns: `id (db.Integer)`, `commodity_type (db.String(100))`, `source_grid_id (db.Integer)`, `source_price_per_kg (db.Float)`, `target_grid_id (db.Integer)`, `target_price_per_kg (db.Float)`, `estimated_transport_cost_usd (db.Float)`, `carbon_tax_offset_usd (db.Float)`, `gross_margin_pct (db.Float)`, `net_arbitrage_profit (db.Float)`, `confidence_score (db.Float)`, `status (db.String(20))`, `discovered_at (db.DateTime)`, `expires_at (db.DateTime)`

`AlgorithmicTradeRecord` (table `AlgorithmicTradeRecord`)

- Source: `backend/models/arbitrage.py`
- Declared columns: `id (db.Integer)`, `opportunity_id (db.Integer)`, `execution_volume_kg (db.Float)`, `actual_buy_price (db.Float)`, `actual_sell_price (db.Float)`, `actual_transport_cost (db.Float)`, `realized_profit_usd (db.Float)`, `slippage_pct (db.Float)`, `ledger_txn_id (db.Integer)`, `executed_at (db.DateTime)`, `settled_at (db.DateTime)`

`AuditLog` (table `AuditLog`)

- Source: `backend/models/audit_log.py`
- Declared columns: `id (db.Integer)`, `user_id (db.Integer)`, `action (db.String(255))`, `resource_type (db.String(100))`, `resource_id (db.String(100))`, `old_values (db.Text)`, `new_values (db.Text)`, `ip_address (db.String(45))`, `user_agent (db.String(512))`, `method (db.String(10))`, `url (db.String(1024))`, `status_code (db.Integer)`, `risk_level (db.String(20))`, `threat_flag (db.Boolean)`, `is_financial (db.Boolean)`, `financial_impact (db.Float)`, `autonomous_decision_flag (db.Boolean)`, `ai_logistics_flag (db.Boolean)`, `irrigation_auto_flag (db.Boolean)`, `nutrient_optimization_flag (db.Boolean)`, `timestamp (db.DateTime)`, `meta_data (db.Text)`

`UserSession` (table `UserSession`)

- Source: `backend/models/audit_log.py`
- Declared columns: `id (db.Integer)`, `user_id (db.Integer)`, `session_token (db.String(255))`, `ip_address (db.String(45))`, `user_agent (db.String(512))`, `login_time (db.DateTime)`, `last_activity (db.DateTime)`, `logout_time (db.DateTime)`, `is_active (db.Boolean)`

`SmartContractOrder` (table `SmartContractOrder`)

- Source: `backend/models/autonomous_supply.py`
- Declared columns: `id (db.Integer)`, `buyer_id (db.Integer)`, `vendor_id (db.Integer)`, `commodity (db.String(100))`, `quantity_kg (db.Float)`, `strike_price_usd (db.Float)`, `auto_execute_at_price (db.Float)`, `status (db.String(30))`, `freight_locked_id (db.Integer)`, `created_at (db.DateTime)`

`FreightGeoFence` (table `FreightGeoFence`)

- Source: `backend/models/autonomous_supply.py`
- Declared columns: `id (db.Integer)`, `order_id (db.Integer)`, `target_lat (db.Float)`, `target_lng (db.Float)`, `radius_meters (db.Float)`, `current_lat (db.Float)`, `current_lng (db.Float)`, `is_triggered (db.Boolean)`, `triggered_at (db.DateTime)`

`BarterTransaction` (table `BarterTransaction`)

- Source: `backend/models/barter.py`
- Declared columns: `id (db.Integer)`, `initiator_id (db.Integer)`, `responder_id (db.Integer)`, `status (db.String(20))`, `initiator_committed (db.Boolean)`, `responder_committed (db.Boolean)`, `initiator_confirmed_fulfillment (db.Boolean)`, `responder_confirmed_fulfillment (db.Boolean)`, `value_index_context (db.Text)`, `created_at (db.DateTime)`, `updated_at (db.DateTime)`

`BarterResource` (table `BarterResource`)

- Source: `backend/models/barter.py`
- Declared columns: `id (db.Integer)`, `transaction_id (db.Integer)`, `provider_id (db.Integer)`, `resource_category (db.String(50))`, `resource_reference_id (db.Integer)`, `resource_name (db.String(100))`, `quantity (db.Float)`, `unit (db.String(20))`, `unit_value_index (db.Float)`, `total_value_index (db.Float)`

`ResourceValueIndex` (table `ResourceValueIndex`)

- Source: `backend/models/barter.py`
- Declared columns: `id (db.Integer)`, `category (db.String(50))`, `item_name (db.String(100))`, `current_value (db.Float)`, `demand_multiplier (db.Float)`, `last_updated (db.DateTime)`

`BiomassStockpile` (table `BiomassStockpile`)

- Source: `backend/models/biomass.py`
- Declared columns: `id (db.Integer)`, `farm_id (db.Integer)`, `stockpile_type (db.String(50))`, `total_mass_kg (db.Float)`, `moisture_content_pct (db.Float)`, `calorific_value_mj_per_kg (db.Float)`, `is_fermenting (db.Boolean)`, `methane_yield_potential_m3 (db.Float)`, `last_updated (db.DateTime)`

`BiogasDigesterLog` (table `BiogasDigesterLog`)

- Source: `backend/models/biomass.py`
- Declared columns: `id (db.Integer)`, `farm_id (db.Integer)`, `stockpile_id (db.Integer)`, `mass_consumed_kg (db.Float)`, `digestion_temp_c (db.Float)`, `ph_level (db.Float)`, `methane_produced_m3 (db.Float)`, `electricity_generated_kwh (db.Float)`, `status (db.String(20))`, `start_time (db.DateTime)`, `end_time (db.DateTime)`

`CarbonTradeEscrow` (table `CarbonTradeEscrow`)

- Source: `backend/models/carbon_escrow.py`
- Declared columns: `id (db.Integer)`, `seller_id (db.Integer)`, `buyer_id (db.Integer)`, `credits_volume (db.Float)`, `price_per_credit_usd (db.Float)`, `total_value_usd (db.Float)`, `status (db.String(20))`, `verification_proof_hash (db.String(128))`, `created_at (db.DateTime)`, `expires_at (db.DateTime)`, `ledger_txn_id (db.Integer)`

`EscrowAuditLog` (table `EscrowAuditLog`)

- Source: `backend/models/carbon_escrow.py`
- Declared columns: `id (db.Integer)`, `escrow_id (db.Integer)`, `action (db.String(50))`, `actor_id (db.Integer)`, `details (db.Text)`, `timestamp (db.DateTime)`

`CarbonCreditWallet` (table `CarbonCreditWallet`)

- Source: `backend/models/carbon_escrow.py`
- Declared columns: `id (db.Integer)`, `user_id (db.Integer)`, `total_credits_minted (db.Float)`, `available_credits_for_sale (db.Float)`, `frozen_credits_in_escrow (db.Float)`, `last_rebalancing_at (db.DateTime)`

`WasteInventory` (table `WasteInventory`)

- Source: `backend/models/circular.py`
- Declared columns: `id (db.Integer)`, `farm_id (db.Integer)`, `waste_type (db.String(100))`, `quantity_kg (db.Float)`, `status (db.String(50))`, `batch_source_id (db.Integer)`, `is_reused_on_farm (db.Boolean)`, `created_at (db.DateTime)`, `transformed_at (db.DateTime)`

`BioEnergyOutput` (table `BioEnergyOutput`)

- Source: `backend/models/circular.py`
- Declared columns: `id (db.Integer)`, `farm_id (db.Integer)`, `waste_id (db.Integer)`, `energy_type (db.String(50))`, `amount_kwh (db.Float)`, `efficiency_ratio (db.Float)`, `carbon_offset_kg (db.Float)`, `timestamp (db.DateTime)`

`CircularCredit` (table `CircularCredit`)

- Source: `backend/models/circular.py`
- Declared columns: `id (db.Integer)`, `user_id (db.Integer)`, `farm_id (db.Integer)`, `credit_amount (db.Float)`, `source_type (db.String(50))`, `audit_chain_hash (db.String(64))`, `created_at (db.DateTime)`, `expires_at (db.DateTime)`

`ClimateZone` (table `ClimateZone`)

- Source: `backend/models/climate.py`
- Declared columns: `id (db.Integer)`, `farm_id (db.Integer)`, `name (db.String(100))`, `description (db.Text)`, `target_temp_min (db.Float)`, `target_temp_max (db.Float)`, `target_humidity_min (db.Float)`, `target_humidity_max`

(db.Float), target_co2 (db.Float), created_at (db.DateTime)

SensorNode (table SensorNode)

- Source: backend/models/climate.py
- Declared columns: id (db.Integer), zone_id (db.Integer), uid (db.String(50)), node_type (db.String(50)), firmware_version (db.String(20)), is_active (db.Boolean), last_heartbeat (db.DateTime), created_at (db.DateTime)

TelemetryLog (table TelemetryLog)

- Source: backend/models/climate.py
- Declared columns: id (db.LongInteger if hasattr(db, 'LongInteger') else db.Integer), node_id (db.Integer), temperature (db.Float), humidity (db.Float), co2_level (db.Float), light_intensity (db.Float), soil_moisture (db.Float), soil_ph (db.Float), battery_level (db.Float), signal_strength (db.Float), timestamp (db.DateTime)

AutomationTrigger (table AutomationTrigger)

- Source: backend/models/climate.py
- Declared columns: id (db.Integer), zone_id (db.Integer), name (db.String(100)), actor_type (db.String(50)), condition_json (db.Text), action_json (db.Text), is_enabled (db.Boolean), last_triggered (db.DateTime), created_at (db.DateTime)

MigrationVector (table MigrationVector)

- Source: backend/models/disease.py
- Declared columns: id (db.Integer), outbreak_zone_id (db.Integer), origin_lat (db.Float), origin_lng (db.Float), target_lat (db.Float), target_lng (db.Float), direction_degrees (db.Float), speed_km_per_day (db.Float), intensity_index (db.Float), wind_influence_factor (db.Float), humidity_breeding_coeff (db.Float), predicted_arrival_time (db.DateTime), created_at (db.DateTime)

ContainmentZone (table ContainmentZone)

- Source: backend/models/disease.py
- Declared columns: id (db.Integer), zone_name (db.String(100)), center_lat (db.Float), center_lng (db.Float), radius_meters (db.Float), enforcement_level (db.String(20)), irrigation_shutdown_triggered (db.Boolean), logistics_blockade_active (db.Boolean), quarantine_start (db.DateTime), quarantine_end (db.DateTime)

DecentralizedEnergyGrid (table DecentralizedEnergyGrid)

- Source: backend/models/energy_grid.py
- Declared columns: id (db.Integer), grid_name (db.String(100)), region_code (db.String(50)), target_capacity_mwh (db.Float), current_active_load_mwh (db.Float), current_supply_mwh (db.Float), dynamic_feed_in_tariff_usd (db.Float), status (db.String(20)), last_rebalancing (db.DateTime)

EnergyTokenMint (table EnergyTokenMint)

- Source: backend/models/energy_grid.py
- Declared columns: id (db.Integer), token_hash (db.String(64)), farm_id (db.Integer), grid_id (db.Integer), digester_log_id (db.Integer), energy_mwh (db.Float), mint_price_usd (db.Float), ledger_txn_id (db.Integer), status (db.String(20)), minted_at (db.DateTime)

GridInjectionLog (table GridInjectionLog)

- Source: backend/models/energy_grid.py
- Declared columns: id (db.Integer), farm_id (db.Integer), grid_id (db.Integer), kwh_injected (db.Float), voltage_v (db.Float), frequency_hz (db.Float), recorded_at (db.DateTime)

Equipment (table Equipment)

- Source: backend/models/equipment.py
- Declared columns: id (db.Integer), owner_id (db.Integer), name (db.String(100)), category (db.String(50)), description (db.Text), hourly_rate (db.Float), daily_rate (db.Float), location (db.String(255)), image_url (db.String(255)), specifications (db.Text), is_available (db.Boolean), created_at (db.DateTime),

reliability_score (db.Float), vibration_level (db.Float), heat_index (db.Float), duty_cycle_rating (db.Float), last_health_check (db.DateTime)

RentalBooking (table RentalBooking)

- Source: backend/models/equipment.py
- Declared columns: id (db.Integer), equipment_id (db.Integer), renter_id (db.Integer), start_time (db.DateTime), end_time (db.DateTime), total_price (db.Float), security_deposit (db.Float), status (db.String(20)), payment_intent_id (db.String(100)), created_at (db.DateTime)

AvailabilityCalendar (table AvailabilityCalendar)

- Source: backend/models/equipment.py
- Declared columns: id (db.Integer), equipment_id (db.Integer), date (db.Date), is_blocked (db.Boolean), reason (db.String(100))

PaymentEscrow (table PaymentEscrow)

- Source: backend/models/equipment.py
- Declared columns: id (db.Integer), booking_id (db.Integer), total_amount (db.Float), held_at (db.DateTime), status (db.String(20)), released_at (db.DateTime), dispute_reason (db.Text), transaction_ref (db.String(100))

Farm (table Farm)

- Source: backend/models/farm.py
- Declared columns: id (db.Integer), name (db.String(100)), description (db.Text), location (db.String(255)), acreage (db.Float), soil_details (db.Text), harvest_readiness_index (db.Float), predicted_yield_volume (db.Float), last_velocity_update (db.DateTime), is_no_till (db.Boolean), cover_crop_active (db.Boolean), organic_certified (db.Boolean), sequestration_tier (db.String(20)), total_carbon_credits_minted (db.Float), spatial_grid_id (db.Integer), created_at (db.DateTime)

FarmMember (table FarmMember)

- Source: backend/models/farm.py
- Declared columns: id (db.Integer), farm_id (db.Integer), user_id (db.Integer), role (db.String(20)), joined_at (db.DateTime), invited_by_id (db.Integer)

FarmAsset (table FarmAsset)

- Source: backend/models/farm.py
- Declared columns: id (db.Integer), farm_id (db.Integer), asset_id (db.String(64)), user_id (db.Integer), asset_type (db.String(100)), asset_name (db.String(200)), manufacturer (db.String(100)), model (db.String(100)), serial_number (db.String(100)), purchase_price (db.Float), status (db.String(30)), health_score (db.Float), alert_threshold_days (db.Integer), total_runtime_hours (db.Float), last_telemetry (db.Text), predicted_days_to_failure (db.Integer), last_maintenance_date (db.DateTime), next_maintenance_due (db.DateTime), name (db.String(100)), category (db.String(50)), purchase_value (db.Float), purchase_date (db.DateTime), condition (db.String(50)), last_maintenance (db.DateTime), current_valuation (db.Float), created_at (db.DateTime)

MaintenanceLog (table MaintenanceLog)

- Source: backend/models/farm.py
- Declared columns: id (db.Integer), log_id (db.String(80)), asset_id (db.Integer), maintenance_type (db.String(30)), description (db.Text), parts_replaced (db.Text), cost (db.Float), pre_maintenance_health (db.Float), post_maintenance_health (db.Float), technician_name (db.String(200)), technician_notes (db.Text), scheduled_date (db.DateTime), completed_date (db.DateTime), status (db.String(30)), created_at (db.DateTime)

FieldMicronutrients (table FieldMicronutrients)

- Source: backend/models/fertigation_v2.py
- Declared columns: id (db.Integer), farm_id (db.Integer), zone_id (db.Integer), zinc (db.Float), iron (db.Float), boron (db.Float), manganese (db.Float), copper (db.Float), microbial_activity_index (db.Float), nutrient_availability_score (db.Float), recorded_at (db.DateTime)

`NutrientInjectionLog` (table `NutrientInjectionLog`)

- Source: `backend/models/fertigation_v2.py`
- Declared columns: `id (db.Integer)`, `zone_id (db.Integer)`, `nitrogen_content_pct (db.Float)`, `phosphorus_content_pct (db.Float)`, `potassium_content_pct (db.Float)`, `sulfur_content_pct (db.Float)`, `total_injected_volume_l (db.Float)`, `autonomous_trigger_code (db.String(50))`, `predicted_runoff_loss_pct (db.Float)`, `recorded_at (db.DateTime)`

`FarmBalanceSheet` (table `FarmBalanceSheet`)

- Source: `backend/models/financials.py`
- Declared columns: `id (db.Integer)`, `farm_id (db.Integer)`, `cash_on_hand (db.Float)`, `inventory_value (db.Float)`, `fixed_assets_value (db.Float)`, `outstanding_loans (db.Float)`, `accounts_payable (db.Float)`, `net_worth (db.Float)`, `period_date (db.Date)`, `created_at (db.DateTime)`

`ProfitabilityIndex` (table `ProfitabilityIndex`)

- Source: `backend/models/financials.py`
- Declared columns: `id (db.Integer)`, `farm_id (db.Integer)`, `roi_machinery (db.Float)`, `roi_labor (db.Float)`, `yield_efficiency (db.Float)`, `overall_profitability_score (db.Float)`, `calculated_at (db.DateTime)`

`SolvencySnapshot` (table `SolvencySnapshot`)

- Source: `backend/models/financials.py`
- Declared columns: `id (db.Integer)`, `farm_id (db.Integer)`, `debt_to_equity_ratio (db.Float)`, `liquidity_ratio (db.Float)`, `bankruptcy_risk_score (db.Float)`, `status (db.String(20))`, `next_review_date (db.Date)`, `created_at (db.DateTime)`

`ForumCategory` (table `ForumCategory`)

- Source: `backend/models/forum.py`
- Declared columns: `id (db.Integer)`, `name (db.String(100))`, `description (db.Text)`, `icon (db.String(50))`, `created_at (db.DateTime)`

`ForumThread` (table `ForumThread`)

- Source: `backend/models/forum.py`
- Declared columns: `id (db.Integer)`, `category_id (db.Integer)`, `user_id (db.Integer)`, `title (db.String(255))`, `content (db.Text)`, `tags (db.String(255))`, `is_pinned (db.Boolean)`, `is_locked (db.Boolean)`, `is_flagged (db.Boolean)`, `flag_reason (db.String(255))`, `sentiment_score (db.Float)`, `toxicity_score (db.Float)`, `is_ai_approved (db.Boolean)`, `view_count (db.Integer)`, `upvote_count (db.Integer)`, `created_at (db.DateTime)`, `updated_at (db.DateTime)`, `last_activity (db.DateTime)`

`PostComment` (table `PostComment`)

- Source: `backend/models/forum.py`
- Declared columns: `id (db.Integer)`, `thread_id (db.Integer)`, `user_id (db.Integer)`, `parent_id (db.Integer)`, `content (db.Text)`, `sentiment_score (db.Float)`, `is_ai_approved (db.Boolean)`, `is_ai_generated (db.Boolean)`, `is_flagged (db.Boolean)`, `flag_reason (db.String(255))`, `upvote_count (db.Integer)`, `created_at (db.DateTime)`, `updated_at (db.DateTime)`

`Upvote` (table `Upvote`)

- Source: `backend/models/forum.py`
- Declared columns: `id (db.Integer)`, `user_id (db.Integer)`, `thread_id (db.Integer)`, `comment_id (db.Integer)`, `created_at (db.DateTime)`

`UserReputation` (table `UserReputation`)

- Source: `backend/models/forum.py`
- Declared columns: `id (db.Integer)`, `user_id (db.Integer)`, `total_score (db.Integer)`, `threads_created (db.Integer)`, `comments_posted (db.Integer)`, `upvotes_received (db.Integer)`, `helpful_answers (db.Integer)`, `is_expert (db.Boolean)`, `is_moderator (db.Boolean)`, `badges (db.Text)`, `updated_at (db.DateTime)`

`AutonomousVehicle` (table `AutonomousVehicle`)

- Source: `backend/models/freight_v2.py`
- Declared columns: `id (db.Integer)`, `serial_number (db.String(100))`, `vehicle_type (db.String(50))`, `current_capacity_kg (db.Float)`, `battery_level_pct (db.Float)`, `status (db.String(20))`, `current_lat (db.Float)`, `current_lng (db.Float)`, `last_ping (db.DateTime)`

`VehicleMission` (table `VehicleMission`)

- Source: `backend/models/freight_v2.py`
- Declared columns: `id (db.Integer)`, `vehicle_id (db.Integer)`, `order_id (db.Integer)`, `assigned_at (db.DateTime)`, `completed_at (db.DateTime)`, `distance_traveled_km (db.Float)`, `energy_consumed_kwh (db.Float)`

`SeedGenomeProfile` (table `SeedGenomeProfile`)

- Source: `backend/models/genomics.py`
- Declared columns: `id (db.Integer)`, `strain_name (db.String(150))`, `species (db.String(50))`, `drought_tolerance_allele (db.Float)`, `heat_shock_protein_expression (db.Float)`, `pest_resistance_marker (db.Float)`, `yield_vigor_multiplier (db.Float)`, `is_crispr_edited (db.Boolean)`, `original_wild_type_id (db.Integer)`, `generation_num (db.Integer)`, `lab_certification_hash (db.String(128))`, `created_at (db.DateTime)`

`LiveCropPhenotype` (table `LiveCropPhenotype`)

- Source: `backend/models/genomics.py`
- Declared columns: `id (db.Integer)`, `farm_id (db.Integer)`, `genome_id (db.Integer)`, `expressed_drought_tolerance (db.Float)`, `expressed_heat_shock_resilience (db.Float)`, `expressed_pest_defense (db.Float)`, `epigenetic_stress_factor (db.Float)`, `current_health_score (db.Float)`, `planted_at (db.DateTime)`, `harvested_at (db.DateTime)`, `status (db.String(20))`

`EpigeneticDriftLog` (table `EpigeneticDriftLog`)

- Source: `backend/models/genomics.py`
- Declared columns: `id (db.Integer)`, `phenotype_id (db.Integer)`, `triggering_event (db.String(100))`, `delta_drought_tolerance (db.Float)`, `delta_stress_factor (db.Float)`, `delta_health_score (db.Float)`, `recorded_at (db.DateTime)`

`DiseaseIncident` (table `DiseaseIncident`)

- Source: `backend/models/gews.py`
- Declared columns: `id (db.Integer)`, `incident_id (db.String(20))`, `user_id (db.Integer)`, `disease_name (db.String(100))`, `crop_affected (db.String(100))`, `severity_level (db.String(20))`, `symptoms (db.Text)`, `latitude (db.Float)`, `longitude (db.Float)`, `affected_area (db.Float)`, `detection_method (db.String(50))`, `verification_status (db.String(20))`, `reported_at (db.DateTime)`, `outbreak_zone_id (db.Integer)`

`OutbreakZone` (table `OutbreakZone`)

- Source: `backend/models/gews.py`
- Declared columns: `id (db.Integer)`, `zone_id (db.String(50))`, `disease_name (db.String(100))`, `crop_affected (db.String(100))`, `severity_level (db.String(20))`, `center_latitude (db.Float)`, `center_longitude (db.Float)`, `radius_km (db.Float)`, `incident_count (db.Integer)`, `total_affected_area (db.Float)`, `status (db.String(20))`, `risk_level (db.String(20))`, `propagation_velocity (db.Float)`, `transmission_radius (db.Float)`, `wind_vector_deg (db.Float)`, `soil_connectivity_score (db.Float)`, `containment_status (db.String(30))`, `containment_applied_at (db.DateTime)`, `created_at (db.DateTime)`

`OutbreakProjection` (table `OutbreakProjection`)

- Source: `backend/models/gews.py`
- Declared columns: `id (db.Integer)`, `outbreak_zone_id (db.Integer)`, `projection_time (db.DateTime)`, `projected_latitude (db.Float)`, `projected_longitude (db.Float)`, `expected_radius (db.Float)`, `confidence_score (db.Float)`, `created_at (db.DateTime)`

`OutbreakAlert` (table `OutbreakAlert`)

- Source: `backend/models/gews.py`

- Declared columns: `id (db.Integer)`, `alert_id (db.String(50))`, `user_id (db.Integer)`, `zone_id (db.Integer)`, `status (db.String(20))`, `priority (db.String(20))`, `message (db.Text)`, `created_at (db.DateTime)`, `acknowledged_at (db.DateTime)`

`CustomsManifest` (table `CustomsManifest`)

- Source: `backend/models/global_trade.py`
- Declared columns: `id (db.Integer)`, `order_id (db.Integer)`, `declaration_number (db.String(50))`, `country_of_origin (db.String(50))`, `destination_country (db.String(50))`, `hs_code (db.String(20))`, `total_value_usd (db.Float)`, `status (db.String(20))`, `rejection_reason (db.Text)`, `created_at (db.DateTime)`, `cleared_at (db.DateTime)`, `phyto_cert_url (db.String(255))`, `commercial_invoice_url (db.String(255))`

`ComplianceCheck` (table `ComplianceCheck`)

- Source: `backend/models/global_trade.py`
- Declared columns: `id (db.Integer)`, `manifest_id (db.Integer)`, `batch_id (db.Integer)`, `check_type (db.String(50))`, `passed (db.Boolean)`, `score (db.Float)`, `details (db.Text)`, `timestamp (db.DateTime)`

`GovernmentScheme` (table `GovernmentScheme`)

- Source: `backend/models/government_schemes.py`
- Declared columns: `id (db.Integer)`, `scheme_code (db.String(50))`, `name (db.String(200))`, `ministry (db.String(150))`, `description (db.Text)`, `farm_size_min (db.Float)`, `farm_size_max (db.Float)`, `income_limit (db.Float)`, `required_crops (db.Text)`, `region_specific (db.Boolean)`, `eligible_states (db.Text)`, `subsidy_percentage (db.Float)`, `max_subsidy_amount (db.Float)`, `application_fee (db.Float)`, `application_start_date (db.Date)`, `application_deadline (db.Date)`, `is_active (db.Boolean)`, `required_documents (db.Text)`, `created_at (db.DateTime)`, `updated_at (db.DateTime)`

`SchemeApplication` (table `SchemeApplication`)

- Source: `backend/models/government_schemes.py`
- Declared columns: `id (db.Integer)`, `scheme_id (db.Integer)`, `user_id (db.Integer)`, `farm_id (db.Integer)`, `application_number (db.String(50))`, `status (db.String(20))`, `eligibility_score (db.Float)`, `eligibility_notes (db.Text)`, `farmer_details (db.Text)`, `farm_details (db.Text)`, `income_proof (db.Text)`, `submitted_documents (db.Text)`, `documents_verified (db.Boolean)`, `assistance_notes (db.Text)`, `assistance_contact (db.String(100))`, `submitted_at (db.DateTime)`, `reviewed_at (db.DateTime)`, `approved_at (db.DateTime)`, `rejection_reason (db.Text)`, `created_at (db.DateTime)`, `updated_at (db.DateTime)`

`SchemeReminder` (table `SchemeReminder`)

- Source: `backend/models/government_schemes.py`
- Declared columns: `id (db.Integer)`, `scheme_id (db.Integer)`, `user_id (db.Integer)`, `reminder_type (db.String(30))`, `reminder_date (db.DateTime)`, `message (db.Text)`, `is_sent (db.Boolean)`, `sent_at (db.DateTime)`, `created_at (db.DateTime)`

`InsurancePolicy` (table `InsurancePolicy`)

- Source: `backend/models/insurance.py`
- Declared columns: `id (db.Integer)`, `user_id (db.Integer)`, `policy_number (db.String(50))`, `coverage_amount (db.Numeric(15, 2))`, `premium_amount (db.Numeric(15, 2))`, `ars_score_at_issuance (db.Float)`, `risk_multiplier (db.Float)`, `crop_type (db.String(100))`, `farm_location (db.String(200))`, `farm_size_acres (db.Float)`, `issue_date (db.DateTime)`, `start_date (db.DateTime)`, `end_date (db.DateTime)`, `status (db.String(20))`, `notes (db.Text)`, `created_at (db.DateTime)`, `current_risk_score (db.Float)`, `is_suspended (db.Boolean)`, `suspension_reason (db.String(255))`, `recursive_discount_applied (db.Boolean)`, `force_majeure_review (db.Boolean)`

`DynamicPremiumLog` (table `DynamicPremiumLog`)

- Source: `backend/models/insurance.py`
- Declared columns: `id (db.Integer)`, `policy_id (db.Integer)`, `old_premium (db.Numeric(15, 2))`, `new_premium (db.Numeric(15, 2))`, `change_reason (db.String(255))`, `triggered_by (db.String(50))`, `timestamp (db.DateTime)`

`RiskFactorSnapshot` (table `RiskFactorSnapshot`)

- Source: `backend/models/insurance.py`

- Declared columns: `id (db.Integer)`, `policy_id (db.Integer)`, `weather_risk_index (db.Float)`, `telemetry_risk_index (db.Float)`, `sustainability_discount_factor (db.Float)`, `recorded_at (db.DateTime)`

`ClaimRequest` (table `ClaimRequest`)

- Source: `backend/models/insurance.py`
- Declared columns: `id (db.Integer)`, `policy_id (db.Integer)`, `user_id (db.Integer)`, `claim_number (db.String(50))`, `claimed_amount (db.Numeric(15, 2))`, `approved_amount (db.Numeric(15, 2))`, `incident_date (db.DateTime)`, `incident_description (db.Text)`, `evidence_photos (db.JSON)`, `status (db.String(20))`, `ai_verification_status (db.String(20))`, `rejection_reason (db.Text)`, `processed_at (db.DateTime)`, `payment_date (db.DateTime)`, `notes (db.Text)`, `created_at (db.DateTime)`

`RiskScoreHistory` (table `RiskScoreHistory`)

- Source: `backend/models/insurance.py`
- Declared columns: `id (db.Integer)`, `user_id (db.Integer)`, `ars_score (db.Float)`, `risk_category (db.String(20))`, `calculation_date (db.DateTime)`, `notes (db.Text)`

`CropPolicy` (table `CropPolicy`)

- Source: `backend/models/insurance_v2.py`
- Declared columns: `id (db.Integer)`, `user_id (db.Integer)`, `farm_id (db.Integer)`, `crop_type (db.String(50))`, `coverage_amount (db.Float)`, `premium_paid (db.Float)`, `start_date (db.Date)`, `end_date (db.Date)`, `risk_score (db.Float)`, `status (db.String(20))`, `parametric_enabled (db.Boolean)`, `yield_at_risk_pct (db.Float)`, `last_climate_check (db.DateTime)`, `created_at (db.DateTime)`

`ClaimRequest` (table `ClaimRequest`)

- Source: `backend/models/insurance_v2.py`
- Declared columns: `id (db.Integer)`, `policy_id (db.Integer)`, `claim_amount (db.Float)`, `reason (db.Text)`, `evidence_url (db.String(255))`, `status (db.String(20))`, `submitted_at (db.DateTime)`, `processed_at (db.DateTime)`

`AdjusterNote` (table `AdjusterNote`)

- Source: `backend/models/insurance_v2.py`
- Declared columns: `id (db.Integer)`, `claim_id (db.Integer)`, `adjuster_id (db.Integer)`, `content (db.Text)`, `verification_score (db.Float)`, `created_at (db.DateTime)`

`PayoutLedger` (table `PayoutLedger`)

- Source: `backend/models/insurance_v2.py`
- Declared columns: `id (db.Integer)`, `claim_id (db.Integer)`, `amount (db.Float)`, `transaction_ref (db.String(100))`, `paid_at (db.DateTime)`

`ParametricAutoSettlement` (table `ParametricAutoSettlement`)

- Source: `backend/models/insurance_v2.py`
- Declared columns: `id (db.Integer)`, `policy_id (db.Integer)`, `alert_id (db.Integer)`, `coverage_amount (db.Float)`, `payout_percentage (db.Float)`, `payout_amount (db.Float)`, `trigger_type (db.String(50))`, `consecutive_days_observed (db.Integer)`, `peak_temperature_c (db.Float)`, `evidence_hash (db.String(64))`, `ledger_transaction_id (db.Integer)`, `settled_at (db.DateTime)`

`AssetTelemetry` (table `AssetTelemetry`)

- Source: `backend/models/iot_maintenance.py`
- Declared columns: `id (db.Integer)`, `asset_id (db.Integer)`, `engine_rpm (db.Float)`, `coolant_temp_c (db.Float)`, `vibration_amplitude (db.Float)`, `fuel_pressure_psi (db.Float)`, `cumulative_hours (db.Float)`, `recorded_at (db.DateTime)`

`MaintenancePrediction` (table `MaintenancePrediction`)

- Source: `backend/models/iot_maintenance.py`
- Declared columns: `id (db.Integer)`, `asset_id (db.Integer)`, `failure_probability (db.Float)`, `estimated_remaining_useful_life_hrs (db.Float)`, `predicted_component_failure (db.String(100))`,

criticality_level (db.String(20)), generated_at (db.DateTime), status (db.String(20))

ComponentWearMap (table ComponentWearMap)

- Source: backend/models/iot_maintenance.py
- Declared columns: id (db.Integer), asset_id (db.Integer), component_name (db.String(100)), wear_percentage (db.Float), last_inspected_at (db.DateTime), replacement_threshold (db.Float)

IrrigationZone (table IrrigationZone)

- Source: backend/models/irrigation.py
- Declared columns: id (db.Integer), farm_id (db.Integer), name (db.String(100)), description (db.Text), moisture_threshold_min (db.Float), moisture_threshold_max (db.Float), current_valve_status (db.String(20)), last_activation (db.DateTime), auto_mode (db.Boolean), created_at (db.DateTime), banned_by_system (db.Boolean), fertigation_enabled (db.Boolean), fertigation_valve_status (db.String(20)), chemical_concentration (db.Float), washout_risk_threshold (db.Float), total_water_withdrawn (db.Float), electricity_usage_kwh (db.Float), pest_control_mode (db.Boolean), pest_neutralization_active (db.Boolean), target_pest_id (db.Integer)

FertigationLog (table FertigationLog)

- Source: backend/models/irrigation.py
- Declared columns: id (db.Integer), zone_id (db.Integer), injectant_type (db.String(50)), concentration_ppm (db.Float), volume_liters (db.Float), washout_risk_score (db.Float), weather_impact_factor (db.Float), timestamp (db.DateTime)

SensorLog (table SensorLog)

- Source: backend/models/irrigation.py
- Declared columns: id (db.Integer), zone_id (db.Integer), moisture (db.Float), temperature (db.Float), ph_level (db.Float), timestamp (db.DateTime)

IrrigationSchedule (table IrrigationSchedule)

- Source: backend/models/irrigation.py
- Declared columns: id (db.Integer), zone_id (db.Integer), start_time (db.Time), duration_minutes (db.Integer), days_of_week (db.String(50)), is_active (db.Boolean), last_run (db.DateTime)

AquiferLevel (table AquiferLevel)

- Source: backend/models/irrigation.py
- Declared columns: id (db.Integer), region_name (db.String(100)), current_depth_meters (db.Float), depletion_rate (db.Float), last_survey_date (db.DateTime)

WaterRightsQuota (table WaterRightsQuota)

- Source: backend/models/irrigation.py
- Declared columns: id (db.Integer), farm_id (db.Integer), aquifer_id (db.Integer), total_quota_liters (db.Float), used_quota_liters (db.Float), renewal_date (db.DateTime), status (db.String(20))

Question (table Question)

- Source: backend/models/knowledge.py
- Declared columns: id (db.Integer), user_id (db.Integer), title (db.String(255)), content (db.Text), category (db.String(50)), view_count (db.Integer), upvote_count (db.Integer), answer_count (db.Integer), is_closed (db.Boolean), has_accepted_answer (db.Boolean), created_at (db.DateTime), updated_at (db.DateTime)

Answer (table Answer)

- Source: backend/models/knowledge.py
- Declared columns: id (db.Integer), question_id (db.Integer), user_id (db.Integer), content (db.Text), upvote_count (db.Integer), is_accepted (db.Boolean), created_at (db.DateTime), updated_at (db.DateTime)

`KnowledgeVote` (table `KnowledgeVote`)

- Source: `backend/models/knowledge.py`
- Declared columns: `id (db.Integer)`, `user_id (db.Integer)`, `question_id (db.Integer)`, `answer_id (db.Integer)`, `vote_type (db.Integer)`, `created_at (db.DateTime)`

`Badge` (table `Badge`)

- Source: `backend/models/knowledge.py`
- Declared columns: `id (db.Integer)`, `name (db.String(50))`, `description (db.String(255))`, `icon_url (db.String(255))`, `criteria_json (db.Text)`

`UserBadge` (table `UserBadge`)

- Source: `backend/models/knowledge.py`
- Declared columns: `id (db.Integer)`, `user_id (db.Integer)`, `badge_id (db.Integer)`, `awarded_at (db.DateTime)`

`UserExpertise` (table `UserExpertise`)

- Source: `backend/models/knowledge.py`
- Declared columns: `id (db.Integer)`, `user_id (db.Integer)`, `category (db.String(50))`, `is_verified (db.Boolean)`, `verification_doc_url (db.String(255))`, `reputation_score (db.Integer)`, `created_at (db.DateTime)`

`WorkerProfile` (table `WorkerProfile`)

- Source: `backend/models/labor.py`
- Declared columns: `id (db.Integer)`, `farm_id (db.Integer)`, `name (db.String(150))`, `contact_info (db.String(255))`, `worker_type (db.String(50))`, `base_hourly_rate (db.Float)`, `piece_rate_kg (db.Float)`, `bank_details (db.Text)`, `is_active (db.Boolean)`, `created_at (db.DateTime)`

`WorkerSkillMap` (table `WorkerSkillMap`)

- Source: `backend/models/labor.py`
- Declared columns: `id (db.Integer)`, `worker_id (db.Integer)`, `skill_name (db.String(100))`, `certification_level (db.String(20))`, `expiry_date (db.Date)`

`WorkShift` (table `WorkShift`)

- Source: `backend/models/labor.py`
- Declared columns: `id (db.Integer)`, `worker_id (db.Integer)`, `start_time (db.DateTime)`, `end_time (db.DateTime)`, `break_duration (db.Integer)`, `total_hours (db.Float)`, `shift_status (db.String(20))`, `notes (db.Text)`

`HarvestLog` (table `HarvestLog`)

- Source: `backend/models/labor.py`
- Declared columns: `id (db.Integer)`, `worker_id (db.Integer)`, `crop_type (db.String(100))`, `quantity_kg (db.Float)`, `logged_at (db.DateTime)`, `is_processed (db.Boolean)`, `payroll_id (db.Integer)`

`PayrollEntry` (table `PayrollEntry`)

- Source: `backend/models/labor.py`
- Declared columns: `id (db.Integer)`, `worker_id (db.Integer)`, `period_start (db.Date)`, `period_end (db.Date)`, `gross_hourly_pay (db.Float)`, `gross_piece_pay (db.Float)`, `overtime_premium (db.Float)`, `bonus_amount (db.Float)`, `tax_deduction (db.Float)`, `advances_deduction (db.Float)`, `net_payable (db.Float)`, `status (db.String(20))`, `payment_ref (db.String(100))`, `processed_at (db.DateTime)`

`LaborROIHistory` (table `LaborROIHistory`)

- Source: `backend/models/labor.py`
- Declared columns: `id (db.Integer)`, `worker_id (db.Integer)`, `period_start (db.Date)`, `period_end (db.Date)`, `total_hours_worked (db.Float)`, `total_yield_kg (db.Float)`, `labor_cost (db.Float)`, `yield_value_estimate (db.Float)`, `roi_percentage (db.Float)`, `calculated_at (db.DateTime)`

`LedgerAccount` (table `LedgerAccount`)

- Source: `backend/models/ledger.py`
- Declared columns: `id (db.Integer)`, `account_code (db.String(20))`, `name (db.String(100))`, `account_type (db.Enum(AccountType))`, `currency (db.String(3))`, `parent_id (db.Integer)`, `entity_type (db.String(50))`, `entity_id (db.Integer)`, `is_active (db.Boolean)`, `is_system (db.Boolean)`, `description (db.Text)`, `created_at (db.DateTime)`, `updated_at (db.DateTime)`

`LedgerTransaction` (table `LedgerTransaction`)

- Source: `backend/models/ledger.py`
- Declared columns: `id (db.Integer)`, `transaction_id (db.String(36))`, `transaction_type (db.Enum(TransactionType))`, `source_type (db.String(50))`, `source_id (db.Integer)`, `description (db.Text)`, `reference_number (db.String(100))`, `base_currency (db.String(3))`, `base_amount (db.Numeric(18, 6))`, `fx_rate_used (db.Numeric(18, 8))`, `fx_rate_date (db.DateTime)`, `created_by (db.Integer)`, `created_at (db.DateTime)`, `is_reversed (db.Boolean)`, `reversed_by_id (db.Integer)`

`LedgerEntry` (table `LedgerEntry`)

- Source: `backend/models/ledger.py`
- Declared columns: `id (db.Integer)`, `transaction_id (db.Integer)`, `account_id (db.Integer)`, `entry_type (db.Enum(EntryType))`, `amount (db.Numeric(18, 6))`, `currency (db.String(3))`, `base_amount (db.Numeric(18, 6))`, `base_currency (db.String(3))`, `fx_rate (db.Numeric(18, 8))`, `entry_date (db.DateTime)`, `memo (db.String(500))`, `metadata_json (db.JSON)`, `created_at (db.DateTime)`

`FXValuationSnapshot` (table `FXValuationSnapshot`)

- Source: `backend/models/ledger.py`
- Declared columns: `id (db.Integer)`, `entity_type (db.String(50))`, `entity_id (db.Integer)`, `snapshot_date (db.DateTime)`, `currency (db.String(3))`, `position_amount (db.Numeric(18, 6))`, `original_fx_rate (db.Numeric(18, 8))`, `current_fx_rate (db.Numeric(18, 8))`, `base_currency (db.String(3))`, `original_base_value (db.Numeric(18, 6))`, `current_base_value (db.Numeric(18, 6))`, `unrealized_gain_loss (db.Numeric(18, 6))`, `unrealized_gain_loss_pct (db.Numeric(10, 4))`, `period_start (db.DateTime)`, `period_end (db.DateTime)`, `cumulative_realized_gain (db.Numeric(18, 6))`, `cumulative_unrealized_gain (db.Numeric(18, 6))`, `created_at (db.DateTime)`

`Vault` (table `Vault`)

- Source: `backend/models/ledger.py`
- Declared columns: `id (db.Integer)`, `vault_id (db.String(36))`, `name (db.String(100))`, `owner_type (db.String(50))`, `owner_id (db.Integer)`, `base_currency (db.String(3))`, `ledger_account_id (db.Integer)`, `allow_multi_currency (db.Boolean)`, `auto_fx_revaluation (db.Boolean)`, `is_active (db.Boolean)`, `is_locked (db.Boolean)`, `locked_reason (db.String(200))`, `created_at (db.DateTime)`, `updated_at (db.DateTime)`

`VaultCurrencyPosition` (table `VaultCurrencyPosition`)

- Source: `backend/models/ledger.py`
- Declared columns: `id (db.Integer)`, `vault_id (db.Integer)`, `currency (db.String(3))`, `ledger_account_id (db.Integer)`, `cost_basis_rate (db.Numeric(18, 8))`, `cost_basis_amount (db.Numeric(18, 6))`, `last_fx_rate (db.Numeric(18, 8))`, `last_revaluation_date (db.DateTime)`, `cumulative_realized_fx_gain (db.Numeric(18, 6))`, `cumulative_unrealized_fx_gain (db.Numeric(18, 6))`, `created_at (db.DateTime)`, `updated_at (db.DateTime)`

`FXRate` (table `FXRate`)

- Source: `backend/models/ledger.py`
- Declared columns: `id (db.Integer)`, `from_currency (db.String(3))`, `to_currency (db.String(3))`, `rate (db.Numeric(18, 8))`, `rate_date (db.Date)`, `rate_time (db.DateTime)`, `source (db.String(50))`, `is_current (db.Boolean)`, `created_at (db.DateTime)`

`LoanRequest` (table `LoanRequest`)

- Source: `backend/models/loan_request.py`
- Declared columns: `id (db.Integer)`, `user_id (db.Integer)`, `loan_type (db.String(100))`, `amount_requested (db.Float)`, `purpose (db.Text)`, `applicant_data (db.Text)`, `eligibility_status (db.String(20))`, `assessment_result`

(db.Text), recommended_amount (db.Float), risk_score (db.Float), status (db.String(20)), submitted_at (db.DateTime), processed_at (db.DateTime), created_at (db.DateTime), updated_at (db.DateTime), report_generated (db.Boolean), report_path (db.String(255)), report_sent (db.Boolean)

RepaymentSchedule (table RepaymentSchedule)

- Source: backend/models/loan_v2.py
- Declared columns: id (db.Integer), loan_request_id (db.Integer), installment_number (db.Integer), due_date (db.Date), principal_amount (db.Float), interest_amount (db.Float), total_emi (db.Float), productivity_index_snapshot (db.Float), adjusted_interest_rate (db.Float), outstanding_balance (db.Float), is_paid (db.Boolean), created_at (db.DateTime)

PaymentHistory (table PaymentHistory)

- Source: backend/models/loan_v2.py
- Declared columns: id (db.Integer), loan_request_id (db.Integer), schedule_id (db.Integer), amount_paid (db.Float), payment_date (db.Date), payment_method (db.String(50)), transaction_ref (db.String(100)), late_fee (db.Float), penalty_interest (db.Float), created_at (db.DateTime)

DefaultRiskScore (table DefaultRiskScore)

- Source: backend/models/loan_v2.py
- Declared columns: id (db.Integer), loan_request_id (db.Integer), risk_score (db.Float), probability_default (db.Float), days_overdue (db.Integer), payment_consistency (db.Float), calculated_at (db.DateTime)

CollectionNote (table CollectionNote)

- Source: backend/models/loan_v2.py
- Declared columns: id (db.Integer), loan_request_id (db.Integer), created_by (db.Integer), note_type (db.String(50)), content (db.Text), created_at (db.DateTime)

DriverProfile (table DriverProfile)

- Source: backend/models/logistics_v2.py
- Declared columns: id (db.Integer), user_id (db.Integer), license_number (db.String(50)), contact_phone (db.String(20)), status (db.String(20)), rating (db.Float), total_trips (db.Integer), created_at (db.DateTime)

DeliveryVehicle (table DeliveryVehicle)

- Source: backend/models/logistics_v2.py
- Declared columns: id (db.Integer), plate_number (db.String(20)), vehicle_type (db.String(50)), capacity_kg (db.Float), fuel_type (db.String(20)), avg_fuel_consumption (db.Float), status (db.String(20)), last_maintenance_date (db.DateTime), mileage (db.Float), created_at (db.DateTime)

TransportRoute (table TransportRoute)

- Source: backend/models/logistics_v2.py
- Declared columns: id (db.Integer), driver_id (db.Integer), vehicle_id (db.Integer), origin (db.String(255)), destination (db.String(255)), origin_coords (db.String(100)), destination_coords (db.String(100)), estimated_distance (db.Float), actual_distance (db.Float), status (db.String(20)), start_time (db.DateTime), end_time (db.DateTime), cargo_weight (db.Float), cargo_description (db.Text), created_at (db.DateTime)

FuelLog (table FuelLog)

- Source: backend/models/logistics_v2.py
- Declared columns: id (db.Integer), vehicle_id (db.Integer), fuel_quantity (db.Float), cost (db.Float), mileage_at_refill (db.Float), carbon_footprint_kg (db.Float), recorded_at (db.DateTime)

PhytoSanitaryCertificate (table PhytoSanitaryCertificate)

- Source: backend/models/logistics_v2.py
- Declared columns: id (db.Integer), route_id (db.Integer), batch_id (db.Integer), certificate_number (db.String(64)), issuing_authority (db.String(150)), origin_country (db.String(80)), destination_country (db.String(80)), commodity (db.String(100)), declared_quantity_kg (db.Float), certificate_payload_json

(db.Text), signature_hash (db.String(64)), status (db.String(20)), issued_at (db.DateTime), valid_until (db.DateTime)

FreightEscrow (table FreightEscrow)

- Source: backend/models/logistics_v2.py
- Declared columns: id (db.Integer), route_id (db.Integer), driver_id (db.Integer), total_freight_amount (db.Float), currency (db.String(3)), destination_lat (db.Float), destination_lng (db.Float), geo_fence_radius_meters (db.Float), confirmed_delivery_lat (db.Float), confirmed_delivery_lng (db.Float), geo_fence_passed (db.Boolean), delivery_proof_hash (db.String(64)), status (db.String(20)), release_ledger_txn_id (db.Integer), base_price (db.Float), fuel_surcharge (db.Float), customs_delay_penalty (db.Float), final_amount (db.Float), created_at (db.DateTime), released_at (db.DateTime)

CustomsCheckpoint (table CustomsCheckpoint)

- Source: backend/models/logistics_v2.py
- Declared columns: id (db.Integer), route_id (db.Integer), checkpoint_name (db.String(150)), country (db.String(80)), arrived_at (db.DateTime), cleared_at (db.DateTime), wait_hours (db.Float), status (db.String(30)), phyto_cert_id (db.Integer), inspector_notes (db.Text)

GPSTelemetry (table GPSTelemetry)

- Source: backend/models/logistics_v2.py
- Declared columns: id (db.Integer), route_id (db.Integer), vehicle_id (db.Integer), latitude (db.Float), longitude (db.Float), speed_kmh (db.Float), heading_degrees (db.Float), fuel_price_per_liter (db.Float), recorded_at (db.DateTime)

EngineHourLog (table EngineHourLog)

- Source: backend/models/machinery.py
- Declared columns: id (db.Integer), equipment_id (db.Integer), booking_id (db.Integer), hours_start (db.Float), hours_end (db.Float), logged_at (db.DateTime)

MaintenanceCycle (table MaintenanceCycle)

- Source: backend/models/machinery.py
- Declared columns: id (db.Integer), equipment_id (db.Integer), service_type (db.String(100)), interval_hours (db.Float), last_service_hour (db.Float), status (db.String(20)), next_due_hour (db.Float), created_at (db.DateTime)

DamageReport (table DamageReport)

- Source: backend/models/machinery.py
- Declared columns: id (db.Integer), equipment_id (db.Integer), booking_id (db.Integer), description (db.Text), estimated_repair_cost (db.Float), escrow_hold_amount (db.Float), is_disputed (db.Boolean), reported_at (db.DateTime)

RepairOrder (table RepairOrder)

- Source: backend/models/machinery.py
- Declared columns: id (db.Integer), damage_report_id (db.Integer), service_center (db.String(150)), actual_cost (db.Float), status (db.String(20)), started_at (db.DateTime), completed_at (db.DateTime)

AssetValueSnapshot (table AssetValueSnapshot)

- Source: backend/models/machinery.py
- Declared columns: id (db.Integer), equipment_id (db.Integer), current_book_value (db.Float), depreciation_accumulated (db.Float), liquidation_priority (db.Integer), hours_impact_factor (db.Float), maintenance_health_bonus (db.Float), calculated_at (db.DateTime)

ComponentWearMap (table ComponentWearMap)

- Source: backend/models/machinery.py

- Declared columns: `id (db.Integer)`, `equipment_id (db.Integer)`, `component_name (db.String(100))`, `current_wear_percentage (db.Float)`, `critical_threshold (db.Float)`, `last_inspection_date (db.DateTime)`, `predicted_failure_date (db.DateTime)`

`MaintenanceEscrow` (table `MaintenanceEscrow`)

- Source: `backend/models/machinery.py`
- Declared columns: `id (db.Integer)`, `farm_id (db.Integer)`, `equipment_id (db.Integer)`, `escrow_balance (db.Float)`, `projected_cost (db.Float)`, `is_locked (db.Boolean)`, `last_sync (db.DateTime)`

`MarketPrice` (table `MarketPrice`)

- Source: `backend/models/market.py`
- Declared columns: `id (db.Integer)`, `crop_name (db.String(100))`, `commodity_code (db.String(50))`, `variety (db.String(100))`, `market (db.String(150))`, `district (db.String(100))`, `state (db.String(100))`, `latitude (db.Float)`, `longitude (db.Float)`, `modal_price (db.Float)`, `min_price (db.Float)`, `max_price (db.Float)`, `arrivals_tonnes (db.Float)`, `unit (db.String(30))`, `price_date (db.Date)`, `created_at (db.DateTime)`, `updated_at (db.DateTime)`

`PriceWatchlist` (table `PriceWatchlist`)

- Source: `backend/models/market.py`
- Declared columns: `id (db.Integer)`, `user_id (db.String(80))`, `crop_name (db.String(100))`, `target_price (db.Float)`, `mandi_name (db.String(150))`, `district (db.String(100))`, `phone_number (db.String(25))`, `alert_channel (db.String(30))`, `alert_enabled (db.Boolean)`, `is_triggered (db.Boolean)`, `triggered_at (db.DateTime)`, `created_at (db.DateTime)`

`ForwardContract` (table `ForwardContract`)

- Source: `backend/models/market.py`
- Declared columns: `id (db.Integer)`, `farm_id (db.Integer)`, `batch_id (db.Integer)`, `crop_type (db.String(100))`, `estimated_quantity (db.Float)`, `locked_price_per_unit (db.Float)`, `maturity_date (db.DateTime)`, `status (db.String(30))`, `hedge_ratio (db.Float)`, `force_majeure_suspended (db.Boolean)`, `climate_risk_discount (db.Float)`, `yield_at_risk_pct (db.Float)`, `quality_penalty_clause (db.Float)`, `created_at (db.DateTime)`

`PriceHedgingLog` (table `PriceHedgingLog`)

- Source: `backend/models/market.py`
- Declared columns: `id (db.Integer)`, `farm_id (db.Integer)`, `action (db.String(80))`, `old_hedge_ratio (db.Float)`, `new_hedge_ratio (db.Float)`, `trigger_reason (db.Text)`, `market_price_snapshot (db.Float)`, `timestamp (db.DateTime)`

`MarketplaceListing` (table `MarketplaceListing`)

- Source: `backend/models/marketplace.py`
- Declared columns: `id (db.Integer)`, `seller_id (db.Integer)`, `product_name (db.String(200))`, `description (db.Text)`, `category (db.String(100))`, `quantity (db.Float)`, `unit (db.String(50))`, `price_per_unit (db.Float)`, `location (db.String(200))`, `is_available (db.Boolean)`, `is_organic (db.Boolean)`, `images (db.JSON)`, `created_at (db.DateTime)`, `updated_at (db.DateTime)`

`MarketplaceOrder` (table `MarketplaceOrder`)

- Source: `backend/models/marketplace.py`
- Declared columns: `id (db.Integer)`, `listing_id (db.Integer)`, `buyer_id (db.Integer)`, `seller_id (db.Integer)`, `quantity (db.Float)`, `total_price (db.Float)`, `delivery_address (db.String(500))`, `delivery_status (db.String(50))`, `tracking_number (db.String(100))`, `status (db.String(50))`, `created_at (db.DateTime)`, `updated_at (db.DateTime)`

`MarketplaceRating` (table `MarketplaceRating`)

- Source: `backend/models/marketplace.py`
- Declared columns: `id (db.Integer)`, `order_id (db.Integer)`, `rater_id (db.Integer)`, `rated_user_id (db.Integer)`, `rating (db.Integer)`, `review (db.Text)`, `created_at (db.DateTime)`

`MediaPayload` (table `MediaPayload`)

- Source: `backend/models/media_payload.py`
- Declared columns: `id (db.Integer)`, `user_id (db.Integer)`, `tracking_id (db.String(100))`, `payload_type (db.String(50))`, `filename (db.String(255))`, `file_path (db.String(512))`, `file_type (db.String(100))`, `file_size (db.Integer)`, `status (db.String(20))`, `task_id (db.String(100))`, `processed_at (db.DateTime)`, `result_data (db.Text)`, `error_log (db.Text)`, `metadata_json (db.Text)`, `created_at (db.DateTime)`

`Notification` (table `Notification`)

- Source: `backend/models/misc.py`
- Declared columns: `id (db.Integer)`, `user_id (db.Integer)`, `title (db.String(100))`, `message (db.Text)`, `type (db.String(50))`, `read_at (db.DateTime)`, `sent_at (db.DateTime)`

`File` (table `File`)

- Source: `backend/models/misc.py`
- Declared columns: `id (db.Integer)`, `user_id (db.Integer)`, `filename (db.String(255))`, `original_name (db.String(255))`, `file_path (db.String(512))`, `file_type (db.String(100))`, `mime_type (db.String(100))`, `file_size (db.Integer)`, `storage_type (db.String(20))`, `created_at (db.DateTime)`

`YieldPool` (table `YieldPool`)

- Source: `backend/models/misc.py`
- Declared columns: `id (db.Integer)`, `pool_id (db.String(50))`, `pool_name (db.String(200))`, `crop_type (db.String(100))`, `target_quantity (db.Float)`, `current_quantity (db.Float)`, `status (db.String(20))`, `min_price_per_ton (db.Float)`, `current_offer_price (db.Float)`, `buyer_name (db.String(200))`, `collection_location (db.String(200))`, `logistics_overhead_percent (db.Float)`, `created_at (db.DateTime)`, `locked_at (db.DateTime)`, `completed_at (db.DateTime)`, `distributed_at (db.DateTime)`

`PoolContribution` (table `PoolContribution`)

- Source: `backend/models/misc.py`
- Declared columns: `id (db.Integer)`, `pool_id (db.Integer)`, `user_id (db.Integer)`, `quantity_tons (db.Float)`, `quality_grade (db.String(20))`, `contribution_percentage (db.Float)`, `estimated_value (db.Float)`, `actual_payout (db.Float)`, `payout_status (db.String(20))`, `contributed_at (db.DateTime)`, `paid_at (db.DateTime)`

`ResourceShare` (table `ResourceShare`)

- Source: `backend/models/misc.py`
- Declared columns: `id (db.Integer)`, `pool_id (db.Integer)`, `owner_id (db.Integer)`, `resource_type (db.String(100))`, `resource_name (db.String(200))`, `resource_value (db.Float)`, `usage_cost_per_hour (db.Float)`, `is_free_for_pool (db.Boolean)`, `availability_status (db.String(20))`, `shared_at (db.DateTime)`, `last_used_at (db.DateTime)`

`PoolVote` (table `PoolVote`)

- Source: `backend/models/misc.py`
- Declared columns: `id (db.Integer)`, `pool_id (db.Integer)`, `user_id (db.Integer)`, `vote (db.String(10))`, `offer_price (db.Float)`, `comment (db.Text)`, `voted_at (db.DateTime)`

`WaterStressIndex` (table `WaterStressIndex`)

- Source: `backend/models/precision_irrigation.py`
- Declared columns: `id (db.Integer)`, `farm_id (db.Integer)`, `zone_id (db.String(50))`, `moisture_level_pct (db.Float)`, `evapotranspiration_rate (db.Float)`, `root_zone_salinity (db.Float)`, `stress_score (db.Float)`, `irrigation_required_liters (db.Float)`, `recorded_at (db.DateTime)`

`IrrigationValveAutomation` (table `IrrigationValveAutomation`)

- Source: `backend/models/precision_irrigation.py`
- Declared columns: `id (db.Integer)`, `farm_id (db.Integer)`, `valve_id (db.String(50))`, `status (db.String(20))`, `flow_rate_lpm (db.Float)`, `total_volume_injected_l (db.Float)`, `last_action (db.String(50))`, `last_action_at (db.DateTime)`, `triggered_by_event_id (db.Integer)`

`AquiferMonitoring` (table `AquiferMonitoring`)

- Source: `backend/models/precision_irrigation.py`
- Declared columns: `id (db.Integer)`, `aquifer_id (db.String(50))`, `current_water_level_m (db.Float)`, `recharge_rate_lps (db.Float)`, `is_critical_depletion (db.Boolean)`, `recorded_at (db.DateTime)`

`PredictionHistory` (table `PredictionHistory`)

- Source: `backend/models/prediction_history.py`
- Declared columns: `id (db.Integer)`, `user_id (db.Integer)`, `prediction_type (db.String(50))`, `input_data (db.Text)`, `result (db.Text)`, `confidence (db.Float)`, `created_at (db.DateTime)`, `ip_address (db.String(45))`, `user_agent (db.String(255))`, `processing_time (db.Float)`, `model_version (db.String(50))`

`ProcessingBatch` (table `ProcessingBatch`)

- Source: `backend/models/processing.py`
- Declared columns: `id (db.Integer)`, `batch_number (db.String(50))`, `product_type (db.String(100))`, `total_weight (db.Float)`, `current_weight (db.Float)`, `current_stage (db.String(20))`, `origin_farms (db.Text)`, `created_at (db.DateTime)`, `updated_at (db.DateTime)`, `completed_at (db.DateTime)`

`StageLog` (table `StageLog`)

- Source: `backend/models/processing.py`
- Declared columns: `id (db.Integer)`, `batch_id (db.Integer)`, `stage_name (db.String(20))`, `started_at (db.DateTime)`, `ended_at (db.DateTime)`, `operator_id (db.Integer)`, `notes (db.Text)`

`QualityCheck` (table `QualityCheck`)

- Source: `backend/models/processing.py`
- Declared columns: `id (db.Integer)`, `batch_id (db.Integer)`, `stage_name (db.String(20))`, `moisture_content (db.Float)`, `purity_level (db.Float)`, `weight_recorded (db.Float)`, `is_passed (db.Boolean)`, `auditor_id (db.Integer)`, `comments (db.Text)`, `timestamp (db.DateTime)`

`SpectralScanData` (table `SpectralScanData`)

- Source: `backend/models/processing.py`
- Declared columns: `id (db.Integer)`, `batch_id (db.Integer)`, `moisture_percentage (db.Float)`, `brix_level (db.Float)`, `protein_percentage (db.Float)`, `fiber_percentage (db.Float)`, `wavelength_range (db.String(50))`, `scan_integrity_score (db.Float)`, `timestamp (db.DateTime)`

`DynamicGradeAdjustment` (table `DynamicGradeAdjustment`)

- Source: `backend/models/processing.py`
- Declared columns: `id (db.Integer)`, `batch_id (db.Integer)`, `old_grade (db.String(10))`, `new_grade (db.String(10))`, `price_penalty_factor (db.Float)`, `adjustment_reason (db.String(255))`, `timestamp (db.DateTime)`

`VendorProfile` (table `VendorProfile`)

- Source: `backend/models/procurement.py`
- Declared columns: `id (db.Integer)`, `user_id (db.Integer)`, `company_name (db.String(100))`, `registration_id (db.String(50))`, `rating (db.Float)`, `is_verified (db.Boolean)`, `created_at (db.DateTime)`

`ProcurementItem` (table `ProcurementItem`)

- Source: `backend/models/procurement.py`
- Declared columns: `id (db.Integer)`, `vendor_id (db.Integer)`, `name (db.String(100))`, `category (db.String(50))`, `base_price (db.Float)`, `unit (db.String(20))`, `stock (db.Float)`, `volume_pricing (db.Text)`, `created_at (db.DateTime)`

`BulkOrder` (table `BulkOrder`)

- Source: `backend/models/procurement.py`
- Declared columns: `id (db.Integer)`, `buyer_id (db.Integer)`, `vendor_id (db.Integer)`, `item_id (db.Integer)`, `quantity (db.Float)`, `unit_price (db.Float)`, `total_amount (db.Float)`, `currency (db.String(3))`, `forex_rate (db.Float)`,

gst_rate (db.Float), vat_rate (db.Float), circular_credits_used (db.Float), credit_discount_amount (db.Float), tax_amount (db.Float), shipping_cost (db.Float), real_time_price_modifier (db.Float), freight_escrow_id (db.Integer), dynamic_freight_surcharge (db.Float), customs_clearance_status (db.String(20)), funds_in_escrow (db.Boolean), status (db.String(20)), delivery_address (db.Text), tracking_number (db.String(100)), created_at (db.DateTime), updated_at (db.DateTime)

OrderEvent (table OrderEvent)

- Source: backend/models/procurement.py
- Declared columns: id (db.Integer), order_id (db.Integer), from_status (db.String(20)), to_status (db.String(20)), comment (db.Text), timestamp (db.DateTime)

ReliabilityLog (table ReliabilityLog)

- Source: backend/models/reliability_log.py
- Declared columns: id (db.Integer), equipment_id (db.Integer), recorded_at (db.DateTime), vibration_peak (db.Float), temp_peak (db.Float), oil_pressure (db.Float), calculated_reliability_score (db.Float), trigger_event (db.String(50))

SoilTest (table SoilTest)

- Source: backend/models/soil_health.py
- Declared columns: id (db.Integer), farm_id (db.Integer), test_date (db.Date), nitrogen (db.Float), phosphorus (db.Float), potassium (db.Float), latitude (db.Float), longitude (db.Float), depth_cm (db.Float), nitrogen_flux_index (db.Float), phosphorus_flux_index (db.Float), leaching_susceptibility (db.Float), estimated_n2o_flux (db.Float), volatile_organic_compounds (db.Float), ph_level (db.Float), organic_matter (db.Float), electrical_conductivity (db.Float), soil_hardness_index (db.Float), calcium (db.Float), magnesium (db.Float), sulfur (db.Float), lab_name (db.String(150)), report_url (db.String(255)), created_at (db.DateTime)

FertilizerRecommendation (table FertilizerRecommendation)

- Source: backend/models/soil_health.py
- Declared columns: id (db.Integer), soil_test_id (db.Integer), crop_type (db.String(100)), rec_nitrogen (db.Float), rec_phosphorus (db.Float), rec_potassium (db.Float), lime_requirement (db.Float), suggested_fertilizers (db.Text), created_at (db.DateTime)

ApplicationLog (table ApplicationLog)

- Source: backend/models/soil_health.py
- Declared columns: id (db.Integer), soil_test_id (db.Integer), applied_by (db.Integer), fertilizer_name (db.String(100)), amount_applied (db.Float), area_covered (db.Float), applied_at (db.DateTime), notes (db.Text)

RegenerativeFarmingLog (table RegenerativeFarmingLog)

- Source: backend/models/soil_health.py
- Declared columns: id (db.Integer), farm_id (db.Integer), soil_test_id (db.Integer), practice_type (db.String(50)), area_hectares (db.Float), soil_organic_carbon_percent (db.Float), bulk_density_gcm3 (db.Float), sampling_depth_cm (db.Float), estimated_co2e_tonnes (db.Float), logged_at (db.DateTime), verified (db.Boolean)

CarbonMintEvent (table CarbonMintEvent)

- Source: backend/models/soil_health.py
- Declared columns: id (db.Integer), farm_id (db.Integer), log_id (db.Integer), credits_minted (db.Float), credit_unit_value_usd (db.Float), total_value_usd (db.Float), mint_hash (db.String(64)), ledger_transaction_id (db.Integer), listed_on_market (db.Boolean), buyer_id (db.Integer), sold_at (db.DateTime), sale_price_usd (db.Float), minted_at (db.DateTime)

SpatialYieldGrid (table SpatialYieldGrid)

- Source: backend/models/spatial_yield.py
- Declared columns: id (db.Integer), region_id (db.String(50)), bounding_box_wkt (db.Text), avg_soil_moisture_pct (db.Float), normalized_difference_vegetation_index (db.Float), canopy_temperature_c (db.Float), projected_biomass_density (db.Float), total_yield_potential_kg (db.Float), last_satellite_pass (db.DateTime),

created_at (db.DateTime)

TemporalYieldForex (table TemporalYieldForex)

- Source: backend/models/spatial_yield.py
- Declared columns: id (db.Integer), grid_id (db.Integer), crop_type (db.String(50)), seasonality_vector (db.JSON), growth_acceleration_factor (db.Float), base_yield_kg_per_hectare (db.Float), predicted_harvest_date (db.Date), confidence_level (db.String(30)), engine_version (db.String(20)), timestamp (db.DateTime)

CarbonPractice (table CarbonPractice)

- Source: backend/models/sustainability.py
- Declared columns: id (db.Integer), name (db.String(100)), description (db.Text), carbon_offset_per_unit (db.Float)

CreditLedger (table CreditLedger)

- Source: backend/models/sustainability.py
- Declared columns: id (db.Integer), farm_id (db.Integer), credits_earned (db.Float), credits_spent (db.Float), timestamp (db.DateTime)

AuditRequest (table AuditRequest)

- Source: backend/models/sustainability.py
- Declared columns: id (db.Integer), farm_id (db.Integer), status (db.String(20)), created_at (db.DateTime)

CarbonLedger (table CarbonLedger)

- Source: backend/models/sustainability.py
- Declared columns: id (db.Integer), farm_id (db.Integer), batch_id (db.Integer), scope_1_direct (db.Float), scope_2_indirect (db.Float), scope_3_supply_chain (db.Float), total_footprint (db.Float), sequestration_offset (db.Float), net_carbon_balance (db.Float), certification_status (db.String(50)), recorded_at (db.DateTime)

EmissionSource (table EmissionSource)

- Source: backend/models/sustainability.py
- Declared columns: id (db.Integer), ledger_id (db.Integer), source_type (db.String(50)), ref_id (db.Integer), emission_value (db.Float), calculation_method (db.String(100)), timestamp (db.DateTime)

SustainabilityScore (table SustainabilityScore)

- Source: backend/models/sustainability.py
- Declared columns: id (db.Integer), farm_id (db.Integer), carbon_efficiency (db.Float), water_efficiency (db.Float), biodiversity_index (db.Float), overall_rating (db.Float), circular_economy_bonus (db.Float), offset_credits_available (db.Float), water_quota_utilization_ratio (db.Float), esg_carbon_score (db.Float), total_credits_minted (db.Float), updated_at (db.DateTime)

ESGMarketListing (table ESGMarketListing)

- Source: backend/models/sustainability.py
- Declared columns: id (db.Integer), farm_id (db.Integer), mint_event_id (db.Integer), credits_offered (db.Float), asking_price_usd (db.Float), status (db.String(20)), description (db.Text), listed_at (db.DateTime), expires_at (db.DateTime), buyer_user_id (db.Integer), purchase_price_usd (db.Float), purchased_at (db.DateTime), settlement_ledger_txn_id (db.Integer)

Token (table Token)

- Source: backend/models/token.py
- Declared columns: id (db.Integer), user_id (db.Integer), token (db.String(128)), type (db.String(32)), expires_at (db.DateTime), created_at (db.DateTime)

SupplyBatch (table SupplyBatch)

- Source: backend/models/traceability.py

- Declared columns: `id (db.Integer)`, `batch_internal_id (db.String(50))`, `qr_code_data (db.Text)`, `crop_name (db.String(100))`, `crop_variety (db.String(100))`, `quantity (db.Float)`, `unit (db.String(20))`, `status (db.String(20))`, `harvest_date (db.DateTime)`, `expiry_date (db.DateTime)`, `farm_location (db.String(255))`, `current_gps_lat (db.Float)`, `current_gps_lng (db.Float)`, `farmer_id (db.Integer)`, `current_handler_id (db.Integer)`, `distributor_id (db.Integer)`, `retailer_id (db.Integer)`, `soil_test_id (db.Integer)`, `insurance_policy_id (db.Integer)`, `is_certified (db.Boolean)`, `certificate_url (db.String(255))`, `integrity_hash (db.String(64))`, `export_compliance_status (db.String(20))`, `target_market (db.String(50))`, `esg_score (db.Float)`, `phyto_cert_id (db.String(100))`, `carbon_footprint_hash (db.String(64))`, `net_zero_qualified (db.Boolean)`, `carbon_saved_kg (db.Float)`, `predicted_quality_grade (db.String(10))`, `quarantine_status (db.String(20))`, `bio_clearance_hash (db.String(64))`, `contact_tracing_metadata (db.Text)`, `created_at (db.DateTime)`, `updated_at (db.DateTime)`

`FarmLifecycleLog` (table `FarmLifecycleLog`)

- Source: `backend/models/traceability.py`
- Declared columns: `id (db.Integer)`, `batch_id (db.Integer)`, `event_type (db.String(60))`, `event_title (db.String(150))`, `stage_category (db.String(50))`, `performed_by (db.String(100))`, `location_name (db.String(255))`, `gps_lat (db.Float)`, `gps_lng (db.Float)`, `payload_json (db.Text)`, `notes (db.Text)`, `carbon_impact_kg (db.Float)`, `previous_hash (db.String(64))`, `block_hash (db.String(64))`, `digital_signature (db.String(128))`, `timestamp (db.DateTime)`

`QualityGrade` (table `QualityGrade`)

- Source: `backend/models/traceability.py`
- Declared columns: `id (db.Integer)`, `batch_id (db.Integer)`, `inspector_id (db.Integer)`, `grade (db.String(10))`, `parameters (db.Text)`, `notes (db.Text)`, `inspection_date (db.DateTime)`

`CustodyLog` (table `CustodyLog`)

- Source: `backend/models/traceability.py`
- Declared columns: `id (db.Integer)`, `batch_id (db.Integer)`, `handler_id (db.Integer)`, `action (db.String(100))`, `from_status (db.String(20))`, `to_status (db.String(20))`, `location (db.String(255))`, `gps_lat (db.Float)`, `gps_lng (db.Float)`, `notes (db.Text)`, `digital_signature (db.String(255))`, `timestamp (db.DateTime)`

`ProduceReview` (table `ProduceReview`)

- Source: `backend/models/transparency.py`
- Declared columns: `id (db.Integer)`, `user_id (db.Integer)`, `stock_item_id (db.Integer)`, `rating (db.Integer)`, `comment (db.Text)`, `processing_batch_id (db.Integer)`, `supply_batch_id (db.Integer)`, `farmer_id (db.Integer)`, `is_processed_for_reputation (db.Boolean)`, `created_at (db.DateTime)`

`PriceAdjustmentLog` (table `PriceAdjustmentLog`)

- Source: `backend/models/transparency.py`
- Declared columns: `id (db.Integer)`, `stock_item_id (db.Integer)`, `old_price (db.Float)`, `new_price (db.Float)`, `freshness_score (db.Float)`, `adjustment_reason (db.String(100))`, `timestamp (db.DateTime)`

`User` (table `User`)

- Source: `backend/models/user.py`
- Declared columns: `id (db.Integer)`, `username (db.String(80))`, `email (db.String(120))`, `password_hash (db.String(255))`, `full_name (db.String(120))`, `role (db.String(20))`, `phone (db.String(20))`, `location (db.String(100))`, `farm_latitude (db.Float)`, `farm_longitude (db.Float)`, `created_at (db.DateTime)`, `last_login (db.DateTime)`, `is_active (db.Boolean)`, `language_preference (db.String(10))`

`PathogenStrain` (table `PathogenStrain`)

- Source: `backend/models/virulence.py`
- Declared columns: `id (db.Integer)`, `disease_id (db.Integer)`, `strain_designation (db.String(100))`, `infectivity_rate (db.Float)`, `spore_dispersal_radius_km (db.Float)`, `pesticide_resistance_factor (db.Float)`, `anti_drought_gene_exploit (db.Float)`, `defense_bypass_capability (db.Float)`, `mutation_generation (db.Integer)`, `first_detected (db.DateTime)`, `extinct_marker (db.Boolean)`

`InfectionCombatSimulation` (table `InfectionCombatSimulation`)

- Source: `backend/models/virulence.py`
- Declared columns: `id (db.Integer)`, `strain_id (db.Integer)`, `phenotype_id (db.Integer)`, `base_attack_power (db.Float)`, `crop_defense_power (db.Float)`, `environmental_modifier (db.Float)`, `infection_success (db.Boolean)`, `damage_inflicted_pct (db.Float)`, `triggered_new_mutation (db.Boolean)`, `simulated_at (db.DateTime)`

`WarehouseLocation` (table `WarehouseLocation`)

- Source: `backend/models/warehouse.py`
- Declared columns: `id (db.Integer)`, `name (db.String(150))`, `address (db.Text)`, `capacity (db.Float)`, `manager_id (db.Integer)`, `created_at (db.DateTime)`

`StockItem` (table `StockItem`)

- Source: `backend/models/warehouse.py`
- Declared columns: `id (db.Integer)`, `warehouse_id (db.Integer)`, `item_name (db.String(150))`, `category (db.String(50))`, `sku (db.String(50))`, `current_quantity (db.Float)`, `unit (db.String(20))`, `reorder_point (db.Float)`, `reorder_quantity (db.Float)`, `batch_number (db.String(50))`, `processing_batch_id (db.Integer)`, `supply_batch_id (db.Integer)`, `base_price (db.Float)`, `current_market_price (db.Float)`, `freshness_score (db.Float)`, `expiry_date (db.Date)`, `created_at (db.DateTime)`

`StockMovement` (table `StockMovement`)

- Source: `backend/models/warehouse.py`
- Declared columns: `id (db.Integer)`, `stock_item_id (db.Integer)`, `movement_type (db.String(20))`, `quantity (db.Float)`, `reference_doc (db.String(100))`, `reason (db.Text)`, `performed_by (db.Integer)`, `movement_date (db.DateTime)`

`ReconciliationLog` (table `ReconciliationLog`)

- Source: `backend/models/warehouse.py`
- Declared columns: `id (db.Integer)`, `warehouse_id (db.Integer)`, `reconciliation_date (db.Date)`, `total_items_checked (db.Integer)`, `discrepancies_found (db.Integer)`, `shrinkage_value (db.Float)`, `notes (db.Text)`, `performed_by (db.Integer)`, `created_at (db.DateTime)`

`WeatherData` (table `WeatherData`)

- Source: `backend/models/weather.py`
- Declared columns: `id (db.Integer)`, `location (db.String(100))`, `latitude (db.Float)`, `longitude (db.Float)`, `temperature (db.Float)`, `humidity (db.Float)`, `rainfall (db.Float)`, `wind_speed (db.Float)`, `wind_direction (db.Float)`, `weather_condition (db.String(50))`, `is_forecast (db.Boolean)`, `timestamp (db.DateTime)`

`CropAdvisory` (table `CropAdvisory`)

- Source: `backend/models/weather.py`
- Declared columns: `id (db.Integer)`, `user_id (db.Integer)`, `crop_name (db.String(50))`, `location (db.String(100))`, `advisory_text (db.Text)`, `growth_stage (db.String(50))`, `priority (db.String(20))`, `weather_summary (db.Text)`, `soil_summary (db.Text)`, `is_read (db.Boolean)`, `feedback_rating (db.Integer)`, `created_at (db.DateTime)`

`AdvisorySubscription` (table `AdvisorySubscription`)

- Source: `backend/models/weather.py`
- Declared columns: `id (db.Integer)`, `user_id (db.Integer)`, `crop_name (db.String(50))`, `location (db.String(100))`, `soil_type (db.String(50))`, `sowing_date (db.Date)`, `is_active (db.Boolean)`, `frequency (db.String(20))`, `created_at (db.DateTime)`

`RiskTrigger` (table `RiskTrigger`)

- Source: `backend/models/weather.py`
- Declared columns: `id (db.Integer)`, `crop_type (db.String(100))`, `max_temp_threshold (db.Float)`, `min_temp_threshold (db.Float)`, `max_rainfall_threshold (db.Float)`, `max_wind_speed (db.Float)`, `severity_level (db.String(20))`, `is_active (db.Boolean)`, `created_at (db.DateTime)`

`ClimateTelemetryEvent` (table `ClimateTelemetryEvent`)

- Source: `backend/models/weather.py`
- Declared columns: `id (db.Integer)`, `farm_id (db.Integer)`, `location (db.String(150))`, `temperature_c (db.Float)`, `rainfall_mm (db.Float)`, `humidity_pct (db.Float)`, `wind_speed_kmh (db.Float)`, `solar_radiation_wm2 (db.Float)`, `is_extreme (db.Boolean)`, `extreme_type (db.String(50))`, `source (db.String(50))`, `recorded_at (db.DateTime)`

`ForceMajeureAlert` (table `ForceMajeureAlert`)

- Source: `backend/models/weather.py`
- Declared columns: `id (db.Integer)`, `farm_id (db.Integer)`, `trigger_type (db.String(50))`, `threshold_value (db.Float)`, `consecutive_days (db.Integer)`, `status (db.String(20))`, `auto_settled (db.Boolean)`, `settlement_ledger_txn_id (db.Integer)`, `triggered_at (db.DateTime)`, `resolved_at (db.DateTime)`

`ParametricPolicyTrigger` (table `ParametricPolicyTrigger`)

- Source: `backend/models/weather.py`
- Declared columns: `id (db.Integer)`, `policy_id (db.Integer)`, `trigger_type (db.String(50))`, `threshold_value (db.Float)`, `required_consecutive_days (db.Integer)`, `payout_percentage (db.Float)`, `is_active (db.Boolean)`, `created_at (db.DateTime)`

Relationship and ER overview (conceptual)

Mermaid diagram source

```
erDiagram
 USER ||--o{ FILE : owns
 USER ||--o{ NOTIFICATION : receives
 USER ||--o{ AUDIT_LOG : performs
 USER ||--o{ PREDICTION_HISTORY : creates
 FARM ||--o{ FARM_MEMBER : includes
 FARM ||--o{ FARM_ASSET : contains
 EQUIPMENT ||--o{ RENTAL_BOOKING : booked
 FORUM_THREAD ||--o{ POST_COMMENT : has
 SUPPLY_BATCH ||--o{ CUSTODY_LOG : tracks
```

This is a conceptual relationship map of common model families; validate actual foreign key declarations in model source before treating every line as a guaranteed relationship.

Transactions, indexes and seeds

Commit/rollback patterns appear in services. Unique fields and indexes are declared model-by-model; no single schema manifest or comprehensive seed command was verified. No global transaction diagram should be inferred beyond each endpoint's actual writes.

API to database mapping

The user model is used by auth; file metadata and user relations are represented in core models; disease incidents and outbreak zones use disease models; market, soil, irrigation, farm and finance routes use their matching model families. A route may be read-only or may not persist its result. Confirm the handler/service calls before asserting writes.

07 Authentication Authorization

Authentication and Authorization

Important: multiple mechanisms exist

The repository contains legacy JWT helpers (`auth_utils.py`), a versioned auth Blueprint (`backend/api/v1/auth.py` using `AuthService`), and some routes using Flask-JWT-Extended decorators. Authentication is not one uniform global system. Trace a specific endpoint to know which token and decorator it accepts.

Registration

The versioned auth route is mounted below `/api/v1/auth`. It accepts username, email and password; checks required values, enforces a Gmail suffix, checks duplicates, creates a `User`, stores a Werkzeug password hash, commits, sends a verification email and logs an audit event. Mail delivery depends on SMTP config. The `User` model also enforces Gmail and name-character constraints. Password minimum/complexity policy must be checked in the actual service and should be strengthened if absent.

Login and token lifecycle

`backend/api/v1/auth.py` defines login and related auth endpoints. `AuthService` performs account behavior; `auth_utils.py` separately encodes/decodes JWT with HS256 and provides `token_required` / `roles_required`. Some login implementations may return access tokens and refresh cookies; verify exact handler and frontend usage before making security claims. Do not store or display token values.

Mermaid diagram source

```
sequenceDiagram
 participant U as User
 participant UI as Login form
 participant A as Auth route/service
 participant DB as User table
 U->>UI: enter email/name and password
 UI->>A: POST credentials over HTTPS in deployment
 A->>DB: find account
 DB-->>A: stored password hash and role
 A->>A: verify password hash; issue token if valid
 A-->>UI: success/token or error
 UI->>A: protected request with accepted token
 A->>A: validate signature, expiry, identity and role
 A-->>UI: protected data or 401/403
```

Password reset and logout

The UI route `/reset-password/<token>` serves `reset-password.html`; this proves a page exists, not that a complete secure reset-token issuance and one-time consumption path is active. Search the auth service and verify expiration, single-use semantics and email validation before presenting it as complete. Logout/revocation behavior depends on which token path is used.

Authorization

Authentication answers “who are you?”; authorization answers “may you do this?” Role guards in `auth_utils.py` accept role lists. Other modules use `jwt_required`. A User role field does not automatically protect routes. Audit all write/admin/file routes for explicit authorization and ownership checks.

Security findings and improvements

- `auth_utils.py` contains a development fallback JWT secret; deployments must require a long unique secret and fail closed.
- Some APIs use different JWT implementations; standardize issuer, audience, expiry, refresh, revocation and error behavior.
- Cookie refresh tokens need Secure, HttpOnly, SameSite and CSRF review.
- Gmail-only validation is product-specific and excludes other valid users.
- Ensure account enumeration and password reset responses do not reveal sensitive state.

08 API Documentation

API Documentation

How to read endpoints

Methods and paths are exact only when verified in decorators and Blueprint registration. API v1 is mounted at `/api/v1`; child prefixes compose with route strings. Parameters vary by endpoint. The source route appendix below is generated from all Python route decorators in scanned project files and links each to its source line. It includes standalone sub-apps too, so not every endpoint is served by the main `app.py`.

Important main-server journeys

Method	Effective endpoint	Purpose / inputs	Auth / result notes
GET	`/`	Main UI (`index.html`)	HTML page
GET	`/farmer`, `/shopkeeper`, `/main`, `/about`, `/blog`, `/contact`, `/chat`	Serve named root pages	HTML
GET	`/yield/`	Yield form with trained model choices	503 if required artifact missing
POST	`/yield/predict`	Form values: crop, season, state, year, area, rainfall, fertilizer, pesticide	Server renders result; 400 invalid fields
GET/POST	`/crop/`, `/crop/predict`	Crop form and recommendation	Crop blueprint under `/crop`; model/encoder artifacts required
POST	`/api/crop/predict-async`	N,P,K,temperature,humidity,pH,rain fall	JWT and farmer/admin/consultant; 202 response with task ID
GET	`/api/task/<task_id>`	Celery status/result	JWT protected
POST	`/api/recommend/fertilizer`	soil_ph, crop_type, optional growth_stage/recent_weather	JWT role guard; JSON recommendation
POST	`/api/v1/ai-disease/predict-trained`	base64 image, optional crop_type	Public with rate limit; classifier result or 400/503/500
POST	`/api/v1/ai-disease/analyze`	image, crop_type, optional location/coordinates	token_required; performs advisory and saves incident
GET	`/api/v1/ai-disease/history`	optional crop_type, days	token_required; user history
GET	`/api/v1/ai-disease/treatment`	crop_type, disease_name	no auth decorator visible; treatment response
POST/GET	`/api/v1/disease/incidents`	disease/crop/severity/symptoms/coordinates; GET supports filters	Flask-JWT-Extended protected
GET	`/api/firebase-config`	no body	rate limited; returns public Firebase client config keys, not private service credentials

These examples are not a substitute for the code contract. Use each route's schema/docstring and implementation for exact required fields and response variants.

Request/response example: trained disease classifier

Request: `POST /api/v1/ai-disease/predict-trained`, JSON `{ "image": "<base64 or data URL>", "crop_type": "Tomato" }`. Success: `{ "status":"success", "prediction": { "class_name":"...", "crop":"...", "disease":"...", "confidence":0, "alternatives":[...], "model":"...", "device":"..." } }`. Scores are raw softmax outputs, not calibrated probabilities. Error cases include missing/invalid image (400), missing model artifact (503) and inference failure (500).

Authentication and response convention

Some endpoints return JSON with `status`, `message`, `data` or `error`; HTML forms return a rendered template. Error status codes are not globally uniform. `Authorization` requirements are route-specific.

Route decorator inventory (repository source paths)

This generated appendix lists route decorators found in Python source. Paths inside Blueprints may receive additional prefixes at registration. `app.url_map` at runtime is authoritative for effective URLs. This inventory is source level and does not certify endpoint health.

`Community/app.py`

- Line 12: `@app.route("/")` → `index`
- Line 16: `@app.route("/communities", methods=['GET'])` → `get_communities`
- Line 21: `@app.route("/add_community", methods=['POST'])` → `add_community`
- Line 31: `@app.route("/delete_community", methods=['POST'])` → `delete_community`

`Crop Yield Prediction/crop_yield_app/app.py`

- Line 94: `@app.route('/')` → `index`
- Line 98: `@app.route('/predict', methods=['POST'])` → `predict`

`Crop_Planning/app.py`

- Line 91: `@app.route('/')` → `home`
- Line 95: `@app.route('/predict', methods=['POST'])` → `predict`

`Crop_Prices_Tracker/app.py`

- Line 82: `@tracker_bp.route('/crop_price_tracker', methods=['GET', 'POST'])` → `crop_price_tracker`
- Line 140: `@tracker_bp.route('/get_states')` → `get_states`
- Line 155: `@tracker_bp.route('/get_markets')` → `get_markets`
- Line 177: `@tracker_bp.route('/api/forecast', methods=['POST', 'GET'])` → `api_forecast`
- Line 194: `@tracker_bp.route('/api/optimize-profit', methods=['POST'])` → `api_optimize_profit`
- Line 216: `@tracker_bp.route('/api/heatmap', methods=['GET'])` → `api_heatmap`
- Line 226: `@tracker_bp.route('/api/alert-subscribe', methods=['POST'])` → `api_alert_subscribe`

`Disease prediction/app.py`

- Line 13: `@app.route('/')` → `index`
- Line 17: `@app.route('/predict', methods=['POST'])` → `predict`

`Forum/app.py`

- Line 49: `@app.route('/forum', methods=['GET', 'POST'])` → `forum_api`

`Labour_Alerts/app.py`

- Line 18: `@app.route('/news')` → `get_news`

`Plant Seedlings Classification/api.py`

- Line 77: `@app.get("/", response_model=HealthResponse)` → `health_check`
- Line 78: `@app.get("/health", response_model=HealthResponse)` → `health_check`
- Line 89: `@app.get("/models")` → `list_models`
- Line 98: `@app.get("/species")` → `list_species`
- Line 107: `@app.post("/predict", response_model=PredictionResponse)` → `predict_seedling`

`app.py`

- Line 159: `@app.route('/api/firebase-config')` → `get_firebase_config`
- Line 182: `@app.route('/api/task/<task_id>', methods=['GET'])` → `get_task_status`
- Line 217: `@app.route('/api/crop/predict-async', methods=['POST'])` → `predict_crop_async`

- Line 252: `@app.route('/api/recommend/fertilizer', methods=['POST'])` → `recommend_fertilizer_endpoint`
- Line 273: `@app.route('/api/loan/process-async', methods=['POST'])` → `process_loan_async`
- Line 308: `@app.route('/process-loan', methods=['POST'])` → `process_loan`
- Line 411: `@app.route('/generate-loan-report', methods=['POST'])` → `generate_loan_report_endpoint`
- Line 488: `@app.route('/download-report/<filename>', methods=['GET'])` → `download_report`
- Line 501: `@app.route('/task-status/<task_id>', methods=['GET'])` → `get_task_status_public`
- Line 545: `@app.route('/')` → `index`
- Line 549: `@app.route('/farmer')` → `farmer`
- Line 553: `@app.route('/shopkeeper')` → `shopkeeper`
- Line 557: `@app.route('/main')` → `main`
- Line 561: `@app.route('/about')` → `about`
- Line 565: `@app.route('/blog')` → `blog`
- Line 569: `@app.route('/contact')` → `contact`
- Line 573: `@app.route('/chat')` → `chat`
- Line 590: `@app.route('/yield/', methods=['GET'])` → `yield_prediction_page`
- Line 601: `@app.route('/yield/predict', methods=['POST'])` → `predict_yield`
- Line 624: `@app.route('/Crop Yield Prediction/crop_yield_app/templates/index.html')` → `legacy_yield_prediction_page`
- Line 625: `@app.route('/Crop Yield Prediction/crop_yield_app/templates/input.html')` → `legacy_yield_prediction_page`
- Line 629: `@app.route('/Crop_Prices_Tracker/templates/crop_price_tracker.html')` → `legacy_crop_price_tracker_page`
- Line 634: `@app.route('/reset-password/<token>')` → `reset_password_page`
- Line 638: `@app.route('/<path:filename>')` → `serve_static`
- Line 669: `@app.route('/rotation')` → `rotation_page`

`backend/api/ingestion.py`

- Line 8: `@ingestion_bp.route('/ingest/upload', methods=['POST'])` → `upload_media`
- Line 48: `@ingestion_bp.route('/ingest/status/<tracking_id>', methods=['GET'])` → `get_status`

`backend/api/v1/advisories.py`

- Line 9: `@advisories_bp.route('/', methods=['GET'])` → `get_advisories`
- Line 20: `@advisories_bp.route('/generate', methods=['POST'])` → `manual_generate`
- Line 43: `@advisories_bp.route('/<int:advisory_id>/read', methods=['PATCH'])` → `mark_read`
- Line 54: `@advisories_bp.route('/<int:advisory_id>/feedback', methods=['POST'])` → `submit_feedback`

`backend/api/v1/ai_disease.py`

- Line 10: `@ai_disease_bp.route("/predict-trained", methods=["POST"])` → `analyze_image_public`
- Line 31: `@ai_disease_bp.route("/analyze", methods=["POST"])` → `analyze_image`
- Line 73: `@ai_disease_bp.route("/history", methods=["GET"])` → `get_history`
- Line 86: `@ai_disease_bp.route("/treatment", methods=["GET"])` → `get_treatment`

`backend/api/v1/answers.py`

- Line 7: `@answers_bp.route('/', methods=['POST'])` → `add_answer`
- Line 30: `@answers_bp.route('/<int:answer_id>/vote', methods=['POST'])` → `vote_answer`
- Line 39: `@answers_bp.route('/<int:answer_id>/accept', methods=['POST'])` → `accept_answer`

backend/api/v1/arbitrage.py

- Line 8: `@arbitrage_bp.route('/opportunities', methods=['GET'])` → `list_active_arbitrage`
- Line 24: `@arbitrage_bp.route('/trades', methods=['GET'])` → `get_executed_bot_trades`
- Line 48: `@arbitrage_bp.route('/scan', methods=['POST'])` → `force_matrix_scan`

backend/api/v1/assets.py

- Line 23: `@assets_bp.route('/register', methods=['POST'])` → `register_asset`
- Line 78: `@assets_bp.route('/telemetry/<asset_id>', methods=['POST'])` → `update_telemetry`
- Line 140: `@assets_bp.route('/predict-failure/<asset_id>', methods=['POST'])` → `predict_failure`
- Line 183: `@assets_bp.route('/maintenance-log/<asset_id>', methods=['POST'])` → `log_maintenance`
- Line 237: `@assets_bp.route('/my-assets', methods=['GET'])` → `get_my_assets`
- Line 281: `@assets_bp.route('/summary', methods=['GET'])` → `get_asset_summary`
- Line 306: `@assets_bp.route('/critical', methods=['GET'])` → `get_critical_assets`
- Line 337: `@assets_bp.route('/<asset_id>', methods=['GET'])` → `get_asset_details`
- Line 374: `@assets_bp.route('/<asset_id>', methods=['PUT'])` → `update_asset`
- Line 438: `@assets_bp.route('/<asset_id>', methods=['DELETE'])` → `delete_asset`

backend/api/v1/audit.py

- Line 7: `@audit_bp.route('/logs', methods=['GET'])` → `get_audit_logs`
- Line 30: `@audit_bp.route('/sessions', methods=['GET'])` → `get_active_sessions`
- Line 43: `@audit_bp.route('/stats', methods=['GET'])` → `get_audit_stats`

backend/api/v1/auth.py

- Line 10: `@auth_bp.route('/register', methods=['POST'])` → `handler name not found`
- Line 79: `@auth_bp.route('/verify-email/<token>', methods=['GET'])` → `handler name not found`
- Line 99: `@auth_bp.route('/forgot-password', methods=['POST'])` → `handler name not found`
- Line 146: `@auth_bp.route('/reset-password/<token>', methods=['POST'])` → `handler name not found`
- Line 187: `@auth_bp.route('/reset-password/<token>/validate', methods=['GET'])` → `validate_reset_password_token`

backend/api/v1/barter.py

- Line 9: `@barter_bp.route('/quote', methods=['POST'])` → `get_quote`
- Line 25: `@barter_bp.route('/propose', methods=['POST'])` → `propose_trade`
- Line 43: `@barter_bp.route('/<int:tx_id>/commit', methods=['POST'])` → `lock_side`
- Line 55: `@barter_bp.route('/<int:tx_id>/confirm-fulfillment', methods=['POST'])` → `confirm_delivery`
- Line 65: `@barter_bp.route('/history', methods=['GET'])` → `list_barters`

backend/api/v1/biomass.py

- Line 8: `@biomass_bp.route('/stockpile/declare', methods=['POST'])` → `declare_stockpile`
- Line 31: `@biomass_bp.route('/digester/burn', methods=['POST'])` → `trigger_combustion`
- Line 52: `@biomass_bp.route('/farm/<int:farm_id>/logs', methods=['GET'])` → `get_digest_logs`

backend/api/v1/biosecurity.py

- Line 11: `@biosecurity_bp.route('/heatmap', methods=['GET'])` → `get_outbreak_heatmap`
- Line 35: `@biosecurity_bp.route('/containment/trigger', methods=['POST'])` → `trigger_blockade`
- Line 51: `@biosecurity_bp.route('/clearance/generate/<int:batch_id>', methods=['POST'])` → `generate_clearance`

backend/api/v1/bookings.py

- Line 8: `@bookings_bp.route('/', methods=['POST'])` → `create_booking`
- Line 35: `@bookings_bp.route('/<int:booking_id>/status', methods=['PATCH'])` → `update_booking_status`
- Line 59: `@bookings_bp.route('/my-bookings', methods=['GET'])` → `get_my_bookings`

backend/api/v1/bot_webhook.py

- Line 21: `@bot_webhook_bp.route("/webhook/twilio", methods=["POST", "GET"])` → `twilio_webhook`
- Line 47: `@bot_webhook_bp.route("/webhook/meta", methods=["GET"])` → `meta_webhook_verify`
- Line 67: `@bot_webhook_bp.route("/webhook/meta", methods=["POST"])` → `meta_webhook_messages`
- Line 86: `@bot_webhook_bp.route("/query", methods=["POST"])` → `bot_query`
- Line 189: `@bot_webhook_bp.route("/stt", methods=["POST"])` → `speech_to_text_endpoint`
- Line 234: `@bot_webhook_bp.route("/tts", methods=["POST"])` → `text_to_speech_endpoint`
- Line 263: `@bot_webhook_bp.route("/audio/<filename>", methods=["GET"])` → `serve_bot_audio`
- Line 279: `@bot_webhook_bp.route("/status", methods=["GET"])` → `bot_status`

backend/api/v1/carbon.py

- Line 24: `@carbon_bp.route('/calculate', methods=['POST'])` → `calculate_sequestration`
- Line 54: `@carbon_bp.route('/impact', methods=['GET'])` → `get_farmer_impact`
- Line 70: `@carbon_bp.route('/practices', methods=['POST'])` → `log_practice`
- Line 97: `@carbon_bp.route('/mint', methods=['POST'])` → `mint_carbon_credits`
- Line 119: `@carbon_bp.route('/rewards-catalog', methods=['GET'])` → `get_rewards_catalog`
- Line 130: `@carbon_bp.route('/redeem', methods=['POST'])` → `redeem_eco_reward`
- Line 150: `@carbon_bp.route('/leaderboard', methods=['GET'])` → `get_leaderboard`

backend/api/v1/carbon_v2.py

- Line 9: `@carbon_v2_bp.route('/escrow/listings', methods=['GET'])` → `get_escrow_listings`
- Line 24: `@carbon_v2_bp.route('/wallet/balance', methods=['GET'])` → `get_carbon_balance`
- Line 41: `@carbon_v2_bp.route('/escrow/fund', methods=['POST'])` → `fund_carbon_trade`

backend/api/v1/circular.py

- Line 9: `@circular_bp.route('/dashboard/<int:farm_id>', methods=['GET'])` → `get_circular_dashboard`
- Line 48: `@circular_bp.route('/waste/report', methods=['POST'])` → `report_waste`

backend/api/v1/climate_control.py

- Line 99: `@climate_bp.route('/field-polygons/<int:farm_id>', methods=['GET'])` → `get_field_polygons`
- Line 139: `@climate_bp.route('/irrigation/calculate', methods=['POST'])` → `calculate_irrigation_schedule`
- Line 210: `@climate_bp.route('/early-warnings/evaluate', methods=['POST', 'GET'])` → `evaluate_early_warnings`
- Line 246: `@climate_bp.route('/satellite-soil-sync', methods=['POST', 'GET'])` → `sync_satellite_soil`
- Line 264: `@climate_bp.route('/telemetry/<string:node_uid>', methods=['POST'])` → `post_telemetry`
- Line 273: `@climate_bp.route('/zones/<int:farm_id>', methods=['GET'])` → `get_farm_zones`
- Line 289: `@climate_bp.route('/analytics/<int:zone_id>', methods=['GET'])` → `get_analytics`

backend/api/v1/config.py

- Line 8: `@config_bp.route('/config', methods=['GET'])` → `get_public_config`
- Line 9: `@config_bp.route('/config/public', methods=['GET'])` → `get_public_config`
- Line 29: `@config_bp.route('/config/features', methods=['GET'])` → `get_feature_flags`

- Line 45: `@config_bp.route('/config/firebase', methods=['GET'])` → `get_firebase_config`

backend/api/v1/contributions.py

- Line 16: `@contributions_bp.route('', methods=['POST'])` → `add_contribution`
- Line 51: `@contributions_bp.route('/pool/<int:pool_id>', methods=['GET'])` → `get_pool_contributions`
- Line 69: `@contributions_bp.route('/user/<int:user_id>', methods=['GET'])` → `get_user_contributions`
- Line 100: `@contributions_bp.route('/<int:contribution_id>', methods=['GET'])` → `get_contribution`
- Line 124: `@contributions_bp.route('/estimate', methods=['POST'])` → `estimate_contribution_value`
- Line 151: `@contributions_bp.route('/user/<int:user_id>/earnings', methods=['GET'])` → `get_user_earnings`
- Line 170: `@contributions_bp.route('/<int:contribution_id>/payout', methods=['POST'])` → `trigger_payout`
- Line 189: `@contributions_bp.route('/stats', methods=['GET'])` → `get_contribution_stats`

backend/api/v1/credits.py

- Line 8: `@credits_bp.route('/my-ledger', methods=['GET'])` → `get_ledger`
- Line 17: `@credits_bp.route('/issue/<int:practice_id>', methods=['POST'])` → `issue_credits`
- Line 38: `@credits_bp.route('/marketplace', methods=['GET'])` → `list_available_credits`

backend/api/v1/crop_advisory.py

- Line 9: `@advisory_bp.route("/recommendations", methods=["GET"])` → `get_recommendations`
- Line 23: `@advisory_bp.route("/alerts", methods=["GET"])` → `get_advisory_alerts`
- Line 31: `@advisory_bp.route("/climate-analysis", methods=["GET"])` → `get_climate_analysis`
- Line 45: `@advisory_bp.route("/seasonal-plan", methods=["GET"])` → `get_seasonal_plan`
- Line 61: `@advisory_bp.route("/subscribe", methods=["POST"])` → `subscribe_advisory`

backend/api/v1/dashboard.py

- Line 6: `@dashboard_bp.route('/dashboard/summary', methods=['GET'])` → `get_dashboard_summary`

backend/api/v1/diagnostics_v2.py

- Line 9: `@diagnostics_v2_bp.route('/reports/history', methods=['GET'])` → `get_diagnostic_history`
- Line 20: `@diagnostics_v2_bp.route('/knowledge-base/pathogen', methods=['GET'])` → `get_pathogen_info`
- Line 38: `@diagnostics_v2_bp.route('/verify-report/<int:report_id>', methods=['POST'])` → `verify_ai_report`

backend/api/v1/disease.py

- Line 20: `@disease_bp.route('/incidents', methods=['POST'])` → `report_disease_incident`
- Line 97: `@disease_bp.route('/incidents', methods=['GET'])` → `get_disease_incidents`
- Line 168: `@disease_bp.route('/incidents/<incident_id>', methods=['GET'])` → `get_disease_incident`
- Line 188: `@disease_bp.route('/outbreaks', methods=['GET'])` → `get_outbreak_zones`
- Line 250: `@disease_bp.route('/outbreaks/<zone_id>', methods=['GET'])` → `get_outbreak_zone_details`
- Line 280: `@disease_bp.route('/my-risk', methods=['GET'])` → `check_my_risk`
- Line 330: `@disease_bp.route('/incidents/geojson', methods=['GET'])` → `get_incidents_geojson`
- Line 357: `@disease_bp.route('/outbreaks/geojson', methods=['GET'])` → `get_outbreaks_geojson`
- Line 382: `@disease_bp.route('/alerts', methods=['GET'])` → `get_my_alerts`
- Line 413: `@disease_bp.route('/alerts/<alert_id>/acknowledge', methods=['POST'])` → `acknowledge_alert`

backend/api/v1/energy_grid.py

- Line 8: `@energy_grid_bp.route('/', methods=['GET'])` → `get_grid_status`
- Line 30: `@energy_grid_bp.route('/farm/<int:farm_id>/tokens', methods=['GET'])` → `get_farm_energy_tokens`

backend/api/v1/equipment.py

- Line 8: `@equipment_bp.route('/', methods=['GET'])` → `get_equipment`
- Line 21: `@equipment_bp.route('/<int:equipment_id>', methods=['GET'])` → `get_equipment_detail`
- Line 31: `@equipment_bp.route('/', methods=['POST'])` → `list_equipment`

backend/api/v1/farm_members.py

- Line 9: `@farm_members_bp.route('/<int:farm_id>/members', methods=['GET'])` → `list_members`
- Line 19: `@farm_members_bp.route('/<int:farm_id>/invite', methods=['POST'])` → `invite_member`
- Line 55: `@farm_members_bp.route('/<int:member_id>', methods=['DELETE'])` → `remove_member`

backend/api/v1/farms.py

- Line 10: `@farms_bp.route('/', methods=['GET'])` → `get_my_farms`
- Line 20: `@farms_bp.route('/', methods=['POST'])` → `create_farm`
- Line 53: `@farms_bp.route('/<int:farm_id>/analytics', methods=['GET'])` → `get_farm_analytics`
- Line 69: `@farms_bp.route('/<int:farm_id>/assets', methods=['POST'])` → `add_asset`

backend/api/v1/files.py

- Line 8: `@files_bp.route('/upload', methods=['POST'])` → `upload_file`
- Line 32: `@files_bp.route('/<int:file_id>', methods=['GET'])` → `get_file_info`
- Line 42: `@files_bp.route('/download/<int:file_id>', methods=['GET'])` → `download_local_file`
- Line 56: `@files_bp.route('/user/<int:user_id>', methods=['GET'])` → `list_user_files`

backend/api/v1/financials.py

- Line 9: `@financials_bp.route('/dashboard/<int:farm_id>', methods=['GET'])` → `get_financial_dashboard`
- Line 31: `@financials_bp.route('/risk-audit', methods=['POST'])` → `manual_risk_audit`
- Line 45: `@financials_bp.route('/liquidity-projections/<int:farm_id>', methods=['GET'])` → `get_projections`

backend/api/v1/forum.py

- Line 13: `@forum_bp.route('/forum/categories', methods=['GET'])` → `get_categories`
- Line 26: `@forum_bp.route('/forum/categories', methods=['POST'])` → `create_category`
- Line 53: `@forum_bp.route('/forum/threads', methods=['GET'])` → `get_threads`
- Line 89: `@forum_bp.route('/forum/threads/<int:thread_id>', methods=['GET'])` → `get_thread`
- Line 106: `@forum_bp.route('/forum/threads', methods=['POST'])` → `create_thread`
- Line 147: `@forum_bp.route('/forum/threads/<int:thread_id>/comments', methods=['POST'])` → `create_comment`
- Line 189: `@forum_bp.route('/forum/threads/<int:thread_id>/upvote', methods=['POST'])` → `upvote_thread`
- Line 209: `@forum_bp.route('/forum/comments/<int:comment_id>/upvote', methods=['POST'])` → `upvote_comment`
- Line 229: `@forum_bp.route('/forum/trending', methods=['GET'])` → `get_trending`
- Line 248: `@forum_bp.route('/forum/unanswered', methods=['GET'])` → `get_unanswered`
- Line 267: `@forum_bp.route('/forum/search-ai', methods=['GET'])` → `ai_search`
- Line 289: `@forum_bp.route('/forum/flag', methods=['POST'])` → `flag_content`
- Line 318: `@forum_bp.route('/forum/reputation/<int:user_id>', methods=['GET'])` → `get_reputation`

backend/api/v1/futures.py

- Line 11: `@futures_bp.route('/liquidity/<int:farm_id>', methods=['GET'])` → `get_yield_liquidity`
- Line 36: `@futures_bp.route('/contract/lock', methods=['POST'])` → `lock_forward_contract`
- Line 79: `@futures_bp.route('/hedging-history/<int:farm_id>', methods=['GET'])` → `get_hedging_history`

- Line 96: `@futures_bp.route('/yield-at-risk/<int:farm_id>', methods=['GET'])` → `get_yield_at_risk`
- Line 109: `@futures_bp.route('/contract/<int:contract_id>/climate-status', methods=['GET'])` → `get_contract_climate_status`

backend/api/v1/genomics.py

- Line 8: `@genomics_bp.route('/seed-profiles', methods=['GET'])` → `get_seed_profiles`
- Line 18: `@genomics_bp.route('/spawn-phenotype', methods=['POST'])` → `plant_phenotype`
- Line 37: `@genomics_bp.route('/epigenetic-drifts', methods=['GET'])` → `view_drift_logs`

backend/api/v1/gews.py

- Line 9: `@gews_bp.route('/heatmap', methods=['GET'])` → `get_heatmap`
- Line 19: `@gews_bp.route('/zone/<int:zone_id>/simulate', methods=['POST'])` → `manual_simulate`
- Line 33: `@gews_bp.route('/zones/containment', methods=['GET'])` → `list_active_containments`

backend/api/v1/government_schemes.py

- Line 10: `@gov_schemes_bp.route("/government-schemes", methods=["GET"])` → `get_schemes`
- Line 33: `@gov_schemes_bp.route("/government-schemes/search", methods=["GET"])` → `search_schemes`
- Line 44: `@gov_schemes_bp.route("/government-schemes/<int:scheme_id>", methods=["GET"])` → `get_scheme`
- Line 54: `@gov_schemes_bp.route( "/government-schemes/<int:scheme_id>/eligibility", methods=["POST"] )` → `check_eligibility`
- Line 72: `@gov_schemes_bp.route("/government-schemes/<int:scheme_id>/assistance", methods=["GET"])` → `get_assistance`
- Line 82: `@gov_schemes_bp.route("/government-schemes/applications", methods=["POST"])` → `create_application`
- Line 105: `@gov_schemes_bp.route( "/government-schemes/applications/<int:application_id>/submit", methods=["POST"] )` → `submit_application`
- Line 117: `@gov_schemes_bp.route( "/government-schemes/applications/user/<int:user_id>", methods=["GET"] )` → `get_user_applications`
- Line 130: `@gov_schemes_bp.route( "/government-schemes/applications/<int:application_id>", methods=["GET"] )` → `get_application`
- Line 142: `@gov_schemes_bp.route("/government-schemes/deadlines", methods=["GET"])` → `get_upcoming_deadlines`
- Line 151: `@gov_schemes_bp.route("/government-schemes/<int:scheme_id>/reminders", methods=["POST"])` → `create_reminders`

backend/api/v1/insurance.py

- Line 52: `@insurance_bp.route('/policies', methods=['POST'])` → `create_policy`
- Line 112: `@insurance_bp.route('/policies', methods=['GET'])` → `get_policies`
- Line 146: `@insurance_bp.route('/policies/<int:policy_id>', methods=['GET'])` → `get_policy_details`
- Line 178: `@insurance_bp.route('/policies/<int:policy_id>/renew', methods=['POST'])` → `renew_policy`
- Line 241: `@insurance_bp.route('/policies/<int:policy_id>/cancel', methods=['POST'])` → `cancel_policy`
- Line 296: `@insurance_bp.route('/claims', methods=['POST'])` → `submit_claim`
- Line 365: `@insurance_bp.route('/claims', methods=['GET'])` → `get_claims`
- Line 398: `@insurance_bp.route('/statistics', methods=['GET'])` → `get_statistics`
- Line 420: `@insurance_bp.route('/admin/claims/<int:claim_id>/process', methods=['POST'])` → `process_claim`
- Line 482: `@insurance_bp.route('/admin/claims/<int:claim_id>/pay', methods=['POST'])` → `mark_as_paid`
- Line 528: `@insurance_bp.route('/admin/statistics', methods=['GET'])` → `get_admin_statistics`
- Line 552: `@insurance_bp.route('/policies/<int:policy_id>/risk-visibility', methods=['GET'])` → `get_risk_visibility`
- Line 578: `@insurance_bp.route('/policies/<int:policy_id>/premium-audit', methods=['GET'])` → `get_premium_audit`

- Line 603: `@insurance_bp.route('/recalculate-risk/<int:policy_id>', methods=['POST'])` → `manual_risk_recalc`

backend/api/v1/insurance_portal.py

- Line 9: `@insurance_v2_bp.route('/policies', methods=['POST'])` → `create_policy`
- Line 33: `@insurance_v2_bp.route('/my-policies', methods=['GET'])` → `list_policies`
- Line 42: `@insurance_v2_bp.route('/claims', methods=['POST'])` → `submit_claim`
- Line 60: `@insurance_v2_bp.route('/claims/<int:claim_id>/process', methods=['PATCH'])` → `adjudicate_claim`
- Line 78: `@insurance_v2_bp.route('/parametric/setup', methods=['POST'])` → `setup_parametric_trigger`
- Line 110: `@insurance_v2_bp.route('/parametric/auto-settlements', methods=['GET'])` → `get_auto_settlements`
- Line 126: `@insurance_v2_bp.route('/risk/yield-at-risk/<int:farm_id>', methods=['GET'])` → `get_farm_yield_risk`

backend/api/v1/irrigation.py

- Line 9: `@irrigation_bp.route('/zones', methods=['GET'])` → `list_zones`
- Line 23: `@irrigation_bp.route('/telemetry', methods=['POST'])` → `receive_telemetry`
- Line 42: `@irrigation_bp.route('/control/<int:zone_id>', methods=['PATCH'])` → `control_valve`
- Line 58: `@irrigation_bp.route('/analytics/<int:zone_id>', methods=['GET'])` → `get_analytics`
- Line 64: `@irrigation_bp.route('/quota/<int:farm_id>', methods=['GET'])` → `get_water_quota`

backend/api/v1/irrigation_v2.py

- Line 9: `@irrigation_v2_bp.route('/stress/logs', methods=['GET'])` → `get_stress_logs`
- Line 20: `@irrigation_v2_bp.route('/valves/status', methods=['GET'])` → `get_valve_states`
- Line 31: `@irrigation_v2_bp.route('/aquifer/health', methods=['GET'])` → `get_aquifer_health`

backend/api/v1/labor_management.py

- Line 9: `@labor_bp.route('/workers', methods=['GET'])` → `list_workers`
- Line 19: `@labor_bp.route('/clock-in', methods=['POST'])` → `clock_in`
- Line 28: `@labor_bp.route('/clock-out', methods=['POST'])` → `clock_out`
- Line 37: `@labor_bp.route('/harvest', methods=['POST'])` → `log_harvest`
- Line 44: `@labor_bp.route('/payroll/generate', methods=['POST'])` → `generate_payroll`
- Line 60: `@labor_bp.route('/payroll/history/<int:worker_id>', methods=['GET'])` → `payroll_history`

backend/api/v1/loan.py

- Line 14: `@loan_bp.route('/loan/process', methods=['POST'])` → `process_loan`

backend/api/v1/loan_repayment.py

- Line 9: `@loan_repayment_bp.route('/schedule/<int:loan_id>', methods=['GET'])` → `get_schedule`
- Line 19: `@loan_repayment_bp.route('/schedule/generate', methods=['POST'])` → `generate_schedule`
- Line 39: `@loan_repayment_bp.route('/pay', methods=['POST'])` → `make_payment`
- Line 59: `@loan_repayment_bp.route('/history/<int:loan_id>', methods=['GET'])` → `payment_history`
- Line 69: `@loan_repayment_bp.route('/risk/<int:loan_id>', methods=['GET'])` → `get_risk_score`

backend/api/v1/logistics.py

- Line 20: `@smart_freight_bp.route('/phyto-cert/issue', methods=['POST'])` → `issue_phyto_cert`
- Line 45: `@smart_freight_bp.route('/phyto-cert/verify/<cert_number>', methods=['GET'])` → `verify_phyto_cert`
- Line 65: `@smart_freight_bp.route('/escrow/lock', methods=['POST'])` → `lock_escrow`
- Line 97: `@smart_freight_bp.route('/gps/ping', methods=['POST'])` → `ingest_gps`
- Line 122: `@smart_freight_bp.route('/customs/arrive', methods=['POST'])` → `customs_arrive`

- Line 140: `@smart_freight_bp.route('/customs/<int:checkpoint_id>/clear', methods=['PATCH'])` → `clear_customs`
- Line 162: `@smart_freight_bp.route('/escrow/<int:route_id>', methods=['GET'])` → `get_escrow_status`

backend/api/v1/logistics_portal.py

- Line 8: `@logistics_portal_bp.route('/dispatch', methods=['POST'])` → `create_dispatch`
- Line 30: `@logistics_portal_bp.route('/routes/active', methods=['GET'])` → `get_active_routes`
- Line 40: `@logistics_portal_bp.route('/route/<int:route_id>/start', methods=['POST'])` → `start_route`
- Line 48: `@logistics_portal_bp.route('/route/<int:route_id>/complete', methods=['POST'])` → `complete_route`
- Line 57: `@logistics_portal_bp.route('/fleet/stats', methods=['GET'])` → `get_fleet_stats`
- Line 66: `@logistics_portal_bp.route('/drivers/available', methods=['GET'])` → `get_available_drivers`
- Line 75: `@logistics_portal_bp.route('/vehicles/idle', methods=['GET'])` → `get_idle_vehicles`

backend/api/v1/machinery.py

- Line 10: `@machinery_bp.route('/fleet', methods=['GET'])` → `list_fleet`
- Line 31: `@machinery_bp.route('/hours', methods=['POST'])` → `log_usage`
- Line 47: `@machinery_bp.route('/maintenance', methods=['POST'])` → `add_service_interval`
- Line 58: `@machinery_bp.route('/damage', methods=['POST'])` → `report_issue`
- Line 74: `@machinery_bp.route('/health/<int:equipment_id>', methods=['GET'])` → `get_health`
- Line 80: `@machinery_bp.route('/telemetry/wear', methods=['GET'])` → `get_wear_telemetry`
- Line 100: `@machinery_bp.route('/auto-repair/<int:equipment_id>', methods=['POST'])` → `trigger_wear_repair`

backend/api/v1/maintenance.py

- Line 11: `@maintenance_bp.route('/<int:equipment_id>/audit', methods=['POST'])` → `manual_audit`
- Line 30: `@maintenance_bp.route('/<int:equipment_id>/skill-check', methods=['POST'])` → `check_operator`
- Line 51: `@maintenance_bp.route('/reliability-report/<int:equipment_id>', methods=['GET'])` → `get_report`

backend/api/v1/maintenance_v2.py

- Line 9: `@maintenance_v2_bp.route('/asset/<int:asset_id>/predictions', methods=['GET'])` → `get_asset_predictions`
- Line 19: `@maintenance_v2_bp.route('/asset/<int:asset_id>/wear', methods=['GET'])` → `get_component_wear`
- Line 33: `@maintenance_v2_bp.route('/telemetry/ingest', methods=['POST'])` → `ingest_iot_telemetry`

backend/api/v1/market.py

- Line 21: `@market_bp.route('/market/commodities', methods=['GET'])` → `get_commodities`
- Line 33: `@market_bp.route('/market/prices', methods=['GET'])` → `get_prices`
- Line 48: `@market_bp.route('/market/forecast', methods=['POST', 'GET'])` → `forecast_prices`
- Line 76: `@market_bp.route('/market/optimize-profit', methods=['POST'])` → `optimize_net_profit`
- Line 105: `@market_bp.route('/market/heatmap', methods=['GET'])` → `get_price_heatmap`
- Line 125: `@market_bp.route('/market/watchlist', methods=['POST'])` → `add_to_watchlist`
- Line 126: `@market_bp.route('/market/alerts/subscribe', methods=['POST'])` → `add_to_watchlist`
- Line 169: `@market_bp.route('/market/alerts/check', methods=['POST', 'GET'])` → `check_price_alerts`
- Line 201: `@market_bp.route('/market/analyze', methods=['GET'])` → `analyze_market`
- Line 215: `@market_bp.route('/market/refresh', methods=['POST'])` → `force_refresh_prices`

backend/api/v1/model_versioning.py

- Line 23: `@model_versioning_bp.route('/registry', methods=['GET'])` → `get_registry`
- Line 43: `@model_versioning_bp.route('/<model_name>/versions', methods=['GET'])` → `list_model_versions`

- Line 78: `@model_versioning_bp.route('/<model_name>/active', methods=['GET'])` → `get_active_model_version`
- Line 117: `@model_versioning_bp.route('/<model_name>/<version>/metadata', methods=['GET'])` → `get_model_version_metadata`
- Line 155: `@model_versioning_bp.route('/<model_name>/set-active', methods=['POST'])` → `set_active_version`
- Line 213: `@model_versioning_bp.route('/<model_name>/rollback', methods=['POST'])` → `rollback_model_version`
- Line 279: `@model_versioning_bp.route('/<model_name>/performance', methods=['POST'])` → `update_model_performance`
- Line 344: `@model_versioning_bp.route('/<model_name>/<version>/delete', methods=['DELETE'])` → `delete_model_version`
- Line 383: `@model_versioning_bp.route('/<model_name>/health', methods=['GET'])` → `check_model_health`

`backend/api/v1/notifications.py`

- Line 7: `@notifications_bp.route('/', methods=['GET'])` → `get_notifications`
- Line 22: `@notifications_bp.route('/<int:notification_id>/read', methods=['POST'])` → `mark_read`
- Line 30: `@notifications_bp.route('/test', methods=['POST'])` → `test_notification`

`backend/api/v1/nutrient_api.py`

- Line 9: `@nutrient_api_bp.route('/nutrients/strategy', methods=['POST'])` → `generate_strategy`
- Line 26: `@nutrient_api_bp.route('/nutrients/micronutrients', methods=['GET'])` → `get_micronutrients`
- Line 46: `@nutrient_api_bp.route('/nutrients/history', methods=['GET'])` → `get_injection_history`

`backend/api/v1/pools.py`

- Line 17: `@pools_bp.route('', methods=['POST'])` → `create_pool`
- Line 46: `@pools_bp.route('', methods=['GET'])` → `list_pools`
- Line 73: `@pools_bp.route('/<int:pool_id>', methods=['GET'])` → `get_pool`
- Line 92: `@pools_bp.route('/<int:pool_id>/state', methods=['POST'])` → `transition_pool_state`
- Line 118: `@pools_bp.route('/<int:pool_id>/offer', methods=['POST'])` → `set_buyer_offer`
- Line 147: `@pools_bp.route('/<int:pool_id>/vote', methods=['POST'])` → `record_vote`
- Line 179: `@pools_bp.route('/<int:pool_id>/voting-status', methods=['GET'])` → `get_voting_status`
- Line 198: `@pools_bp.route('/<int:pool_id>/distribution', methods=['GET'])` → `get_distribution`
- Line 217: `@pools_bp.route('/<int:pool_id>/distribute', methods=['POST'])` → `execute_distribution`
- Line 236: `@pools_bp.route('/<int:pool_id>/resources', methods=['GET'])` → `get_shared_resources`
- Line 253: `@pools_bp.route('/<int:pool_id>/resources', methods=['POST'])` → `share_resource`
- Line 280: `@pools_bp.route('/roi-comparison', methods=['POST'])` → `compare_roi`

`backend/api/v1/processing.py`

- Line 9: `@processing_bp.route('/batches', methods=['GET'])` → `list_batches`
- Line 19: `@processing_bp.route('/batches', methods=['POST'])` → `create_batch`
- Line 41: `@processing_bp.route('/batches/<int:batch_id>/audit', methods=['POST'])` → `submit_audit`
- Line 62: `@processing_bp.route('/batches/<int:batch_id>/advance', methods=['PATCH'])` → `advance_batch`
- Line 80: `@processing_bp.route('/batches/<int:batch_id>/genealogy', methods=['GET'])` → `get_genealogy`
- Line 93: `@processing_bp.route('/batches/<int:batch_id>/spectral-scan', methods=['POST'])` → `submit_spectral_scan`
- Line 115: `@processing_bp.route('/batches/<int:batch_id>/valuation', methods=['GET'])` → `get_batch_valuation`

`backend/api/v1/procurement.py`

- Line 9: `@procurement_bp.route('/items', methods=['GET'])` → `list_items`

- Line 30: `@procurement_bp.route('/orders', methods=['POST'])` → `place_order`
- Line 60: `@procurement_bp.route('/orders/<int:order_id>', methods=['GET'])` → `get_order`

backend/api/v1/questions.py

- Line 8: `@questions_bp.route('/', methods=['GET'])` → `get_questions`
- Line 20: `@questions_bp.route('/<int:question_id>', methods=['GET'])` → `get_question_detail`
- Line 38: `@questions_bp.route('/', methods=['POST'])` → `create_question`
- Line 60: `@questions_bp.route('/<int:question_id>/vote', methods=['POST'])` → `vote_question`

backend/api/v1/risk.py

- Line 26: `@risk_bp.route('/score', methods=['GET'])` → `get_risk_score`
- Line 73: `@risk_bp.route('/score/calculate', methods=['POST'])` → `trigger_score_calculation`
- Line 105: `@risk_bp.route('/history', methods=['GET'])` → `get_score_history`
- Line 147: `@risk_bp.route('/trend', methods=['GET'])` → `get_score_trend`
- Line 189: `@risk_bp.route('/insights', methods=['GET'])` → `get_improvement_insights`
- Line 234: `@risk_bp.route('/premium/estimate', methods=['POST'])` → `estimate_premium`
- Line 294: `@risk_bp.route('/categories', methods=['GET'])` → `get_risk_categories`
- Line 355: `@risk_bp.route('/factors', methods=['GET'])` → `get_risk_factors`
- Line 443: `@risk_bp.route('/health', methods=['GET'])` → `health_check`

backend/api/v1/schemes.py

- Line 7: `@schemes_bp.route('/schemes', methods=['GET'])` → `get_schemes`
- Line 23: `@schemes_bp.route('/schemes/refresh', methods=['POST'])` → `refresh_schemes`

backend/api/v1/soil_analysis.py

- Line 8: `@soil_bp.route('/test', methods=['POST'])` → `submit_test`
- Line 25: `@soil_bp.route('/history/<int:farm_id>', methods=['GET'])` → `get_history`
- Line 34: `@soil_bp.route('/recommendations/<int:test_id>', methods=['GET'])` → `get_recommendations`
- Line 43: `@soil_bp.route('/apply', methods=['POST'])` → `log_application`
- Line 53: `@soil_bp.route('/flux-map/<int:farm_id>', methods=['GET'])` → `get_flux_map`

backend/api/v1/soil_analytics.py

- Line 8: `@soil_analytics_bp.route("/health-score/<int:farm_id>", methods=["GET"])` → `get_health_score`
- Line 29: `@soil_analytics_bp.route("/trends/<int:farm_id>", methods=["GET"])` → `get_trends`
- Line 39: `@soil_analytics_bp.route("/comparison/<int:farm_id>", methods=["GET"])` → `get_comparison`
- Line 50: `@soil_analytics_bp.route("/report/<int:farm_id>", methods=["GET"])` → `get_comprehensive_report`
- Line 61: `@soil_analytics_bp.route("/summary/<int:farm_id>", methods=["GET"])` → `get_quality_summary`

backend/api/v1/spatial_yield.py

- Line 8: `@spatial_yield_bp.route('/grids', methods=['GET'])` → `get_spatial_grids`
- Line 29: `@spatial_yield_bp.route('/grids/<string:region_id>/simulate-telemetry', methods=['POST'])` → `simulate_telemetry`

backend/api/v1/supply_chain.py

- Line 9: `@supply_chain_bp.route('/contract/init', methods=['POST'])` → `init_smart_contract`
- Line 28: `@supply_chain_bp.route('/contract/<int:order_id>/telemetry', methods=['POST'])` → `update_order_telemetry`
- Line 44: `@supply_chain_bp.route('/orders/active', methods=['GET'])` → `get_active_orders`

backend/api/v1/sustainability.py

- Line 9: `@sustainability_bp.route('/report/farm/<int:farm_id>', methods=['GET'])` → `get_farm_sustainability_report`
- Line 30: `@sustainability_bp.route('/report/batch/<int:batch_id>', methods=['GET'])` → `get_batch_certification`
- Line 47: `@sustainability_bp.route('/audit', methods=['POST'])` → `trigger_manual_audit`

backend/api/v1/tasks.py

- Line 7: `@tasks_bp.route('/tasks/<task_id>', methods=['GET'])` → `get_task_status`

backend/api/v1/traceability.py

- Line 21: `@traceability_bp.route('/generate-qr/<batch_id>', methods=['GET'])` → `generate_qr_sticker`
- Line 22: `@traceability_bp.route('/qr/<batch_id>', methods=['GET'])` → `generate_qr_sticker`
- Line 41: `@traceability_bp.route('/batches/<batch_id>', methods=['GET'])` → `get_batch`
- Line 42: `@traceability_bp.route('/batch/<batch_id>', methods=['GET'])` → `get_batch`
- Line 61: `@traceability_bp.route('/verify/<batch_id>', methods=['GET'])` → `verify_batch_cryptography`
- Line 83: `@traceability_bp.route('/log-event', methods=['POST'])` → `log_lifecycle_event`
- Line 126: `@traceability_bp.route('/batches', methods=['POST'])` → `create_batch`
- Line 127: `@traceability_bp.route('/create-batch', methods=['POST'])` → `create_batch`

backend/api/v1/translations.py

- Line 138: `@translation_bp.route("/translate", methods=["POST"])` → `translate`
- Line 198: `@translation_bp.route("/languages", methods=["GET"])` → `get_languages`
- Line 210: `@content_bp.route("/blogs", methods=["GET"])` → `get_localized_blogs`
- Line 230: `@content_bp.route("/crops", methods=["GET"])` → `get_localized_crops`
- Line 250: `@content_bp.route("/resources", methods=["GET"])` → `get_localized_resources`

backend/api/v1/transparency.py

- Line 9: `@transparency_bp.route('/trace/<int:stock_item_id>', methods=['GET'])` → `get_traceability`
- Line 20: `@transparency_bp.route('/review', methods=['POST'])` → `submit_review`
- Line 43: `@transparency_bp.route('/pricing/audit/<int:stock_item_id>', methods=['GET'])` → `get_pricing_history`

backend/api/v1/vaults.py

- Line 25: `@vaults_bp.route('/', methods=['POST'])` → `create_vault`
- Line 62: `@vaults_bp.route('/', methods=['GET'])` → `list_vaults`
- Line 79: `@vaults_bp.route('/<vault_id>', methods=['GET'])` → `get_vault`
- Line 100: `@vaults_bp.route('/<vault_id>/deposit', methods=['POST'])` → `deposit`
- Line 151: `@vaults_bp.route('/<vault_id>/withdraw', methods=['POST'])` → `withdraw`
- Line 202: `@vaults_bp.route('/transfer', methods=['POST'])` → `transfer`
- Line 253: `@vaults_bp.route('/<vault_id>/ledger', methods=['GET'])` → `get_ledger_history`
- Line 299: `@vaults_bp.route('/<vault_id>/audit', methods=['GET'])` → `audit_vault`
- Line 337: `@vaults_bp.route('/<vault_id>/revalue', methods=['POST'])` → `revalue_positions`
- Line 379: `@vaults_bp.route('/<vault_id>/fx-exposure', methods=['GET'])` → `get_fx_exposure`
- Line 403: `@vaults_bp.route('/<vault_id>/fx-history', methods=['GET'])` → `get_fx_valuation_history`
- Line 445: `@vaults_bp.route('/<vault_id>/lock', methods=['POST'])` → `lock_vault`
- Line 468: `@vaults_bp.route('/<vault_id>/unlock', methods=['POST'])` → `unlock_vault`
- Line 490: `@vaults_bp.route('/ledger/transaction/<transaction_id>', methods=['GET'])` → `audit_transaction`

- Line 514: `@vaults_bp.route('/ledger/transaction/<int:transaction_id>/reverse', methods=['POST'])` → `reverse_transaction`
- Line 544: `@vaults_bp.route('/ledger/trial-balance', methods=['GET'])` → `get_trial_balance`
- Line 569: `@vaults_bp.route('/fx/rates', methods=['GET'])` → `get_fx_rates`
- Line 584: `@vaults_bp.route('/fx/rates', methods=['POST'])` → `update_fx_rates`
- Line 619: `@vaults_bp.route('/fx/rate/<from_currency>/<to_currency>', methods=['GET'])` → `get_fx_rate`
- Line 646: `@vaults_bp.route('/fx/history/<from_currency>/<to_currency>', methods=['GET'])` → `get_rate_history`
- Line 680: `@vaults_bp.route('/fx/calculate-delta', methods=['POST'])` → `calculate_fx_delta`

`backend/api/v1/vendors.py`

- Line 9: `@vendors_bp.route('/orders', methods=['GET'])` → `get_vendor_orders`
- Line 22: `@vendors_bp.route('/orders/<int:order_id>/update', methods=['PATCH'])` → `update_order_status`
- Line 41: `@vendors_bp.route('/profile', methods=['POST'])` → `setup_vendor`

`backend/api/v1/virulence.py`

- Line 8: `@virulence_bp.route('/strains/active', methods=['GET'])` → `active_pathogen_strains`
- Line 18: `@virulence_bp.route('/combat-simulation/run', methods=['POST'])` → `execute_deterministic_combat_step`
- Line 35: `@virulence_bp.route('/combat-logs', methods=['GET'])` → `view_virulence_combat_logs`

`backend/api/v1/warehouse.py`

- Line 9: `@warehouse_bp.route('/stock', methods=['GET'])` → `list_stock`
- Line 23: `@warehouse_bp.route('/stock/in', methods=['POST'])` → `stock_in`
- Line 48: `@warehouse_bp.route('/stock/out', methods=['POST'])` → `stock_out`
- Line 73: `@warehouse_bp.route('/reconcile', methods=['POST'])` → `reconcile`
- Line 104: `@warehouse_bp.route('/expiring', methods=['GET'])` → `get_expiring`

`backend/api/v1/weather.py`

- Line 8: `@weather_bp.route('/banner', methods=['GET'])` → `get_weather_banner`
- Line 47: `@weather_bp.route('/current', methods=['GET'])` → `get_weather`
- Line 62: `@weather_bp.route('/subscribe', methods=['POST'])` → `subscribe`

`backend/auth/decorators.py`

- Line 15: `@app.route('/protected')` → `protected_route`
- Line 53: `@app.route('/admin')` → `admin_route`
- Line 59: `@app.route('/dashboard')` → `dashboard`
- Line 97: `@app.route('/public')` → `public_route`

`backend/auth/routes.py`

- Line 25: `@auth_bp.route('/register', methods=['POST'])` → `handler name not found`
- Line 159: `@auth_bp.route('/login', methods=['POST'])` → `handler name not found`
- Line 271: `@auth_bp.route('/refresh', methods=['POST'])` → `handler name not found`
- Line 336: `@auth_bp.route('/logout', methods=['POST'])` → `handler name not found`
- Line 374: `@auth_bp.route('/me', methods=['GET'])` → `handler name not found`
- Line 409: `@auth_bp.route('/validate', methods=['GET'])` → `handler name not found`

`backend/docs/swagger.py`

- Line 81: `@swagger_bp.route("/openapi.json", methods=["GET"])` → `openapi_json`
- Line 86: `@swagger_bp.route("/", methods=["GET"])` → `swagger_ui`

backend/monitoring/routes.py

- Line 7: `@health_bp.route('/health', methods=['GET'])` → `liveness`
- Line 18: `@health_bp.route('/health/ready', methods=['GET'])` → `readiness`
- Line 36: `@health_bp.route('/health/live', methods=['GET'])` → `live`
- Line 42: `@health_bp.route('/metrics', methods=['GET'])` → `prometheus_metrics`
- Line 52: `@health_bp.route('/health/details', methods=['GET'])` → `health_details`

crop_recommendation/app.py

- Line 54: `@app.route('/')` → `home`
- Line 58: `@app.route('/predict', methods=['GET', 'POST']) # Allow GET requests` → `predict`
- Line 109: `@app.route('/download_report', methods=['POST'])` → `download_report`

crop_recommendation/routes.py

- Line 105: `@crop_bp.route('/')` → `home`
- Line 109: `@crop_bp.route('/predict', methods=['GET', 'POST']) # Allow GET requests` → `predict`
- Line 187: `@crop_bp.route('/download_report', methods=['POST'])` → `download_report`

server/Routes/irrigation_routes.py

- Line 6: `@irrigation_bp.route('/api/irrigation-schedule', methods=['POST'])` → `get_schedule`

server/Routes/rotation_routes.py

- Line 16: `@rotation_bp.route('/api/analyze-rotation', methods=['POST'])` → `analyze`

spatial_analytics/routes.py

- Line 13: `@spatial_bp.route('/fields', methods=['POST'])` → `create_field`
- Line 54: `@spatial_bp.route('/fields', methods=['GET'])` → `get_fields`
- Line 63: `@spatial_bp.route('/analyze/<int:field_id>', methods=['POST'])` → `analyze_field`

tests/conftest.py

- Line 30: `@flask_app.route('/api/firebase-config')` → `_firebase_config`
- Line 47: `@flask_app.route('/api/crop/predict-async', methods=['POST'])` → `_predict_crop_async`
- Line 57: `@flask_app.route('/api/loan/process-async', methods=['POST'])` → `_process_loan_async`
- Line 71: `@flask_app.route('/generate-loan-report', methods=['POST'])` → `_generate_loan_report`

09 Application Workflows

Application Workflows

Crop recommendation (synchronous)

1. Farmer opens crop page and enters soil/climate values.
2. Browser posts form values to `/crop/predict`.
3. Crop route requires N, P, K, temperature, humidity, pH and rainfall; it converts and range-checks numeric values.
4. Route uses model and label encoder loaded from model-versioning manager or legacy pickle files.
5. It predicts crop label and renders result template with input parameters/model version. Missing model returns unavailable error; invalid values return form with error.

Mermaid diagram source

```
sequenceDiagram
 participant F as Farmer
 participant P as Crop page
 participant R as /crop/predict
 participant M as Loaded crop model
 F->>P: enter soil/weather data
 P->>R: POST form
 R->>R: validate required fields and ranges
 R->>M: predict ordered features
 M-->>R: encoded or text crop label
 R->>R: decode label if needed
 R-->>P: render result or validation error
```

Yield prediction

GET `/yield/` asks the initialized YieldPredictionService for valid crop/season/state choices and renders `yield_prediction.html`. At startup the service looks for a joblib bundle containing model, encoders, scaler and feature schema. POST `/yield/predict` validates numbers/categories, encodes categories, arranges feature columns, scales as trained and predicts; missing bundle returns 503, invalid form returns 400 and unexpected error logs then shows generic failure.

Disease image workflow

`disease.html`/`disease.js` reads selected photo, creates a resized JPEG data URL and POSTs to `/api/v1/ai-disease/predict-trained`. Server verifies base64, size and supported crop, converts image to RGB 256x256 tensor, applies same [0,1] preprocessing used by its notebook, gets CNN logits, applies softmax and returns top prediction/alternatives. UI now gates low/ambiguous raw scores as uncertain; it does not certify disease. Authenticated `/analyze` is a separate service path that may save a diagnosis; incident reporting is another API.

Fertilizer, irrigation and rotation

Fertilizer: authenticated role-restricted JSON POST to `/api/recommend/fertilizer` → parse soil pH/crop/stage/weather → `agri_utils.recommend_fertilizer` → JSON. Irrigation and rotation use separate Blueprints under server Routes. Their input/output contracts are endpoint specific; there is no assumed automatic pipeline from crop recommendation.

Authentication

Registration/login forms send credentials to auth routes; service checks account and hash, persistence writes user metadata and route returns result/tokens. Protected calls are decorated; failed/missing token gets an auth error. Role checks further limit actions. Frontend token storage varies and should be traced in `auth.js`.

Async task

Crop task route authenticates/authorizes, checks required fields, enqueues Celery task, returns 202/task ID. The browser polls status route. Worker computes and state is surfaced as pending, processing, completed/result or failed. Redis and worker availability are required.

Market prices and other services

Price tracker Blueprint includes market/state lookup, forecast, profit optimization, heatmap and alert subscription. Trace each specific page to route and service before assuming data freshness, persistence or external provider. API v1 includes many additional module families; route appendix is a navigation index, not a claim that all are production ready.

10 Setup and Installation

Setup and Installation

Requirements and install

README documents Python virtual environment and requirements installation. On Windows PowerShell from project root:

powershell code

```
py -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
if (-not (Test-Path .env)) { Copy-Item .env.example .env }
```

Edit `.env` locally using only credentials for desired services. Never share or commit it. README says `GEMINI_API_KEY` is required by normal app startup; testing mode supplies a test placeholder.

Run the main app

powershell code

```
python app.py
```

Open `http://127.0.0.1:5000/`; health route is documented in README. `app.py` starts Socket.IO. The main frontend is served by Flask, so a separate frontend build is not needed.

Optional services

powershell code

```
docker compose up --build
```

Compose defines Redis, MailHog, Flask and Celery worker. Ensure `.env` is configured. Optional Node service: `npm install`, then `npm start` or `npm run dev`; this starts `server.js`, not Flask.

Tests

`pytest.ini` configures pytest paths including `tests/` and backend suites. Run `python -m pytest` from root. This is a repository command reference; tests were not run as part of documentation generation. Flake8 is listed in requirements; use `python -m flake8` if installed.

Production

No verified frontend build command or production hosting provider was found. README recommends a WSGI server such as Gunicorn instead of Flask development server. Dockerfile uses Python 3.10 slim. Production requires strong secret values, HTTPS, restricted CORS, real mail/storage/database, migrations, model assets, observability, and independently managed Redis/worker.

Environment variables (names only)

`FLASK_ENV`, `DEBUG`, `SECRET_KEY`, `HOST`, `PORT`, `CORS_ORIGINS`, `DATABASE_URL`, `DEV_DATABASE_URL`, `DATABASE_ECHO`, `DATABASE_POOL_SIZE`, `DATABASE_MAX_OVERFLOW`, `DATABASE_POOL_RECYCLE`, `JWT_SECRET`, `JWT_ALGORITHM`, `ACCESS_TOKEN_EXPIRE_MINUTES`, `REFRESH_TOKEN_EXPIRE_DAYS`, `PASSWORD_HASH_ROUNDS`, `ENABLE_AI_DISEASE`, `ENABLE_PREDICTION_COMPARISON`, `ENABLE_SEEDLING_CLASSIFICATION`, `ENABLE_WEATHER_ADVISORY`, `ENABLE_SPATIAL_ANALYTICS`, `ENABLE_CROP_RECOMMENDATION`, `ENABLE_CARBON_PORTAL`, `ENABLE_VOICE_INPUT`, `ENABLE_OFFLINE_CACHE`, `ENABLE_RATE_LIMITER`, `ENABLE_MULTILINGUAL_SUPPORT`, `GEMINI_API_KEY`, `GEMINI_MODEL_ID`, `OPENAI_API_KEY`, `DISEASE_CONFIDENCE_THRESHOLD`, `MAX_IMAGE_UPLOAD_SIZE_MB`, `BOT_DEFAULT_LANGUAGE`, `TWILIO_ACCOUNT_SID`, `TWILIO_AUTH_TOKEN`, `TWILIO_PHONE_NUMBER`, `TWILIO_WHATSAPP_NUMBER`, `META_WHATSAPP_TOKEN`, `META_WHATSAPP_PHONE_NUMBER_ID`, `META_WHATSAPP_VERIFY_TOKEN`, `BHASHINI_API_KEY`, `BHASHINI_USER_ID`, `BHASHINI_PIPELINE_ID`, `GOOGLE_TRANSLATION_API_KEY`, `WEATHER_API_KEY`, `WEATHER_API_URL`, `FX_API_URL`, `REDIS_URL`, `CACHE_TYPE`, `CACHE_DEFAULT_TIMEOUT`, `CELERY_BROKER_URL`, `CELERY_RESULT_BACKEND`, `STORAGE_TYPE`, `MAX_CONTENT_LENGTH`, `S3_BUCKET`, `S3_ACCESS_KEY`, `S3_SECRET_KEY`, `S3_REGION`, `S3_ENDPOINT_URL`, `FIREBASE_API_KEY`, `FIREBASE_AUTH_DOMAIN`, `FIREBASE_PROJECT_ID`, `FIREBASE_STORAGE_BUCKET`, `FIREBASE_MESSAGING_SENDER_ID`, `FIREBASE_APP_ID`, `FIREBASE_MEASUREMENT_ID`, `MAIL_SERVER`, `MAIL_PORT`, `MAIL_USE_TLS`, `MAIL_USERNAME`, `MAIL_PASSWORD`, `MAIL_DEFAULT_SENDER`

Names are from `.env.example`; not every variable is used by every run. Never copy actual `.env` values into documentation.

11 Security and Code Audit

Security and Code Audit

Scope

This is a source-based review, not a penetration test, dependency scan or deployment configuration review. Severity describes code/config risk, conditional on deployment. No secrets or `.env` values are reproduced.

Findings

Severity	Evidence	Risk	Recommendation
High, conditional	`app.py` refuses normal startup without `GEMINI_API_KEY` even when an operator may not use Gemini	Unrelated app features become unavailable due to optional provider credential	Initialize provider lazily; disable only AI endpoint with explicit readiness status
High, deployment	`auth_utils.py` uses a fallback JWT secret if `JWT_SECRET` absent	Known/default signing key could allow forged tokens if deployed unchanged	Fail startup in production when missing; rotate keys and validate expiry/issuer/audience
Medium	`backend/config/settings.py` declares `ENABLE_*` values but no code consumers found	Operators may believe features are controlled when these flags do nothing	Wire each flag to routes/UI or remove misleading settings; test flag-off behavior
Medium	Multiple token systems (`auth_utils`, Flask-JWT-Extended, service auth)	Inconsistent validation, refresh, role and revocation behavior	Standardize auth contract and audit route-by-route protection
Medium	Broad exception handlers in some routes serialize exception text	Internal diagnostics may leak through API responses	Return stable generic error IDs; log details server-side
Medium	Direct `db.create_all()` at app startup plus migration tree	Schema changes can be unmanaged or deployment startup may race	Run migrations as release step; avoid implicit production schema changes
Medium, agronomic safety	Model scores may be uncalibrated; frontend previously labeled scores >=40% as disease with static severity/treatment	False positive could prompt unnecessary chemical use	UI now marks under-70% or close competing scores uncertain; calibrate using independent field data; never infer severity from class label
Low/Medium, varies by route	Many feature modules have different decorators and input validation patterns	Authorization or validation gaps possible on a route not guarded consistently	Build endpoint-level auth/validation matrix; add integration tests for every write route

Controls visible in source

Werkzeug password hashing is used in User; ORM queries reduce raw SQL construction risk; CORS and Flask-Limiter are configured; input validation/sanitization utilities, audit middleware and environment-based secrets exist. Coverage and production configuration must still be verified.

Checks not established

No dependency vulnerability scanner was run. XSS/CSRF/injection safety cannot be certified globally from a quick scan. Cookie attributes, file ownership checks, upload scanning, CSP, secure headers, production debug status, all-route authorization and external API TLS handling require endpoint/deployment-specific review.

12 Deployment and DevOps

Deployment and DevOps

Docker

`Dockerfile` starts from `python:3.10-slim`, installs gcc, installs `requirements.txt`, copies repository and runs `python app.py` on port 5000. It uses Flask development entry point; for production replace with a production WSGI/Socket.IO-compatible server configuration.

`docker-compose.yml` defines Redis 7, MailHog, Flask backend, Celery worker, persistent Redis and uploads volumes. The worker command is `celery -A backend.celery_app worker --loglevel=info`. Compose mounts source into `/app`, appropriate for development but not an immutable production image.

CI/CD and hosting

`.github/workflows/` contains issue/PR automation such as labels/assignee actions. A deployment workflow/hosting provider was not found in the scanned files. Do not infer CI build, release or production domain.

Deployment checklist

- Choose and configure production database; apply migrations intentionally.
- Provide strong `SECRET_KEY`, `JWT_SECRET`, provider keys and SMTP/storage credentials through secret manager.
- Use HTTPS, restrictive CORS, secure cookies, rate limits and non-debug server settings.
- Include model bundles/class metadata in image/artifact storage and verify readiness.
- Run Redis, Celery worker and Socket.IO support if required by enabled journeys.
- Persist uploads outside ephemeral container and enforce access controls.
- Add health checks, logs, error monitoring, backups and rollback plan.
- Confirm normal startup does not require unused optional integrations.

13 Troubleshooting

Troubleshooting

Symptom	Likely cause	What to check
App startup reports missing `GEMINI_API_KEY`	Startup currently requires Gemini outside tests	Set a key in local `.env`, or adjust optional-service initialization for a future source change
Crop prediction unavailable	Crop checkpoint/label encoder missing or version manager cannot load	Verify `crop_recommendation/model/` or configured model registry artifacts and logs
Yield page returns 503	`crop_yield_model_bundle.joblib` absent/invalid	Follow message: run notebook export cells; ensure bundle has model, encoders, scaler and feature schema; restart app
Disease image endpoint returns 503	Plant disease checkpoint or class metadata missing	Confirm `Disease prediction/plant_disease_model.pth` and `plant_disease_classes.json`
Disease result marked uncertain	Raw model score below conservative UI gate or top candidates too close	Retake clear close-up in natural light; do not use result alone for treatment
Async tasks stay pending	Redis/broker or Celery worker is not running/configured	Inspect Redis URL, worker logs, task ID and result backend
Email verification not delivered	SMTP settings missing or MailHog/SMTP unavailable	Check configured mail server/port and local MailHog UI if using Compose
API returns 401/403	Missing/invalid token or insufficient role; auth systems vary	Check endpoint decorator, token issuer and Authorization format; never paste token into chat
DB migration/schema error	Database URL mismatch or unapplied migration	Check active `DATABASE_URL`, migration history, backups; apply migrations deliberately
External weather/AI/translation call fails	Key, URL, network or quota missing	Check variable names and service health without exposing credentials
Static page 404	Wrong route/path or missing asset	Check root static catch-all, filename case, script/style references and browser network tab

14 Interview Preparation

Interview Preparation

Two-minute introduction

“AgriTech is a multi-page smart agriculture web application. Its main backend is Flask, with feature Blueprints and browser pages written in HTML, CSS and JavaScript. It offers crop recommendation, fertilizer guidance, yield prediction, plant disease image classification, and broader farmer services such as weather, irrigation, markets and community workflows. SQLAlchemy stores structured records, and Flask-Migrate manages schema changes. Selected longer tasks use Celery and Redis. I focus on explaining verified flows: a browser request is validated by Flask, routed to a service, processed by a model or database, then returned as HTML or JSON. Some modules require model artifacts or external credentials, and the disease score is not a calibrated probability, so I communicate those limits clearly.”

Five-minute structure

1. Problem and users (45 sec).
2. Architecture: browser → Flask route → service/model/database → response (60 sec).
3. Demonstrate one crop or yield request with validation and model preprocessing (90 sec).
4. Explain database/auth/async choices (60 sec).
5. Show tests actually run, model evaluation evidence, limitations and next steps (60 sec).

50 project-grounded questions and answer guides

1. **What problem does AgriTech address?** It collects selected crop guidance and farmer services into one web entry point; it does not replace local agronomists.
2. **What is the main backend entry point?** `app.py`, which loads configuration/extensions and registers direct routes and Blueprints.
3. **What is a Blueprint?** A Flask grouping for related routes; registration can add a URL prefix.
4. **Is the frontend React?** No main React app was verified; the main UI is multi-page HTML/CSS/JS.
5. **What does GET mean?** Ask the server to retrieve a page or information.
6. **What does POST mean?** Send data or request an action; endpoint semantics are defined by handler.
7. **What is an API?** A structured contract for browser and server messages.
8. **How does crop prediction work?** Form POST, required/range checks, ordered feature DataFrame, model inference, label decode and result template.
9. **What inputs does crop prediction accept?** N, P, K, temperature, humidity, pH and rainfall in the crop route.
10. **Why use a label encoder?** To map text crop labels to integers during training and back for output.
11. **What is model inference?** Using a trained model to estimate an output for new input.
12. **What is yield prediction’s key constraint?** Feature encoding/scaling and order must match training; bundle is required.
13. **What happens if yield bundle is missing?** Service fails initialization; yield route returns 503 with readiness info.
14. **How does disease classification work?** Base64 image decoded, resized, RGB tensor made in [0,1], CNN returns logits and softmax scores.
15. **Is 60% disease confidence a 60% real-world probability?** No. It is an uncalibrated model score unless calibration has been separately validated.
16. **Does the classifier measure disease severity?** No, class label does not quantify severity; UI now calls it typical disease impact or not assessed.
17. **Why show alternatives?** They reveal competing labels and can help identify uncertain outputs.
18. **What is SQLAlchemy?** ORM that maps Python model classes to relational tables and queries.
19. **What is a migration?** Versioned change to database schema.
20. **What is a transaction?** A group of database writes committed together or rolled back on failure.

21. **Which database is used?** Development defaults to SQLite; environment may configure another DB.
22. **What is a foreign key?** A column that references a row in another table.
23. **What is authentication?** Verifying a user identity.
24. **What is authorization?** Checking which action that identity may perform.
25. **How are passwords stored?** Werkzeug password hashes are stored, not plaintext.
26. **What is JWT?** A signed token carrying claims such as identity/expiry; server verifies it.
27. **Does every route require login?** No; inspect decorators per endpoint. Some routes are public and auth systems vary.
28. **What is RBAC?** Role-based access control, where role grants or denies actions.
29. **What is Celery used for?** Background task processing outside the web request.
30. **Why Redis?** Configured as Celery broker/result backend and cache option.
31. **What is polling?** Browser repeatedly asks a task status endpoint using a task ID.
32. **What is Socket.IO?** Realtime event transport used where server/client handlers are wired.
33. **What is CORS?** Browser rule controlling which origins may call an API cross-origin.
34. **Why validate on the server?** Browser checks can be bypassed, so server is the trust boundary.
35. **What does rate limiting do?** Caps request frequency to reduce abuse/load.
36. **What is the purpose of audit logs?** Record security-sensitive actions for traceability.
37. **Where are secrets stored?** Environment configuration; never commit or expose actual values.
38. **What is the major startup configuration caveat?** Normal app startup requires Gemini key currently.
39. **Are feature flags effective?** Flags are defined, but no consumers found; presently they appear non-operative.
40. **What makes an integration optional?** Its provider credentials/network and code path must be configured.
41. **What tests exist?** Pytest suites and security-related tests; verify exact coverage before claims.
42. **How evaluate classification?** Independent holdout, per-class precision/recall/F1, confusion matrix and field generalization.
43. **How evaluate regression?** MAE/RMSE/R^2 with target units, baselines and subgroup error.
44. **How protect file uploads?** Check type/size/content, safe names, storage isolation, ownership authorization and path safety.
45. **What is XSS?** Attacker-controlled content running as script in another user's browser; escape/render safely.
46. **What is CSRF?** A browser tricked into sending an authenticated state-changing request; use SameSite/CSRF defenses.
47. **What is the known JWT concern?** Legacy helper has a fallback secret; production must require a unique secret.
48. **What do Docker services run?** Flask app, Redis, MailHog and Celery worker.
49. **Is there a frontend build step?** No main frontend build script detected; npm scripts launch optional Node server.
50. **Name a technical limitation.** Large modular repository includes partially integrated modules; demonstrate route→service→DB/model path.
51. **What would you improve first?** Standardize auth, make flags effective, remove optional-key startup coupling, add calibrated field-based ML evaluation and route integration tests.

Interview honesty rule

Say "implemented in source" only if you can point to the page, active route, service/model, persistence path and a successful demo. Say "needs configuration" for external services/model artifacts. Say "not verified" for production status or unmeasured model quality.